Linköping University | Department of Computer and Information Science
Master's thesis, 30 ECTS | Datateknik
2023 | LIU-IDA/LITH-EX-A–23/054–SE

Analyzing the Performance of Linux Networking Approaches for Packet Processing

– A Comparative Analysis of DPDK, io_uring and the standard Linux network stack

---

En jämförelsestudie av nätverkspaketprocessning i DPDK, io_uring och Linux standardnätverksstack

Isak Toivanen
Maximilian Vorbrodt

Supervisor : Markus Fritzsche
Examiner : Jendrik Seipp

External supervisor : Emil Berg, Johnny Jonsson and Elmedin Zildzic

LINKÖPING UNIVERSITY

Linköpings universitet
SE–581 83 Linköping
+46 13 28 10 00 , www.liu.se

Upphovsrätt

Detta dokument hålls tillgängligt på Internet - eller dess framtida ersättare - under 25 år från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida `http://www.ep.liu.se/`.

Copyright

The publishers will keep this document online on the Internet - or its possible replacement - for a period of 25 years starting from the date of publication barring exceptional circumstances.

The online availability of the document implies permanent permission for anyone to read, to download, or to print out single copies for his/hers own use and to use it unchanged for non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional upon the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its www home page: `http://www.ep.liu.se/`.

© Isak Toivanen
Maximilian Vorbrodt

Abstract

This thesis investigates the network performance of DPDK, io_uring, and the standard Linux network stack in terms of latency, packet loss, throughput, and packet rate. DPDK is widely adopted in the telecommunication industry and has been for several years. io_uring is a new Linux system call interface, as of this writing, used for asynchronous I/O operations and could potentially offer better network performance than the standard Linux network stack. The analysis involves benchmarking of DPDK, io_uring and Linux network stack both with a continuous flow of data traffic and bursty traffic to see how the applications are able to handle different kinds of data traffic conditions. The Linux kernel network parameters were tuned in order to investigate what kind of performance increase can be achieved for an application using io_uring to gain a better understanding of how io_uring compares to DPDK. From the analysis, it was concluded that DPDK had the best overall performance when considering packet loss, throughput, latency, and packet rate. io_uring performed better than the Linux network stack in terms of throughput, packet loss, and packet rate but was not considerably better in terms of latency. When comparing the latency of bursty traffic for DPDK, io_uring, and the standard Linux network stack, it was shown that the kernel-based alternatives were better able at handling traffic over their maximum performance than DPDK, thus it may offer better scalability. After tuning the Linux kernel network parameters it was concluded that tuning may impact the performance network metrics of an application using io_uring. We conclude that io_uring shows some promising results compared to the standard Linux network stack, but is not yet able to compare with DPDK in network performance. The development of io_uring is ongoing and it may improve further in the future.

Acknowledgments

We are deeply thankful to our supervisors from Ericsson: Emil Berg, Johnny Jonsson, and Elmedin Zildzic. Their support, guidance, and feedback throughout the process have been a great help in shaping this thesis. Their will to share knowledge has had a big impact on the quality of the thesis.

We would also like to show appreciation to our supervisor from Linköping University, Markus Fritzsche. His knowledge, feedback, and dedication have shaped the direction and structure of this thesis in a positive way.

In addition, we would also like to express our gratitude to Jendrik Seipp, our examiner from Linköping University, for taking the time to review our thesis and help with scoping and setting reasonable goals.

Maximilian Vorbrodt & Isak Toivanen
Linköping, June 2023

Contents

Abstract iii

Acknowledgments iv

Contents v

1 Introduction 1
1.1 Motivation . . . 1
1.2 Aim . . . 2
1.3 Research Questions . . . 2
1.4 Approach . . . 2
1.4.1 Research Methods . . . 2
1.4.2 Delimitations . . . 3
1.5 Contributions . . . 3

2 Background 4
2.1 Network Metrics . . . 4
2.1.1 Throughput and Latency Trade-off . . . 5
2.2 Memory Management in Linux . . . 5
2.2.1 Virtual Memory . . . 5
2.2.2 Memory Pages . . . 5
2.2.3 Hugepages . . . 6
2.2.4 Translation Lookaside Buffer . . . 6
2.2.5 Non-Uniform Memory Access . . . 6
2.3 Linux Network Stack . . . 6
2.3.1 Sending and Receiving Data Packets . . . 7
2.3.2 Network Access Layer . . . 7
2.3.3 Internet Layer - IP . . . 8
2.3.4 Transport Layer - TCP and UDP . . . 8
2.3.4.1 Data Packet Terminology . . . 8
2.3.4.2 TCP . . . 8
2.3.4.3 UDP . . . 8
2.3.5 Application Layer . . . 9
2.4 Interactions Between Kernel and Device . . . 9
2.4.1 Interrupts . . . 9
2.4.2 Polling . . . 10
2.5 Data Plane Development Kit . . . 11
2.5.1 Poll Mode Drivers . . . 12
2.5.2 Core Components . . . 12
2.5.3 Run-to-completion and Pipeline Model . . . 12
2.5.4 Environment Abstraction Layer . . . 13
2.5.5 TestPMD . . . 14

2.6 io_uring . . . 14
2.6.1 Polling in io_uring . . . 15
2.6.2 liburing . . . 15
2.6.3 io_uring-udp.c . . . 15
2.7 Network Interface Card . . . 15
2.8 Single Root Input/Output Virtualization (SR-IOV) . . . 16
2.9 Pktgen . . . 16
2.9.0.1 Measuring CPU Cycles with RDTSC . . . 17
2.10 Poisson Process and Switched Poisson Process . . . 17
2.11 Linux Kernel Tuning . . . 17
2.11.1 Receive Side Scaling . . . 17
2.11.2 Receive Packet Steering . . . 17
2.11.3 Accelerated Receive Flow Steering . . . 18
2.11.4 Transmit Packet Steering . . . 18
2.11.5 ip_early_demux . . . 18
2.11.6 Socket Receive Queue Memory . . . 18
2.11.7 Socket Transmit Queue Memory . . . 18
2.11.8 netdev_max_backlog . . . 18
2.11.9 Generic Receive Offloading . . . 19
2.11.10 netdev_budget . . . 19
2.11.11 RX Flow Hash Indirection Table and Hash Function . . . 19
2.11.12 Hardware Buffer Queue . . . 19

3 Related Work **20**
3.1 Comparison of Frameworks for High-Performance Packet IO . . . 20
3.2 Understanding Modern Storage APIs: A systematic study of Libaio, SPDK, and io_uring . . . 21
3.3 Power Efficient High Performance Packet I/O . . . 21
3.4 A Study of Tools to Develop a Traffic Generator for L4 - L7 layers . . . 21
3.5 Understanding Host Network Stack Overheads . . . 22

4 Method **23**
4.1 Setup . . . 23
4.1.1 Hardware Setup and Specifications . . . 23
4.1.2 Software Setup and Configuration . . . 24
4.1.2.1 Versions . . . 24
4.1.2.2 Configurations . . . 24
4.1.3 Network Traffic Generator Configuration . . . 24
4.1.4 Base Setup . . . 24
4.2 Architecture . . . 25
4.2.1 Setup Using DPDK . . . 25
4.2.2 Setup Using io_uring . . . 26
4.2.3 Setup Using Linux Network Stack . . . 26
4.3 Calculations . . . 27
4.4 Changes to Source Code . . . 27
4.5 Determining Optimal Transmission Rate . . . 28
4.6 Methodology for Benchmarking DPDK, io_uring and the Standard Linux Network Stack . . . 28
4.6.1 Applications . . . 28
4.6.1.1 Application Using DPDK . . . 28
4.6.1.2 Application using io_uring . . . 29
4.6.1.3 Application Using Standard Linux Network Stack . . . 29
4.6.2 Data Collection and Evaluation . . . 29

4.6.2.1 Scripts . . . 29
4.7 Simulating and Evaluating Bursty Traffic . . . 30
4.7.1 Overview . . . 30
4.7.2 Applications . . . 30
4.7.3 Data Collection . . . 30
4.8 Approach for Tuning Linux Kernel Parameters . . . 31
4.8.1 Tuning Linux Kernel Parameters . . . 32
4.8.1.1 Receive Side Scaling (RSS) . . . 32
4.8.1.2 Receive Packet Steering (RPS) . . . 32
4.8.1.3 Accelerated Receive Flow Steering (aRFS) . . . 32
4.8.1.4 netdev_max_backlog . . . 32
4.8.1.5 ip_early_demux . . . 33
4.8.1.6 Socket Receive Queue Memory . . . 33
4.8.1.7 Socket Transmit Queue Memory . . . 33
4.8.1.8 Generic Receive Offloading (GRO) . . . 33
4.8.1.9 netdev_budget . . . 33
4.8.1.10 RX Flow Hash Indirection Table and Hash Function . . . 33
4.8.1.11 Hardware Buffer Queue . . . 33
4.8.2 Data Collection and Evaluation . . . 34

5 Results **35**
5.1 Benchmarking DPDK, io_uring and the Linux Network Stack . . . 35
5.1.1 Absolute Transmission Rate . . . 35
5.1.2 DPDK . . . 35
5.1.3 io_uring . . . 39
5.1.4 Standard Linux Network Stack . . . 41
5.1.5 Summary of Benchmarking Results . . . 44
5.2 Bursty Traffic with Switched Poisson Process . . . 45
5.2.1 Switched Poisson Process . . . 45
5.2.2 DPDK . . . 47
5.2.3 io_uring . . . 48
5.2.4 Standard Linux Network Stack . . . 50
5.2.5 Summary of Bursty Traffic Results . . . 51
5.3 Result of Tuning Linux Kernel Parameters . . . 52
5.3.1 Configurable Parameters . . . 52
5.3.1.1 RX Flow Hash Indirection Table and Hash Function . . . 52
5.3.1.2 Hardware Buffer Queue . . . 54
5.3.1.3 netdev_budget . . . 56
5.3.1.4 Socket Receive Queue Memory . . . 57
5.3.1.5 Socket Transmit Queue Memory . . . 60
5.3.1.6 Receive Packet Steering (RPS) . . . 61
5.3.1.7 Generic Receive Offloading (GRO) . . . 66
5.3.1.8 netdev_max_backlog . . . 67
5.3.1.9 ip_early_demux . . . 67
5.3.2 Non-Configurable Parameters . . . 67
5.3.2.1 Accelerated Receive Flow Steering (aRFS) . . . 67
5.3.3 Summary of Tuning Kernel Parameters . . . 67

6 Discussion **68**
6.1 Results . . . 68
6.1.1 Benchmarking DPDK, io_uring and Linux Network Stack . . . 68
6.1.2 Handling of Bursty Traffic . . . 69
6.1.3 Tuning the Linux Kernel Network Parameters . . . 71

6.1.3.1 netdev_budget . . . 71
6.1.3.2 RX Flow Hash Indirection Table and Hash Function . . . 71
6.1.3.3 Hardware Buffer Queue . . . 71
6.1.3.4 Socket Receive Queue Memory . . . 72
6.1.3.5 Socket Transmit Queue Memory . . . 72
6.1.3.6 Receive Packet Steering . . . 72
6.1.3.7 Generic Receive Offloading . . . 73
6.1.3.8 ip_early_demux . . . 73
6.1.3.9 netdev_max_backlog . . . 73
6.2 Evaluation of Methodologies . . . 73
6.2.1 General Evaluation of Method . . . 73
6.2.2 Evaluation of Benchmarking Methodology . . . 74
6.2.3 Evaluation Methodology for Simulating Bursty Traffic . . . 74
6.2.4 Evaluation of Methodology Used for Tuning the Kernel . . . 75
6.3 The Work in a Wider Context . . . 75

7 Conclusion **76**
7.1 Benchmarking DPDK, io_uring and the standard Linux Network Stack . . . 76
7.2 Handling of Bursty Traffic . . . 77
7.3 Tuning Linux Kernel Parameters for Network Performance . . . 78
7.4 Future Work . . . 78

Bibliography **80**

A Kernel tuning graphs **84**
A.1 ip_early_demux . . . 84
A.2 RX flow indirection table and hash function . . . 87
A.3 Hardware buffer queue . . . 88
A.4 netdev_budget . . . 91
A.5 Socket receive queue memory . . . 93
A.6 Socket transmit queue memory . . . 97
A.7 Generic Receive Offloading (GRO) . . . 100
A.8 netdev_max_backlog . . . 103

B Code **106**
B.1 Simulating network traffic with Pktgen . . . 106
B.1.1 network_metrics.lua . . . 106
B.1.2 latency_histogram.lua . . . 110
B.1.3 bursty_traffic.lua . . . 112
B.2 Changes to source code . . . 116
B.2.1 hash_table.c . . . 116
B.2.2 hash_table.h . . . 117
B.2.3 pktgen.c . . . 117
B.2.4 lpktgenlib.c . . . 118
B.2.5 pktgen-latency.h . . . 118
B.2.6 meson.build . . . 118
B.3 Linux network stack application (udp_fwd_socket.c) . . . 118
B.4 Poisson distribution (switched_poisson_process.py) . . . 120
B.5 io_uring application (liburing_fwd.c) . . . 121

1 Introduction

This chapter presents an introduction to the thesis. The introduction is divided into five parts. The motivation for why this thesis was conducted is given in Section 1.1. The aim of what the thesis wants to accomplish is available in Section 1.2. Research questions that are investigated and answered are in Section 1.3. A brief explanation of the approach used for investigating the research questions is described in Section 1.4. Lastly, in Section 1.5 the contributions of the thesis are summarized.

1.1 Motivation

Packet processing [24] is an important part of any software organization developing networks that must support large amounts of data transfer. In an increasingly digitalized world where performance and speed are key factors, it is important for organizations within the telecommunications industry to continuously explore shifting trends to be able to capitalize on the technologies which are the most beneficial for their organization and their users. Increased speed in network communication is, however, not only of interest to companies that could benefit from it but also to all users connected to the internet as it could potentially increase the quality of experience.

DPDK (Data Plane Development Kit) was developed by Intel to accelerate packet processing and was released to the public in the year 2010 [43]. It accelerates network I/O by allowing data packets to transition to the user space directly without having to go through kernel space, thus avoiding overheads such as memory copying and context switching [3].

In the year 2019, a new API, for handling asynchronous I/O in the Linux kernel, called io_uring, was released [18]. io_uring could potentially offer better performance for packet processing in Linux, by adding asynchronous I/O and polling to the Linux network stack [18].

There are several research papers focusing on DPDK [31, 57, 34, 8] as it has been widely adopted for accelerating packet processing within telecommunications. io_uring on the other hand is still new and therefore there are considerably fewer scientific publications examining it. One of the very first in-depth studies of io_uring was conducted in 2022 [18] where it was compared with the software library solutions libaio [37] and SPDK [12]. SPDK is more focused on high performance in storage applications whereas libaio is more related to io_uring where libaio also performs asynchronous operations to increase performance.

However, io_uring is still seeing development, as new kernel versions are released. In addition, DPDK has a different use case than SPDK. Therefore, it is not certain that results from previous studies will translate into results of future studies.

To the best of the authors knowledge, there are currently no other scientific papers comparing the network performance of io_uring and DPDK. Therefore, an in-depth comparison of DPDK and io_uring in regard to network performance can lead to valuable new insights.

1.2 Aim

The aim of this study is to investigate how DPDK, io_uring, and the standard Linux network stack compare to each other in terms of networking performance.

The study will also give insights into how the Linux kernel can be tuned to increase the network performance of applications utilizing io_uring.

The results are of interest and provide value to organizations mainly in the telecommunications industry, as the study will give further understanding of how different packet processing techniques can be utilized and how they compare to one another. The comparison between DPDK, io_uring, and the standard Linux network stack will also give a better awareness of which design principles and architectures perform best under certain conditions.

1.3 Research Questions

As motivated in the earlier sections, the goal of the thesis is to analyze DPDK, io_uring, and the standard Linux network stack in terms of various network metrics. To be more precise the following three questions will be answered:

1. How does the network performance of io_uring and DPDK compare to the standard Linux network stack in terms of latency, packet loss, throughput, and packet rate?
2. How well does io_uring, DPDK, and the standard Linux network stack handle bursty traffic data considering latency and packet loss?
3. What impact does tuning network parameters in the Linux kernel have on applications utilizing io_uring?

1.4 Approach

This section provides an overview of the methods used for investigating the research questions stated in Section 1.3 and an explanation of the delimitations of the thesis.

1.4.1 Research Methods

To investigate the research questions, a test environment that utilizes a network traffic generator was set up. The traffic generator was used to generate data packets and store values that were used for calculations of network metrics.

Three forwarding applications were used for testing io_uring, DPDK, and the standard Linux network stack. DPDK includes an application specifically made for packet forwarding and testing the limits of DPDK. For io_uring and the standard Linux network stack forwarding applications were created which utilized the different techniques and mechanisms of each technology.

To make sure that io_uring had optimal conditions to perform, kernel tuning was used. The results of how different kinds of tuning impact the networking performance of io_uring were evaluated. Note that kernel tuning does not affect DPDK since it works in the user space.

1.4.2 Delimitations

The study is limited to measuring the network metrics of throughput, packet loss, latency, and packet rate to determine the performance of the packet processing libraries. These metrics should provide a good enough foundation for conclusions. However, there are other network metrics that could be explored and measured such as jitter, which is the variation in the latency on a flow of data.

The scope is limited to a selected subset of libraries for packet processing, io_uring, DPDK, and the standard Linux network stack.

The testing is done using one hardware setup, meaning the results of the study were limited to the performance of the hardware the study is conducted on. Only one data flow was used, meaning that packets were sent from one source to one destination and then back, in other words, parallelism will not be applied.

The scope of this thesis is limited to investigating the communication protocol UDP. Testing TCP could give some further insight into the packet processing libraries' networking capabilities. In light of performance, UDP does not have congestion control and can therefore give a more true value in the packet processing performance aspect. UDP is better when sending large amounts of data without checking which packets reached the destination or not. This is therefore a better choice for a first performance evaluation of the frameworks and libraries.

Due to the scope and focus of this thesis, only Internet Protocol Version 4 (IPv4) was tested in the network layer as it is sufficient enough to address the research question of the thesis.

When tuning Linux kernel parameters the application using io_uring will be the subject when looking at performance increase.

1.5 Contributions

The main contributions from the study to the subject of packet processing and computer networking are as follows:

- DPDK greatly outperforms io_uring and the standard Linux network stack when looking at latency, throughput, packet rate, and packet loss. This may be because DPDK uses kernel-bypassing thus overheads are reduced.
- io_uring outperforms the Linux network stack in terms of throughput, packet rate, and packet loss. There was no significant difference in latency between the two.
- Through simulating bursty traffic to mimic real-world data traffic it was concluded that in terms of latency io_uring and the standard Linux network stack are better at handling traffic above their maximum performance than DPDK. This may be because io_uring and the Linux network stack have access to the kernel which can help facilitate the management of congestion handling.
- It is possible to tune Linux kernel parameters to increase the network performance of io_uring applications. The kernel parameter RPS has the most significant positive impact on network performance. However, the impacts of tuning Linux kernel parameters are not significant enough to bring io_uring to the network performance of DPDK.

2 Background

In this chapter, the relevant theoretical basis for the thesis project is presented. Section 2.1 covers the necessary networking metrics that will be used to analyze the network performance of the libraries. Key concepts of memory management are described in Section 2.2. The basics of Linux networking are described in Section 2.3. In Section 2.4 the different ways devices may communicate with the kernel are explained. Section 2.5 gives an overview of how DPDK works and what its core components are. How io_uring works and its most important concepts are covered in Section 2.6. In Section 2.7 a description of the network interface card (NIC) is given. Section 2.8 covers SR-IOV and what its use cases are. The network traffic generator, Pktgen, that is used to generate data packets and measure network metrics is presented in Section 2.9. To generate bursty traffic that simulates real-life data traffic flow a mathematical model called Poisson statistical distribution is described in Section 2.10. Finally, in Section 2.11 different ways of tuning the Linux kernel to impact network performance are explained.

2.1 Network Metrics

When conducting research on network performance there are several different metrics of interest. Amongst the most commonly used are:

- **Throughput**: Throughput represents the amount of data transferred per time unit and can be measured in bits per second. To define throughput, take the example where Host A is trying to send a video clip to Host B. During the transfer of data two types of throughput can be measured: instantaneous and average throughput. Instantaneous throughput is the rate (bits/sec) at which Host B is receiving the file at a given time. Average throughput is defined as the number of bits transferred divided by the number of seconds the transfer took to finish [33]. In general, in this document, throughput refers to average throughput.
- **Latency (Delay)**: Latency refers to the time it takes for data to be transferred across a network to its destination measured from the time the first bit is sent to the time the last bit is received [21]. Latency can be measured as round-trip time (RTT), that is the time between the start of transmitting data to receiving data as a response. Latency in this context will refer to RTT.

- **Packet rate**: While throughput is measured in bits per second, the packet rate is dependent on the packet size where smaller packets will increase the packet rate and larger packets will have the reverse effect. Usually, the packet rate is measured with minimum-sized packets [42]. Although in the context of this document, the packet size may vary.
- **Packet loss**: Packet loss can be calculated as a percentage of packets that were sent but never received at the destination. There are several reasons why packet loss can occur and it can be a potential indicator that congestion is occurring somewhere in the data path [33]. An example is when a packet arrives in a full queue, with nowhere to store the packet the router will drop it. From the end-systems viewpoint, it looks like the packet has been transmitted but never received at its destination.

2.1.1 Throughput and Latency Trade-off

One of the fundamental problems of parallel computing is finding the most optimal way to distribute a parallel program across the processors of a parallel system. Both latency and throughput are important factors when judging the quality of the mapping, but increasing or decreasing one of them does not correlate to an increase or decrease of the other [53].

If the goal of the mapping is to minimize latency, then parallel mapping to all processors is often the optimal solution as all processors are then available to each task and can therefore process incoming individual data sets at the fastest possible speed. If the goal is to maximize the throughput it may instead be more advantageous to process multiple data sets simultaneously as a pipeline. Maximizing the throughput this way would however increase the latency for individual data sets but would increase the throughput as single data sets would be more efficiently processed by fewer processors [53].

2.2 Memory Management in Linux

Functionality in memory management in Linux is continuously added. In this section, some of the key concepts of memory management in Linux are explained.

2.2.1 Virtual Memory

Virtual memory is a technique used in Linux that abstracts physical memory on the system from the application software. This makes it possible for different applications to address the same virtual memory, which in turn maps to different physical addresses. When using virtual memory every memory access uses a virtual address that needs to be translated in the Memory Management Unit (MMU) from the virtual address to its physical address counterpart that the memory controller can understand [16].

2.2.2 Memory Pages

The physical memory is divided into page frames called memory pages. The size of the memory pages varies between architectures and can also to an extent be configured manually. Every memory page can be mapped to one or more virtual pages, and these mappings are described in page tables that are used for the translation of virtual addresses to physical memory addresses [16]. Memory pages play a central role in Linux memory management and is important to have knowledge of, to better understand other memory management features and functionalities such as hugepages, which are a crucial part of DPDK.

2.2.3 Hugepages

Hugepages is a feature that is available from kernel version 2.6 in Linux [30]. On modern CPUs, memory pages can be mapped through higher levels in the page table. It is possible to map 2MB and even 1GB pages through second and third-level pages. These larger mapped pages are called Hugepages in Linux. By using fewer but bigger pages, the number of entries in the TLB will decrease and thus decrease overhead since fewer misses will occur [16].

2.2.4 Translation Lookaside Buffer

There is overhead in terms of processing power any time the CPU has to translate memory addresses as translation requires memory access which is relatively slow compared to the CPU clock frequency. To avoid this as much as possible the CPU maintains a cache of frequently used translations called the Translation Lookaside Buffer (TLB). However, the TLB is a scarce resource and applications may need to use more memory addresses than can be stored in the TLB causing TLB misses that can lead to poor performance [16].

2.2.5 Non-Uniform Memory Access

Non-Uniform Memory Access (NUMA) is a type of memory architecture that can be seen from two different perspectives: hardware and Linux software.

From a hardware perspective, a NUMA system can be seen as a computer platform that is comprised of multiple physical components such as CPUs, IO buses, and local memory. Multiple physical components containing one or more of such components can be thought of as a subset of a larger computer system architecture, where multiple independent CPUs share a common memory and can perform tasks simultaneously [51].

This system architecture is also referred to as a Symmetric Multi-Processor (SMP) system. The hardware components within the NUMA system connect to each other via a system interconnect e.g., crossbar or point-to-point link. NUMA systems are used to provide scalable memory bandwidth, meaning the computer system can handle increased amounts of memory access without a performance decrease. However, the memory access time and memory bandwidth can vary depending on the distance between the target memory and the CPU or IO bus that wants to access it [51].

From the Linux software perspective, the system hardware is divided into multiple software abstractions called *nodes*, which may contain local memory, CPUs, and IO buses. The distance between the nodes will impact bandwidth and memory access times. As a result of this, it is important to keep in mind where the nodes are in relation to each other to optimize memory access times and overall system performance in Linux [51].

2.3 Linux Network Stack

Linux uses the TCP/IP network model, a suite of data communication protocols such as TCP, UDP, and IP. The TCP/IP model shares some similarities with the generic OSI model but also has some structural differences [26].

While the OSI model contains seven layers, the TCP/IP model is composed of fewer layers and there is no universal agreement on exactly how TCP/IP can be described as a layered model. Most descriptions contain between three to five layers [26]. The architecture described by Hunt [26] contains 4 layers and is based on the three layers from the Department of Defence (DoD) Protocol Model (Application, Network Access, Host-to-Host Transport) [14] in combination with the Internet layer. The layout and order of each of the models are depicted in Figure 2.1.

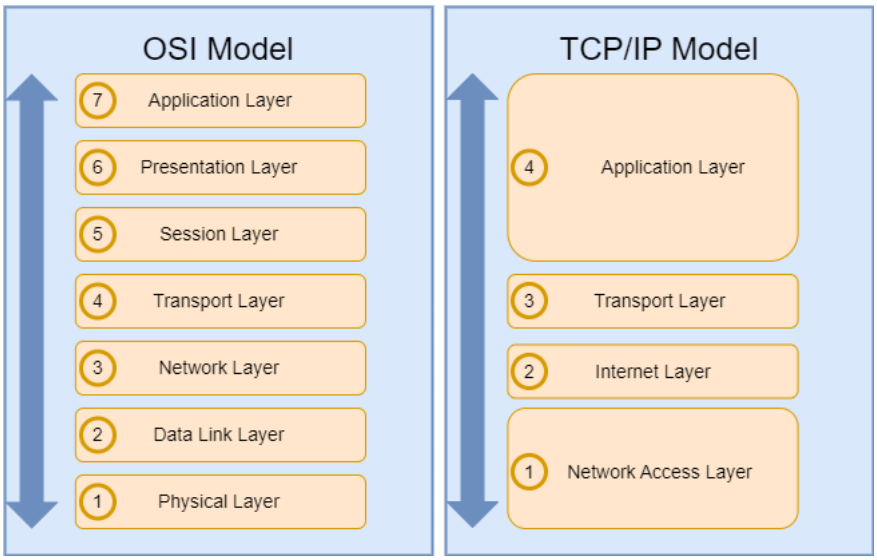

Figure 2.1: The OSI and TCP/IP architectures (Figure inspired by Hunt [26])

2.3.1 Sending and Receiving Data Packets

In both the OSI and TCP/IP model, data travels down the stack when being sent to the network and up the stack when being received from the network.

As data is passed down the stack, each layer adds its own control information to the outgoing data packet to ensure that the packet will be properly delivered to its destination. The process of adding delivery information at each layer the packet passes by is called encapsulation and the control information is called header, as it is placed before the actual payload which holds the data being sent [26].

The reverse process happens when receiving data packets, called decapsulation. Instead of each layer adding information it instead strips off the part of the header that it needs before passing it to the layer above [26]. The path of a data packet being transmitted and received in the TCP/IP model is shown in Figure 2.2.

Figure 2.2: End-to-end path of a data packet (image inspired by Hunt [26])

2.3.2 Network Access Layer

The TCP/IP Network Access Layer in Linux encompasses the core functionalities of the three lowest layers in the OSI Model (Physical, Data Link, and Network). The layer contains protocols that the system uses to deliver data to other devices on the network. It also handles en-

capsulation of IP datagrams as well as mapping of IP addresses to physical addresses (MAC addresses) [26].

2.3.3 Internet Layer - IP

Above the Network Access Layer is the Internet layer. In this layer, the most significant protocol is the connectionless Internet Protocol (IP). IP provides the heart of the TCP/IP, providing basic packet delivery functionality and all protocols rely on IP to deliver incoming and outgoing data. As the building block of the Internet, some of the functionalities of IP include defining the datagram; routing datagrams to remote hosts, defining the Internet addressing scheme, and being the link between Network Access Layer and Transport Layer [26].

The two IP versions used today are IPv4 and IPv6, where IPv6 provides some expanded addressing capacity among other things [26].

2.3.4 Transport Layer - TCP and UDP

The Transport Layer is located above the Internet Layer. The two most important protocols within the Transport Layer are the User Datagram Protocol (UDP) and the Transmission Control Protocol (TCP) [26].

2.3.4.1 Data Packet Terminology

Different terms are used within different layers of TCP/IP depending on if UDP or TCP is being used. Applications using UDP refer to data as a message while TCP applications use the term stream. In the transport layer, TCP refers to data as a segment while UDP calls it a packet. Within the internet layer, all data is viewed as blocks which are referred to as datagrams. Finally in the network access layer data is most commonly called frames or packets [26]. The terms related to each network layer in TCP or UDP are shown in Figure 2.3.

2.3.4.2 TCP

TCP is connection-oriented and can ensure reliable delivery of data over its connections. It sets up the end-to-end connection between the hosts by using a three-way handshake to ensure that the remote TCP is ready to start receiving. Reliability is provided through using mechanisms such as Positive Acknowledgement with Retransmission (PAR), essentially meaning that the system will send the data again unless it hears a response from the remote system where the data arrived. TCP also uses checksums that the receiver of the data can use to verify that the data it received has not been corrupted. If the data is damaged the receiver discards it and if it is undamaged the receiver sends a positive acknowledgment back [26].

2.3.4.3 UDP

UDP is a connectionless protocol, meaning no connection between source and destination needs to be established before data transmission can begin. Because of this UDP provides low protocol overhead [26] and offers a greater possibility for stress-testing the performance of high-performance I/O frameworks compared to TCP [33]

UDP is often referred to as unreliable as it has no techniques in place to verify that data is being received at the intended destination. There are multiple reasons why applications use UDP as their data transport service. One of these is when the data transfer is small the overheads of creating connections and ensuring reliability might be greater than the cost of retransmitting the data. Applications using a query-response model also work well with UDP, when a response is not given within a set time period another query is sent [26]. UDP is also relevant for real-time applications such as live speech and video streams, where the reception

of old data packets is useless as the most up-to-date data packets need to be processed to give a smoother experience for the user.

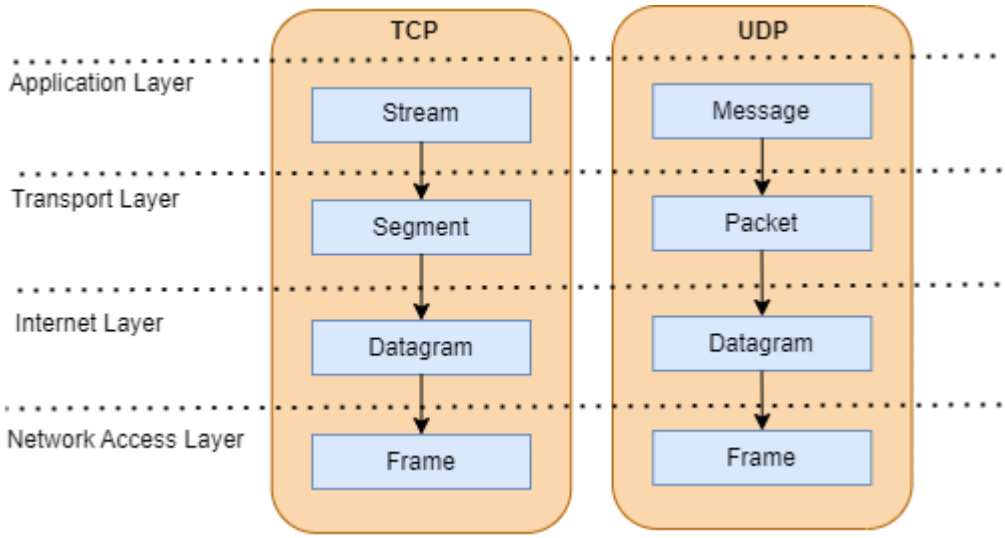

Figure 2.3: Data terms for TCP and UDP (figure inspired by Hunt [26])

2.3.5 Application Layer

The top layer is the Application Layer. It includes all processes which rely on the Transport Layer protocols for data delivery. Within the Application Layer, there are many different protocols intended for different things, most of which provide user services. New services are continuously being added to the layer. Some of the more well-known protocols are Simple Mail Transfer Protocol (SMTP), Hypertext Transfer Protocol (HTTP), File Transfer Protocol (FTP), and Domain Name System (DNS) [26].

2.4 Interactions Between Kernel and Device

There are two main techniques for the Network Interface Card (NIC) to communicate with the kernel, interrupts and polling. They can be used separately, but a combination of them is also an option [9].

Interrupt-driven I/O is currently more commonly used than polling, where polling is more often used for high-throughput I/O. Sometimes device drivers are configured to use polling when the I/O rate is high and switches to use interrupts when the I/O rate drops in rate [52].

2.4.1 Interrupts

Interrupt signals are a way of diverting the CPU from its normal flow of execution. After the CPU receives an interrupt it is forced to stop its current process and handle the interrupt, before it can continue with what it was doing before [10]. An example of this is when the NIC has received a new input frame (data packet), then it could notify the kernel of this by using an interrupt. One advantage of using interrupts is that they allow for very low latency between receiving frames and beginning the processing of them [9].

There are several types of interrupts. The NIC could for example tell its driver about the reception of a frame, transmission failure, or if a DMA (Direct Memory Access) transfer has been completed successfully [9].

Input frames are handled by interrupt handlers, which can be divided into two parts: top-half and bottom-half [9].

The top-half contains the immediate response to an interrupt request, meaning everything that must be executed to preserve the packet data when the CPU first receives the interrupt notification. Whenever the associated CPU is running the top-half handler it is disabled from handling other interrupt requests and therefore it can only be executed when the associated CPU is not already serving another interrupt. The bottom-half, sometimes referred to as a delayed handler, contains things related to the processing of stored interrupts containing the packet data. It is considered less time-critical compared to the top-half and is executed whenever the top-half has been completed and the kernel and associated CPU are free to process interrupts. The division between top-half and bottom-half allows for reducing the time when the CPU cannot handle incoming interrupts, as the top-half used for data preservation is always prioritized before bottom-half [9].

The following steps describe a simplified version of how the process of receiving and processing a packet using bottom-half and top-half interrupts handlers could work [9]:

1. The NIC receives a data packet and sends a signal to the CPU informing it of the interrupt.
2. The CPU stops the current execution, starts executing the associated top-half, and disables further interrupt notifications.
3. During the execution of the top-half the following actions a typically performed:
   - Save necessary data in RAM for processing the interrupt.
   - Mark the memory with a flag to make sure the kernel knows about the interrupt and where the data is located.
   - Re-enable interrupt notifications on the local CPU.
4. When the kernel is free to handle other processes it checks for the flag that was set during the top-half execution. If the flag is found, the bottom-half starts the execution to process the interrupt containing the packet. When the packet has been processed it will clear the flag, thus allowing it to be set again when the top-half is executed once again.

One of the drawbacks of using interrupts is that it has low performance during high traffic loads, as forcing the CPU to queue frames continuously will make it unable to complete other tasks or process the already queued frames. If the traffic load is too high the system queue could become full and since the processing of the interrupts has lower priority they cannot be dequeued causing the system to collapse. Incoming new frames cannot be queued since the queue is already full and frames in the queue cannot be processed as there is no available processing capacity in the CPU. The scenario is called receive livelock and causes the throughput to fall to zero [9].

2.4.2 Polling

Polling is a way for the hardware to check if an event has happened which needs attention by checking regularly for an event. [52].

This is done by periodically checking status bits to determine if it is time for handling the next I/O instruction. When an event happens the I/O device will put the information in a status register where the processors can read the information from and handle incoming events directly [40].

One of the advantages of polling is that it can be used to handle higher I/O rates. Real-time embedded applications sometimes use polling when interacting with I/O devices since the I/O rates are predetermined, which makes it easier to control associated overhead [40].

One advantage is that events can be handled directly after they have occurred. The downside is that it comes at a cost as it wastes a lot of processing time, by continuously reading the

status register and therefore cannot handle other processes. On top of that, the processor is also faster than I/O devices and may therefore read the status register many times while the I/O device has not yet been able to complete an I/O operation. When the I/O operation has finished the processor must still read the status to confirm that the operation was successful [40].

2.5 Data Plane Development Kit

Data Plane Development Kit (DPDK) was released by Intel in 2010 to help increase the performance of packet processing [43]. The main purpose of DPDK is to be a simple yet complete framework for fast packet processing within the data plane where users have the alternative of adding their own protocol stacks [47]. Unbinding a network interface from the kernel and then binding it to a DPDK-specific module enables user space IO for DPDK applications [44]. This bypasses the Linux kernel and avoids the overheads created by the kernel when doing packet processing.

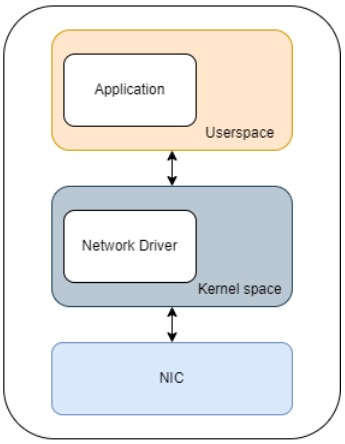

Figure 2.4: Packet processing using the standard Linux network stack

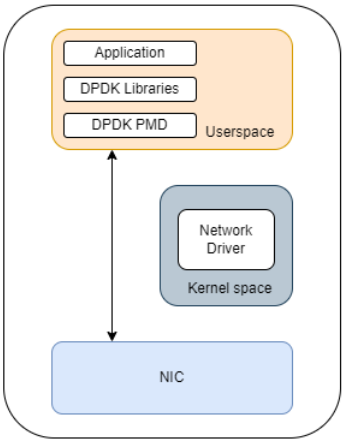

Figure 2.5: DPDK packet processing with bypass of the kernel

Figure 2.4 shows a simplified version of core components involved in packet processing using the standard Linux network stack, while Figure 2.5 shows the components involved in packet processing when using DPDK which excludes the Linux kernel from the processing flow.

2.5.1 Poll Mode Drivers

DPDK includes 1, 10, and 40 Gigabit Poll Mode Drivers (PMD) among others. PMDs are software components that have APIs allowing for interacting with NICs and their queues from within the user space. DPDK applications can access RX and TX descriptors without the use of interrupts by accessing the NIC directly using PMDs, thus eliminating the need for the kernel to handle network traffic [48].

2.5.2 Core Components

In DPDK there are several components that are necessary for making the bypassing of the kernel possible, some of which are:

- The Memory Pool Manager (rte_mempool) manages the memory allocation of pools of objects, where each pool of objects can be identified by name and uses a ring buffer to store free objects. rte_mempool also provides other services such as an alignment helper and a per-core object cache [47].
- Network Packet Buffer Management (rte_mbuf) gives a DPDK application the ability to create and destroy message buffers (mbufs), which are used to hold data packets, through the API it provides. Mbufs are created by the application at start-up and are stored within a rte_mempool [47].
- Ring Manager (rte_ring) is a fixed-size lockless table designed to work with multiple producers and consumers. rte_ring also implements the FIFO (first in first out) principle. rte_mempool uses the ring structure provided by rte_ring to as a communication mechanism between CPU cores or as execution blocks that are connected on a core [47].

2.5.3 Run-to-completion and Pipeline Model

DPDK applications for packet processing can be categorized into two different types: run-to-completion or pipeline model [48].

With the run-to-completion model, shown in Figure 2.6, all resources are first allocated and assigned to logical cores before calling Data Plane applications. The Run-to-completion model has no support from a scheduler and I/O devices are accessed through polling, with the purpose of avoiding overheads caused by interrupts. [47].

In the synchronous run-to-completion model, a CPU core associated with a specific port on the NIC reads incoming packets by polling the RX descriptor ring using a DPDK API. The packet is then processed on the same CPU core which did the polling, and it is then placed on a port's TX descriptor ring [48], see 2.5.3.

The processing of packets in logical cores in the run-to-completion model includes the following steps [48]:

1. Get input packets through Poll Mode Driver receive API
2. Process every packet one at a time up to its forwarding
3. Transmit packets ready for output using Poll Mode Driver transmit API

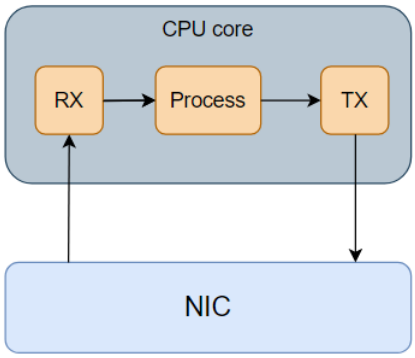

Figure 2.6: Run-to-completion model in DPDK

In addition to the run-to-completion model, it is also possible to use a pipeline model by passing packets between cores via rings, as shown in Figure 2.7. Work can be divided between cores, with the pipeline model, potentially making the processing more efficient [47].

With the asynchronous pipeline model, some master cores are dedicated to the retrieval of packets received, while worker cores handle the processing of packets and transmitting the packet forward. In Figure 2.7 the packet flow of the pipeline model is shown. The packets are exchanged between the cores through rings. The processing of packets with the pipeline model includes the following steps [48]:

1. Master core gets input packets through Poll Mode Driver receive API
2. Master core places received packets in packet queues making them available for worker cores to process
3. Worker core receives a packet from the packet queue
4. Worker core processes the received packet
5. Once the packet has been processed it may be forwarded to other worker cores for further processing or to a dedicated transmit core that will transmit the packet to the NIC

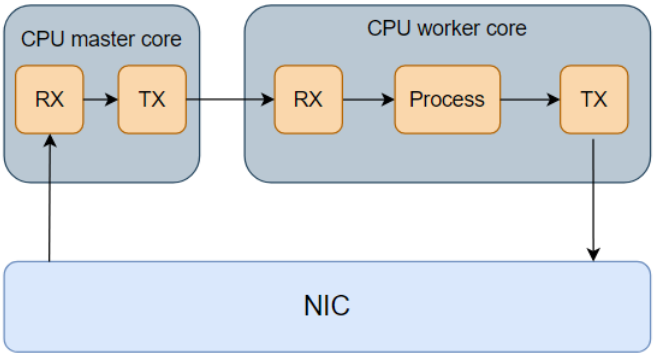

Figure 2.7: Pipeline model in DPDK

2.5.4 Environment Abstraction Layer

The Environment Abstraction Layer (EAL) plays a very important part in DPDK when gaining access to low-level resources and to required hardware, as well as abstracting the hardware away from the user [19]. Some other features that are handled by the EAL include:

- Handling interrupts by supplying an interface for callbacks by registering and unregistering.
- Enables debugging and logging, allowing for locating errors more easily. This includes writing logs, dumping the stack, and panicking.

For DPDK to be able to access memory the EAL uses a function called `mmap()`.

2.5.5 TestPMD

TestPMD is an application that is distributed together with DPDK which can be used to test DPDK in packet forwarding mode [46]. The main purpose of TestPMD is to forward packets between network ports on a NIC [23]. TestPMD is commonly used to test the performance of DPDK.

There are three different forwarding modes that TestPMD can be configured to run with: Input/Output, RX-only, and TX-only [23].

Input/Output mode, generally referred to as IO mode, is the default mode of TestPMD. With IO mode a CPU core will receive data packets from one RX port and transmit the packets on a TX port, where the same port can be used for both receiving and transmitting [23].

In RX-only mode the TestPMD application only polls packets from the RX ports and frees them without any transmission, while TX-only mode locally generates 64-byte IP packets and transmits them from TX ports [23].

2.6 io_uring

io_uring was originally created in the Linux kernel by Jens Axboe [28]. io_uring gets its name from the idea of using ring buffers as shown in Figure 2.8. These ring buffers handle the communication between kernel space and user space [28].

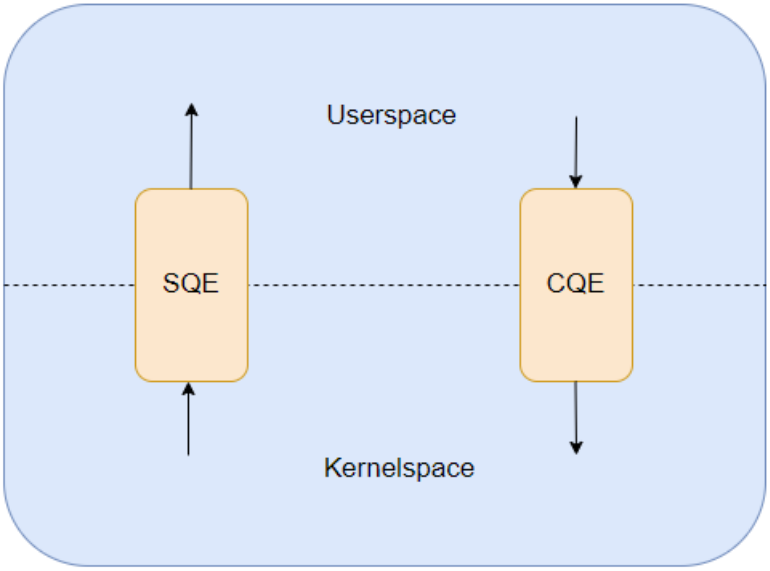

Figure 2.8: Showing how the submission queue and completion queue are used between kernel space and user space

System calls are expensive to make, but with io_uring they can be made more seldom using batch submissions together with the ring buffers. Instead of sending multiple system calls io_uring will batch multiple system calls into one, thus it reduces the number of system calls

that will be made. The system calls are then handled asynchronously which furthermore decreases overheads. It can avoid overheads by implementing a zero-copy functionality where information between the buffers is shared, no copy is then needed to transfer information from the kernel space to the user space [28].

io_uring is still seeing continuous updates and optimizations, as of writing this. The latest updates for io_uring in 2023 were described in a post made by the author of io_uring [5]. Among these were two features called multi-shot and IORING_SETUP_DEFER_TASKRUN which could allow for further performance increases of io_uring [5].

2.6.1 Polling in io_uring

io_uring focuses on reducing overhead and therefore it is also very important to reduce the number of system calls. To solve this problem io_uring gives the option of using submission queue polling. Submission queue polling allows the user to submit input and output requests without doing a system call. When using io_uring with submission queue polling the kernel will first, as usual when using io_uring, configure a shared queue between the user space and the kernel space. Then requests can be submitted to the queue. When a request is added to the submission queue, polling will be used to determine that a request has been added and will then handle them accordingly. If packets are coming in continuously the kernel thread will poll to handle the incoming packets, but if no packet has been received within a given time frame configured by the user, the polling mode needs to be started again. In other words, it means that if polling mode is active and packets continuously flow, then the polling will never stop by itself.

2.6.2 liburing

io_uring works on a low-level in the user space and the kernel space, but to make it more accessible to use for developers, high-level implementations are available. One of these implementations is an open-source library called liburing. The library helps with setting up an io_uring instance and tearing it down if needed. With the liburing, the abstraction layers do not have to be implemented manually [27].

By using liburing the application does not have to manage the ring buffers or take care of memory allocation which would be the case if using io_uring without the library. In other words, liburing handles much of the lower-level features of io_uring which makes the API more comprehensible and user-friendly [32].

2.6.3 io_uring-udp.c

Among the sample applications given in the liburing GitHub repository is an application named io_uring-udp.c [4]. It is described as an application used to show the usage of multishot recvmsg and buffer rings. It is specifically made to work as a UDP echo server that responds to all incoming packets with the same packet [4]. However, it was not made with networking performance in mind and does add additional overheads when receiving packets.

2.7 Network Interface Card

Network Interface Card (NIC) is also known as a network adapter, or LAN adapter. It is used as an interface between the network and the physical computer. The NIC is often used with ethernet but can also use other protocols. A NIC often comes preinstalled with a computer and is a fundamental part of the computer configuration. Some functions of a NIC are that it sends and transmits network packets. This can be done using a wireless connection or using a cable for example. [7].

2.8 Single Root Input/Output Virtualization (SR-IOV)

SR-IOV stands for single root I/O virtualization and is used for dividing a single physical function such as a NIC into multiple virtual functions that act as network interfaces and can be accessed through applications, as shown in Figure 2.9. SR-IOV makes each virtual function act like it has direct access to the NIC by creating virtual functions instead of using the NIC. This then allows for faster IO since it can bypass the Virtual Machine Manager and the Virtual Switch.[41]

According to Kun et.al [54] SR-IOV is fast since it can bypass some functions of the NIC. SR-IOV scales well and does not use more resources than are needed for the operations. It is common to use SR-IOV for high-performing virtual computing.

A problem with SR-IOV is that even though the overhead is considered low, it is still too high when processing interrupts virtualizations and could therefore possibly present a bottleneck [54].

To use SR-IOV it needs to be supported by the systems NIC. One limitation of SR-IOV is that even though it is in theory possible to have an infinite number of virtual functions, each virtual function will take up a part of the physical resources on the NIC. This means that the hardware of the NIC determines how many and how powerful each virtual function can be [3].

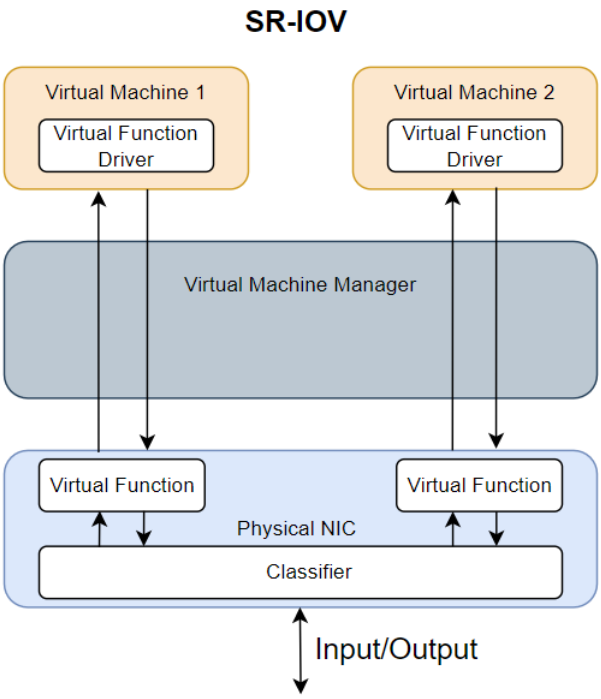

Figure 2.9: Figure shows how one SR-IOV enabled NIC can create virtual functions to handle data coming from two different virtual machines at the same time

2.9 Pktgen

Pktgen is a software-based traffic generator powered by DPDK and can act as both transmitter and receiver at line rate [56].

The packet generation can be managed by configuring what IP addresses, MAC addresses, packet size, and ports Pktgen should use when running it. The user interface of

Pktgen allows for displaying real-time metrics for each port being used, which is useful for analyzing network performance [56].

2.9.0.1 Measuring CPU Cycles with RDTSC

Pktgen uses the x86 instruction RDTSC to get the number of CPU cycles since the last CPU reset [55]. The number returned by the RDTSC instruction is used as a time stamp added to the packet before the packet is transmitted from Pktgen. When Pktgen receives a packet the difference between the added time stamp and the current number of CPU cycles is evaluated. The difference in CPU cycles may then be used to calculate the latency in a suitable time unit.

2.10 Poisson Process and Switched Poisson Process

The Poisson process is a mathematical concept consisting of points randomly distributed over a mathematical space, usually where events happen at a certain rate [15].

The mathematical properties of Poisson processes have led to it frequently being adopted within telecommunications. The Poisson process by itself is not deemed suitable for simulating bursty traffic and it may even show strong long-term correlations [2].

Unlike the Poisson process, the switched Poisson process is very good at simulating real-world bursty traffic. The Switched Poisson Process is a type of Poisson process, where the rate is determined by at least two underlying states. The states relate to each other through a Markov chain where there is a transition rate between states that determines how likely they are to switch to another state. Each state will simulate either a high or low flow of data [2].

2.11 Linux Kernel Tuning

There are plenty of parameters that can be tuned in the kernel that can have an impact on network performance. In this section, some of these are explained.

2.11.1 Receive Side Scaling

When the NIC receives an incoming packet it can apply a filter, usually, a hash function, to distribute the packet to different receive queues to allow for processing to be distributed between CPUs. Every receive queue has an interrupt request (IRQ) related to it which the NIC will trigger once a packet is added to it to notify the CPU that a new packet has arrived and needs processing. The mechanism for distributing incoming packets on different receive queues is known as Receive side scaling (RSS) [25].

If latency is of concern or receive interrupt processing creates a bottleneck, RSS may be useful. To lower latency the optimal setting would in theory be to allocate as many queues as there are CPUs or the number of queues the NIC can support, as spreading the load between different CPUs will decrease the average queue length. If the goal is instead to have a high-rate configuration it is better to have as few receive queues as possible without reaching overflows due to an overloaded CPU [25].

2.11.2 Receive Packet Steering

Logically Receive Packet Steering (RPS) is a software implementation of RSS. Because of this, it is also called later in the data path. Whereas RSS is used to determine the CPU for interrupt handling, RPS is instead used to determine the CPU to handle protocol processing above the interrupt handler. RPS uses hash functions to select the target CPU [25].

The list of CPUs that RPS can forward the protocol processing task to can be configured through a file entry. RPS is more likely to be useful if there are fewer hardware queues than CPUs [25].

2.11.3 Accelerated Receive Flow Steering

Receive Flow Steering (RFS) takes application locality into consideration when using software for steering incoming packets. RFS's goal is to increase the data cache hit rate by steering the kernel processing of incoming packets to the CPU where the application consuming the packet is running [25].

Accelerated Receive Flow Steering (aRFS) offloads flow steering to hardware which should, in theory, perform better than RFS since the packets will be sent directly to a target CPU where the application runs or to a CPU that is close to the CPU that is running the application thread [25].

Other than enabling aRFS no configuration is necessary, as mapping of queues to CPU is automatically set up from IRQ affinities that are configured for each receive queue by the driver on the NIC [25].

2.11.4 Transmit Packet Steering

Transmit Packet Steering (XPS) works as RPS, described in Section 2.11.2, but instead works on transmit steering rather than steering of incoming packets. When configuring XPS there are a few things to keep in mind. If the network device has a single transmission queue there is no point in configuring XPS as it only works for multi-queue systems. If the system is a multi-queue system XPS should preferably be configured so that each CPU will be mapped onto one queue. If the number of queues is fewer than the number of CPUs it may be beneficial to choose CPUs that share the cache with the CPU that is dealing with the processing of transmit completion of a particular queue [25].

2.11.5 ip_early_demux

ip_early_demux is a boolean in the Linux network stack which is used to optimize the processing of incoming packets by reducing the number of demux operations. A demux operation is distributing or delivering outputs from one input. It applies to TCP and connected UDP sockets. Although it is used to optimize input packet processing it may also reduce the overall throughput as it can potentially create additional costs for routing workloads if enabled. If that is the case it should be disabled to not decrease performance [36].

2.11.6 Socket Receive Queue Memory

In the kernel, the rmem_max parameter determines the maximum receive socket buffer size in bytes, while rmem_default is the default setting for the socket receive buffer [20]. Tuning rmem_max can result in decreases in latency and increases in throughput [11], as the parameters control the size of receive buffers used by network applications.

2.11.7 Socket Transmit Queue Memory

Similarly to what was described in Section 2.11.6, the socket send buffer can also be configured by configuring the variables wmem_default and wmem_max at run-time [11]. Adjusting these kernel parameters can yield differences in networking performance.

2.11.8 netdev_max_backlog

netdev_max_backlog represents the maximum number of packets on the network interface's receive queue, where packets are added whenever the network interface receives packets at a faster pace than the kernel is able to process [11]. This is different from rmem_max and rmem_default since it controls the number of packets that can be queued on the NIC and does not affect the network applications' receive buffers.

Increasing the maximum number of packets the network interface receive queue can hold should reduce the risk of the queue becoming full and dropping packets, thus the packet loss rate should decrease when netdev_max_backlog is increased [17].

However, it is important to note that it can have unexpected side effects on cache and memory paging. If the throughput is too high it will not be useful as the queues will fill up and packets will continue to be dropped.

2.11.9 Generic Receive Offloading

Generic Receive Offloading (GRO) is a software offloading technique that is used for reassembling small packets into larger packets. This will allow applications using GRO to process fewer larger packets, thus it should reduce packet processing overhead [45].

2.11.10 netdev_budget

netdev_budget is used when the data received by the kernel comes in faster than the speed at that the kernel can handle the requests [29]. The netdev_budget goal is to minimize soft interrupt queue misses. The value that is set to netdev_budget is the number of requests that the kernel handles before leaving the CPU. This can be increased if each soft interrupt did not get enough CPU time.

2.11.11 RX Flow Hash Indirection Table and Hash Function

To determine which queue an incoming packet should be assigned to, the driver uses an RX flow hash indirection table that assigns hashed packet headers to determine where the packet should be queued. The default configuration is to distribute incoming network traffic evenly between queues, but this can be configured at run-time by either setting weights for queues or by configuring the queues available in the RX flow hash indirection table. The results of this would be that some queues will receive more packets than others, which affects performance depending on how the queues are mapped to CPUs [25].

The hash function used by RSS to hash incoming packets headers to determine what queue to choose in the indirection table is usually a Toeplitz hash [25], but the hash function can be configured to run hash functions other than Toeplitz.

2.11.12 Hardware Buffer Queue

The hardware buffer queue can be increased in size to reduce the number of packets being dropped as it will not overflow as easily. This can be done by using the ethtool Linux utility to modify the RX and TX ring parameters that represent the size of the hardware ring buffers [38].

3 Related Work

This chapter describes some of the most relevant works which are related to the topic of this thesis through either the research questions, methodology, or the topic itself. Although no direct comparison has been conducted between DPDK and Linux kernel with io_uring to the best of the authors' knowledge, there have been comparison studies made on other packet processing frameworks. There are also several studies investigating the performance aspects of DPDK.

3.1 Comparison of Frameworks for High-Performance Packet IO

Due to the implementation of network stacks in operating systems, expensive overheads are generated when handling packets and the CPU often becomes the bottleneck in regard to packet handling [22]. Software frameworks for high-speed packet handling such as Intel DPDK, Netmap [50], and PF_RING ZC [39] claim they are the solution by offering various alternatives to the Linux network stack. Some of the main techniques used by these frameworks to accelerate packet processing are bypassing the default network stack in the kernel, using polling for receiving packets instead of interrupts, preallocating packet buffers when an application starts, and avoiding copying between user space and kernel space.

Gallenmüller et al. [22] put these software frameworks to the test by introducing a new model specifically designed for estimating the performance of high-speed IO frameworks. The model uses limitations and potential bottlenecks for determining an upper bound of the packet processing system capabilities. Some of the most significant limitations and bottlenecks that were studied were the maximum transfer rate of the NIC, RAM restricting network bandwidth, and how the CPU can lower the transfer rate when certain packet processing algorithms are performed. Using these bottlenecks and limitations the upper bound could be determined. During their experiment, they studied several factors such as transmission efficiency, the influence of caches, batch sizes, and latency.

Gallenmüller et al.[22] found that the high-speed packet IO frameworks did offer an increase in performance. The increase was primarily a result of preallocated buffers, processing in batches, and avoiding interrupts. The results were also aligned with the model predictions which indicate that the model worked as planned. According to the results of the experiment, there was a trade-off between latency and throughput depending on the queue sizes.

Studying mere latency figures and performance the results seemed to suggest that DPDK and PF_RING ZC are the superior choices over Netmap.

3.2 Understanding Modern Storage APIs: A systematic study of Libaio, SPDK, and io_uring

In a study executed by Didona et al. [18] comparative research was conducted to compare io_uring, which was created to fill the void of high-performing storage stacks, to some older more well-known APIs: libaio and SPDK. The comparison between io_uring and the other API concluded that when io_uring is used correctly it can get an equal amount of performance as SPDK if the hardware is powerful enough. Even though it is possible to get the same performance as the SPDK the article concludes that io_uring needs more CPU cores to be available to reach that performance. The article also states that when using io_uring it is important to understand when io_uring is preferable to use and that io_uring is sensitive when it comes to tuning it to perform at its best. Didona et al. [18] also found that polling design also is an important factor regarding the performance of a system. This was concluded since the tests were executed on three different polling settings on io_uring. To test the performance the article used Flexible I/O Tester (fio) [6] to create the workload to test the performance. Overall the article concluded that the SPDK does outperform io_uring when performing at its best. Lastly, the article points out that io_uring is compatible with sockets and that there are areas of future studies that could cover how io_uring performs in a network.

3.3 Power Efficient High Performance Packet I/O

A study performed by Li et al. [35] was conducted to find a solution for how to make high-performing IO systems consume less power. The article explains that for systems to reach a powerful amount of packet processing, polling is often used. The problem with polling is that it makes the CPU busy even if there are no packets to handle, which is a waste of power. The article conducted the tests on the DPDK, which has the ability to possibly decrease the latency of packet processing using different approaches built into the framework. These techniques are busy polling, zero-copy, and batch processing. Li et al. [35] found two ways to decrease power consumption but still maintain a high-performance output when studying high throughput and latency. The first approach is to use historical traffic information to help predict and adjust the CPU to lower frequency when possible. The other solution stated by the article is to regulate CPU frequency by counting the empty polling number.

3.4 A Study of Tools to Develop a Traffic Generator for L4 - L7 layers

In the paper by Mahalakshmi et al. [1], tools required to create a software-based traffic generator for network layers L4-L7 in the OSI model were described. The main goal of the paper was to provide a complete high-performance framework for packet processing in data plane applications. This was done through the usage of existing tools on the market for building traffic generators and then comparing them against various parameters including scalability, reliability, and speed in terms of saturation. Several well-established tools were analyzed such as TRex, Pktgen, MoonGen, and Ostinato. The paper concluded that Pktgen and MoonGen were the most efficient in obtaining the goal of building a traffic generator for the L4-L7 layers, where efficiency is defined as the speed and effectiveness of the packet processing.

3.5 Understanding Host Network Stack Overheads

Cai et al. [13] state that in recent years, the traditional end-host network stacks have mostly remained stagnant while data center link bandwidths have increased by a factor of 4-10 times over the past few years. As a result, designing CPU-efficient network stacks have become a significant issue as the more traditional approaches to solving the problem are no longer as effective. To address the challenges various solutions have been explored, including optimization of the Linux network stack by reducing the amount of packet processing required for packet handling.

Cai et al. [13] found that the bottleneck of the network stack has started to shift from packet processing to data copy, where more than 50% of the CPU cycles on the receiver is used for data copy within a single flow. This is opposed to the common belief that packet processing uses most of the CPU cycles however, it might still be true for short flows. Zero-copy protocols could potentially alleviate these kinds of bottlenecks.

The NIC DMA pipeline has become less efficient, there is a throughput difference of 24% depending on if the pipeline is configured with optimal versus default parameters. When configuring the pipeline the software and hardware parameters such as window size, and NIC Rx descriptors much be carefully determined to optimize efficiency.

Network flows sharing resources on a single CPU core could decrease the throughput by 66%, thus host resource sharing is considered bad if optimal network performance is the goal. To avoid harmful host resource sharing, receiver-side scheduling of host resources and network-aware CPU scheduling should be taken into consideration.

4 Method

In this chapter, the methodology for investigating the research questions in Section 1.3 is described. The chapter is divided into nine parts. Section 4.1 shows the hardware and software setup used when conducting the study. Section 4.2 covers the specific architecture used for each of the applications. Section 4.3 describes the equations and calculations that were used to determine the network metrics. Section 4.4 explains which additions had to be made to the source code to plot latency as a histogram. Section 4.5 gives a definition of how the optimal transmission rate was determined. Sections 4.6, 4.7, and 4.8 describe the methodology used for investigating research questions 1, 2, and 3 respectively.

4.1 Setup

In this section, the setup for transmitting and receiving data packets between a traffic generator and an application is presented. It consists of one hardware setup in combination with a software setup.

4.1.1 Hardware Setup and Specifications

The hardware setup was based on the available resources when conducting the study. This includes a computer system that was representative of a server handling networking traffic with high networking performance. Below follows the specifications and features of the computer system.

- **Network interface card**: Intel Ethernet Controller XL710 with dual port, Ethernet speed of 40 GbE (Gigabit Ethernet), and transceiver module QSFP+ 1583.
- **CPU**: Intel Xeon E5-2680 v4 and 28 physical cores of which each can run one thread per core running at a clock speed of 2400 MHz. The dynamic frequency scaling, sometimes also referred to as Turbo boost, was disabled to allow for more consistent results.
- **RAM**: Eight 16 GB DDR4 2400 MHz modules, totaling 128 GB of RAM.

4.1.2 Software Setup and Configuration

The software setup contains versions of software used and configurations on the computer system. The NIC uses the network driver iavf.

4.1.2.1 Versions

Below follows a list of the library and system versions used for the study. The choice of versions was a compromise between novelty but also compatibility between different software.

- **Linux kernel**: 6.1.9-060109-generic
- **Operating system**: Ubuntu 22.04 LTS
- **DPDK**: 23.03.0-rc0
- **Pktgen**: 22.07.2
- **liburing**: 2.4

4.1.2.2 Configurations

This section describes the necessary configurations for doing the performance testing.

Memory hugepages were enabled in order to run DPDK. The configuration of hugepages includes creating 60 hugepages where each hugepage was configured to be 1 GB. The basis of this configuration was the DPDK quick start guide [49] and ensuring that hugepage memory was not a limiting factor.

The NIC contains two Physical Functions (PF). One of the two PFs was configured to be running two Virtual Functions (VF) by using SR-IOV, where one VF was assigned an IP address to allow for sending and receiving data traffic.

The MTU for the network interface for testing io_uring and the Linux network stack was set to 9000 Bytes. This was to enable all possible sizes that were transmitted in this study to be able to pass the MTU.

The default configuration of the system was that CPU cores with even numbers were isolated and shared the same NUMA node. CPU cores with odd numbers were not isolated and share the same NUMA node.

4.1.3 Network Traffic Generator Configuration

The configurations that were made to the network traffic generator Pktgen are presented in this section. As Pktgen was powered by DPDK, one of the VFs on the NIC was bound to the vfio-pci driver, which enables the VF to run Pktgen.

Pktgen was configured to be running IPv4 in the Internet layer and UDP in the Transport layer. The packet rate that the NIC uses was set to 100% meaning it was running at its full capacity. Finally, Pktgen was configured to use either a MAC address or an IP address and port as the destination.

4.1.4 Base Setup

There were some similarities between the setups that were used for DPDK, io_uring and the standard Linux network stack. These similarities will hereafter be referred to as the base setup. As mentioned earlier, even-numbered CPU cores were isolated and run on the same NUMA node. To get performance benefits and consistent results only even CPU cores from the same NUMA node were chosen when running Pktgen and the forwarding applications.

The base setup contains Pktgen which uses the network driver iavf. SR-IOV was used on the NIC to enable the use of virtual functions, as it was the only NIC that was available during

the study for testing and therefore the use of virtual functions was a necessity for conducting the experiments.

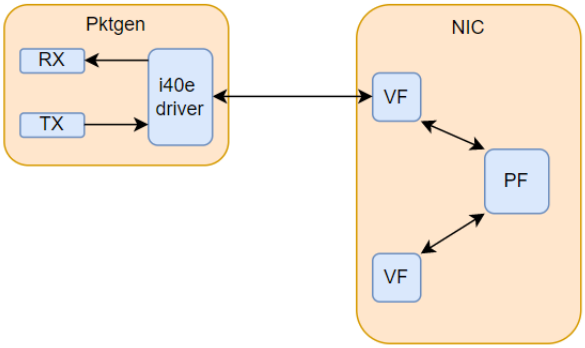

Figure 4.1: Base setup used for testing DPDK, io_uring and the standard Linux network stack

Pktgen will generate the packets and transmit them to a virtual function on the NIC. The virtual function will then transmit the data to the physical function that forwards the packet to another virtual function. The virtual function will later be connected to an application that utilizes DPDK, io_uring or the standard Linux network stack for packet processing and forwarding. In Figure 4.1 the architecture of the setup was shown and the arrows represent the flow of data between the components.

For measuring the networking metrics the setup relies on the data packets reaching the forwarding application which then forwards the packets back to Pktgen where they can be used to calculate network metrics.

4.2 Architecture

In this section, the architecture that was used for DPDK, io_uring, and the standard Linux network stack was described.

4.2.1 Setup Using DPDK

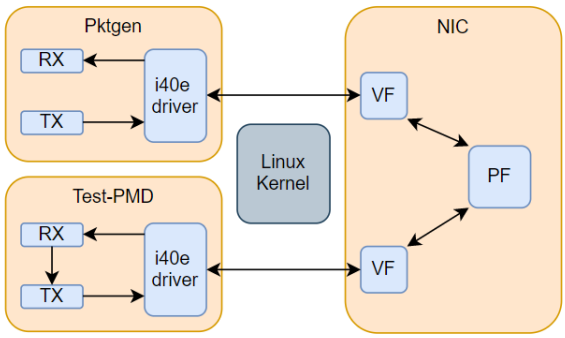

Figure 4.2: Setup for testing DPDK with SR-IOV

The DPDK application revolves around the setup of how the data was sent, DPDK does not use the kernel at all as shown in 4.2. The application revolves around Pktgen sending data from an i40e driver directly to an SR-IOV-enabled NIC with virtual functions that capture the traffic. The traffic was then passed through to the Physical function and then to another virtual function that was connected to the TestPMD. The packets were then sent to TestPMD and forwarded back to the same path that the packet came from.

4.2.2 Setup Using io_uring

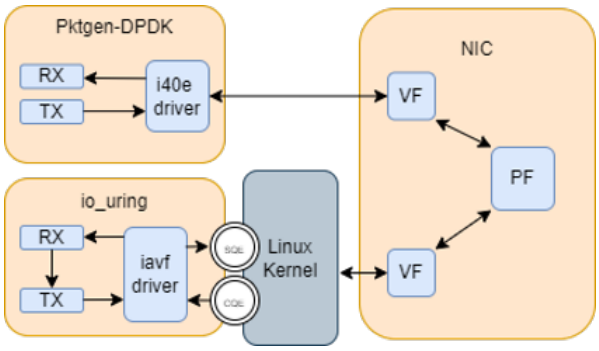

Figure 4.3: Setup for testing io_uring with SR-IOV

Smaller changes were required for setting up the io_uring application for forwarding, compared to the DPDK setup. DPDK avoids the usage of the kernel whilst io_uring removes some of the overheads of using the kernel. As shown in Figure 4.3, the same base setup was used for DPDK. Pktgen sends packets to the virtual function of SR-IOV-enabled NIC. From there the packets were sent to the kernel and from the kernel to the io_uring application, ring buffers were used instead of doing an ordinary system call. The system calls that were coming from the kernel to the application was added to the Submission Queue Entry (SQE) from where the io_uring applications get the packets. When the packets were sent back, they were placed in the Completion Queue Entry (CQE) from which the packets were taken by the kernel and sent to the NIC.

4.2.3 Setup Using Linux Network Stack

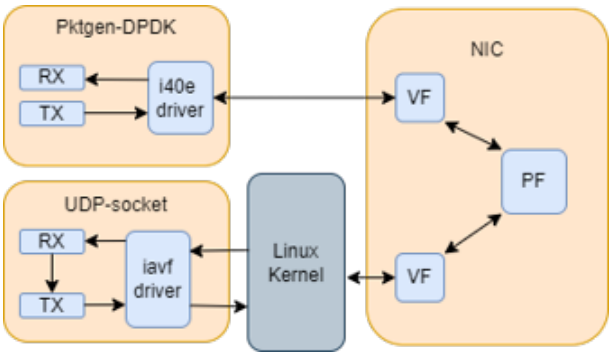

Figure 4.4: Setup for testing UDP socket using Linux network stack with SR-IOV

The standard Linux kernel stack application was the default way of sending packets in a Linux system as shown in Figure 4.4. The base setup was the same from where the packets were sent from Pktgen to the SR-IOV enabled NIC and then sent to the kernel. The kernel handles the packets one by one and creates a system call from the kernel to the application for each packet that was sent from the kernel. The packets were handled in the application using an ordinary UDP socket that then forwards the packets back to the kernel and then back to Pktgen.

4.3 Calculations

Pktgen has built-in functionality for counting the number of incoming and outgoing packets that can be used for measuring packet rate (pps), packet loss, and calculating the throughput (bps).

The packet rate was calculated by dividing the number of packets by the time.

$$\text{Packets per second (pps)} = \frac{\text{Incoming packets}}{\text{Duration}} \tag{4.1}$$

Packet loss can be calculated by subtracting incoming packets from outgoing packets and dividing this difference by the total number of outgoing packets. Multiplying by 100 gives the packet loss in percentage.

$$\text{Packet loss (\%)} = \frac{\text{Outgoing packets - Incoming packets}}{\text{Outgoing packets}} \times 10^2 \tag{4.2}$$

There was lacking support in Pktgen for calculating the average throughput in Mbits/s. However, as the number of incoming packets and packet payload size was known it can be calculated. First, the overhead of the Ethernet protocol added to each packet must be calculated:

$$\begin{aligned}\text{Packet overhead size} = {} & \text{interframe gap} \\ & + \text{start frame delimiter} \\ & + \text{preamble} \\ & + \text{length of ethernet CRC}\end{aligned} \tag{4.3}$$

The start frame delimiter was the bit sequence that indicates the beginning of the Ethernet frame, interframe gap was the time interval between Ethernet frames, and preamble was a sequence of bits used for synchronization purposes of the data transmission. With the packet overhead size, it was possible to calculate bits per frame.

$$\text{Bits per frame} = (\text{Packet size} + \text{Packet overhead size}) \times 8 \tag{4.4}$$

Once bits per frame were known, Mbits/sec, which was the throughput, can be calculated with the following equation:

$$\text{Throughput Mbits/s} = \frac{\text{Incoming packets} \times \text{Bits per frame}}{10^6 \times \text{Duration}} \tag{4.5}$$

Pktgen also has support for measuring the minimum, average, and maximum latency which was done using a special flag for enabling the measurement of latency values in microseconds. For measuring these values no further calculation was needed and the latency values were returned.

4.4 Changes to Source Code

When investigating latency values, it was often not enough to look at the minimum, average, and maximum latency values over a given time period. It was more valuable to look at the individual latencies of each packet through a histogram. After enabling the measurement of latency in Pktgen packets were sent out containing a magic number indicating they have not been altered and may be used for latency measurement.

With this in mind, changes in the source code were made to allow the measurement of each packet's latency. The latency values were stored in a hash table allowing for plotting as a histogram. As Pktgen uses the x86 RDTSC instruction to generate time stamps of when the

packet was sent and received, the latency was represented as CPU cycles. To convert the CPU cycles to microseconds the following formula was used:

$$\text{Latency } \mu\text{s} = \frac{\text{CPU Cycles} \times 10^9}{\text{Frequency RDTSC} \times 10^3} \tag{4.6}$$

Frequency RDTSC was the frequency of the RDTSC timer.

4.5 Determining Optimal Transmission Rate

Determining at what transmission rate an application performs optimally and starts to bottleneck was important to know to be able to do a fair comparison between how the applications compare to each other.

The maximum transmission rate of the NIC was 40 Gbps. As the transmission rate was high, the likelihood of the applications creating a bottleneck was considered high. To determine at what transmission rate an application has reached its optimal performance there were a few indicators to take into account.

An application indicates to have reached its optimal performance at a certain transmission rate from the NIC when there was no increase in throughput and/or packet rate after increasing the transmission rate further. Another indication that the optimal transmission rate of the application has been reached was when the packet loss and latency start to increase while the throughput stagnates.

4.6 Methodology for Benchmarking DPDK, io_uring and the Standard Linux Network Stack

In this section, the approach and methods used to benchmark the network performance of DPDK, io_uring, and the standard Linux network stack were described. The benchmarking of the packet processing solutions was related to research question 1.

To be able to compare io_uring and DPDK with the standard Linux network stack, three different setups were used to represent each of them. Although the base setup will remain the same for all, the applications will differ.

The purpose of each application was to receive packets from Pktgen and forward the same packet back to Pktgen by changing the MAC and/or IP addresses. Pktgen will count the number of received packets and gather the information contained within each incoming packet. The information was turned into statistics that can then be used to calculate the network metrics.

4.6.1 Applications

Various applications can be used for packet forwarding. In this section, the applications that were used for packet forwarding using DPDK, io_uring, and the standard Linux network stack were explained.

4.6.1.1 Application Using DPDK

The application that was used for testing the networking performance of DPDK was TestPMD. The reasoning behind using this application was that the application has a packet forwarding mode and was included in the DPDK package. It was described in the DPDK documentation as a staple application for doing general performance testing of DPDK, thus indicating that it has been optimized by maintainers of DPDK for the purpose of testing its networking capabilities.

4.6.1.2 Application using io_uring

In the liburing repository on GitHub there was a sample application called io_uring-udp.c, which has been designed for packet forwarding UDP packets.

It was not developed with high performance in mind and does some unnecessary processing of received packets. However, it shows off a technique called multishot recvmsg which was known to potentially yield some performance increases. There was also a lack of other examples showing how io_uring with the liburing library can be utilized. Because of this io_uring-udp.c were used as the foundation when building the io_uring application. There will however be some differences between the two. One of the key differences was that the application in this thesis used IORING_SETUP_DEFER_TASKRUN as it was known to give a boost in performance. Another difference was removing unnecessary processing of received packets as much as possible.

The reasoning behind utilizing liburing was that it provides a simplified interface for applications using io_uring, which simplifies the setup process.

4.6.1.3 Application Using Standard Linux Network Stack

For testing the network performance of the standard Linux network stack a UDP socket was created for packet forwarding. It was using interrupts when handling packet processing. There were several ways a UDP socket can be configured. The goal of the UDP socket was to act as a baseline and because of this, it was made as simple as possible for packet forwarding.

4.6.2 Data Collection and Evaluation

The process of collecting the metrics for evaluating the networking performance was comprised of several parts, including scripts for running Pktgen, making additions to the source code, calculations of metrics, and creating graphs for evaluation. The metrics that were evaluated were the following:

- Packet loss rate (%)
- Throughput (bps)
- Packet rate (pps)
- Latency (s)

The equations and calculations that were used for determining the network metrics for each application were available in Section 4.3.

4.6.2.1 Scripts

To be able to benchmark the packet processing performance in a fair way, two scripts were created to allow testing of DPDK, io_uring, and Linux network stack under the same circumstances. The scripts were written using Lua as it was the only programming language that was supported for writing scripts using Pktgen.

The first script was used for calculating the network metrics mentioned in Section 4.6.2. The testing was done by testing the packet sizes 64, 128, 256, 512, 1024, 1280, and 1518 bytes together with the transmission rates 1, 2, 4, 8, 16, 32, 64, and 100%. The packet sizes were picked based on packet size intervals in Pktgen where size 64 bytes was the smallest and 1518 bytes was the largest. Packet sizes 128, 256, 512, and 1024 bytes were the starting packet sizes of the packet size intervals in Pktgen. The transmission rate at which the NIC should execute was decided based on the pattern of powers of two. Transmission rates 1 and 100 were selected as they represent the minimum and maximum transmission rates.

This means that for each of the seven packet sizes, there were a total of eight measurements. Each network metric was plotted in a diagram where DPDK, io_uring and the standard Linux network stack will have their own graphs displaying their performance for each metric.

The second script was used to plot a latency histogram of packet size 1518. A packet size of 1518 bytes was chosen as it was the largest packet size that can be sent with Pktgen and larger packet sizes were less likely to cause congestion. This should make the resulting histogram less prone to errors due to congestion and more likely to provide greater clarity on how packet sizes may differ around a latency mean value. To produce a fair comparison between the histogram of DPDK, io_uring, and the standard Linux network stack the transmission rate was chosen in a way that minimizes the chances of congestion errors contaminating the result. This was done by first finding the minimum optimal transmission rate for the three applications, based on the indicators described in Section 4.5 and then cutting the transmission rate in half to lessen any chances of congestion and packet loss occurring that could otherwise alter the final results.

4.7 Simulating and Evaluating Bursty Traffic

This section describes the methodology used to answer research question 2.

4.7.1 Overview

Bursty traffic was used to simulate how DPDK, io_uring, and the Linux network stack would be able to handle real-world data traffic. This was achieved through the use of the Switch Poisson process distributions, which were used to determine the transmission rates in a high and a low data traffic state.

4.7.2 Applications

The applications used for testing how DPDK, io_uring, and the Linux network stack handle bursty traffic were described in Section 4.2.

4.7.3 Data Collection

To simulate the two bursty traffic states of high and low traffic, while at the same time creating a fair comparison of how the applications were able to perform relative to their maximum performance capacity, the mean values of each state in the Poisson distribution were determined based on the applications' optimal transmission rate.

The formula used for Poisson distribution is:

$$P(X = k) = \frac{e^{-\lambda}\lambda^k}{k!} \tag{4.7}$$

- X here was the random variable that represents the number of different events
- λ represents the mean rate of events in the intervals
- k represents the number of observed events in an interval

When investigating how the applications were able to handle bursty traffic data the main interest was seeing how they perform in terms of latency when there were large and small bursts of data. The latency was evaluated by looking at the average latency during bursts of traffic and through a histogram showing all the latency values received during the entire test run of bursty traffic. As the high-traffic state will push the application beyond its limits it will

likely result in packet loss. Therefore, packet loss was also of interest as it will likely have a strong correlation with latency.

The bursty traffic state representing low traffic was based on a Poisson distribution that has a mean value of 10% of the optimal transmission rate of the application. The state representing a high traffic rate was based on a mean value that was 20% above the optimal transmission rate of the application. These values were chosen with the intent of simulating a situation where the bursts of low-traffic data can easily be handled by the application with little to no packet loss, while the high-traffic bursts should represent a heavy load on the system that results in significant packet loss and increase in latency. These values should prove interesting as they will show how much latency and packet loss will increase during heavy loads of the application relative to the application's performance capacity.

The switched Poisson process uses a Markov chain that can be used to simulate a real-world network traffic situation, where it was more common with low bursts of traffic while higher traffic bursts were less common. The Markov chain in the switched Poisson process works by having different probabilities of transitioning to the other state.

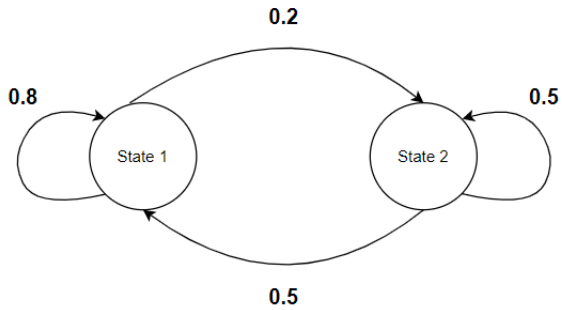

Figure 4.5: Markov chain used for switching states

In Figure 4.5 state 1 represents the low data traffic state while state 2 represents the high traffic state. The probability of staying in state 1 was 80% and 20% to switch to state 2. When in state 2 it was 50% that it will stay in state 2 and 50% it will change to state 1 again.

When doing the simulation of bursty traffic, two Poisson distributions called state 1 and 2, were generated for each application based on the application's maximum performance. After the Poisson distributions were generated burst values were picked from one of the states. The starting state was state 1. Then the Markov chain was iterated over 19 times to pick what the next state would be, totaling 20 burst values.

Lua scripts will then be used to simulate sending bursts of data at the specified transmission rates, where there was a pause of no data traffic between each burst.

4.8 Approach for Tuning Linux Kernel Parameters

To investigate research question 3 some parameters of the Linux kernel need to be tuned. Here follows an explanation of what the purpose was behind the research question and which approaches were used to investigate it.

The underlying goal behind the research question was to try to find the full potential of io_uring. io_uring uses the kernel when sending and receiving data, and therefore tuning the kernel could possibly improve the performance of io_uring. There were several parameters in the Linux kernel that can be tuned. The ones that were focused on in this thesis were described in Section 2.11.

By tuning the kernel a more accurate interpretation of io_uring's maximal performance can be established.

4.8.1 Tuning Linux Kernel Parameters

In this section, the course of actions and commands for tuning the kernel in Linux were described.

There were no clear guidelines as to what the network kernel parameters should be to reach optimal network performance. The goal was therefore to test tuning kernel parameters in such a way that each value should be given a fair opportunity of impacting the network performance. The values selected for testing were based on known facts about the parameter and what its default value was determined to be. It was expected that not all parameters will have the same impact on the performance. In cases where multiple values will have to be tested for the same parameter to find a significant impact on performance, only those values that showed a significant impact were included in the report.

4.8.1.1 Receive Side Scaling (RSS)

If the NIC has support for Receive Side Scaling (RSS, then the number of receive queues can be adjusted using ethtool. The default number of receive queues on the system for the network interface can vary but the goal was the set the number of queues to the number of cores. The following command can be used to set the number of receive queues for a specified network interface.

```
ethtool -L <NETWORK INTERFACE> combined <RECEIVE QUEUES>
```

RSS will not be configured in this thesis since a delimitation was to only use one data flow and RSS requires applications to be run in parallel therefore it could not be tested in this study.

4.8.1.2 Receive Packet Steering (RPS)

One way of tuning Receive Packet Steering (RPS) was writing to the smp_affinity_list of a certain IRQ number to specify what CPUs were allowed for the IRQ source. This was done by first investigating what IRQ numbers were related to the network interface with the following command:

```
egrep "CPU|<NETWORK INTERFACE>" /proc/interrupts
```

After running the command a number of IRQ numbers will show. When these were known it was possible to set only certain CPUs allowed for handling interrupts coming in on these interrupt lines. Thus, RPS can then be configured using:

```
echo <CPUs> > /proc/irq/<IRQ#>/smp_affinity_list
```

Where <CPUs> was a comma-separated list of zero-indexed CPU core numbers.

4.8.1.3 Accelerated Receive Flow Steering (aRFS)

For Accelerated Receive Flow Steering (aRFS) to work RFS must be enabled and configured, and for RFS to be enabled RPS must be enabled and configured. After that CONFIG_RFS_ACCEL needs to be enabled at compile time and ntuple support needs to be enabled for the device. Once these configurations were enabled and configured aRFS should be used automatically.

4.8.1.4 netdev_max_backlog

To increase the maximum number of packets on a network interface's receive queue sysctl was used. As default netdev_max_backlog was set to 1000 on the system. Increasing this value to 5000 may yield some difference in performance. The change was made using the following command:

```
sysctl -w net.core.netdev_max_backlog=5000
```

4.8.1.5 ip_early_demux

To adjust the boolean ip_early_demux sysctl was used. To enable early demultiplexing of incoming packets, ip_early_demux sysctl must be set to 1. This can be done by running the following command:

```
sysctl -w net.ipv4.ip_early_demux=1
```

4.8.1.6 Socket Receive Queue Memory

To increase the socket receive queue memory size the values of rmem_default and rmem_max must be increased. As default, both of the parameters were set to 212992 on our system. To see how tuning socket receive queue memory can affect performance multiple values of various sizes were tested to see how the results may vary. Running the following commands will update rmem_default and rmem_max respectively:

```
sysctl -w net.core.rmem_default=<VALUE>
sysctl -w net.core.rmem_max=<VALUE>
```

4.8.1.7 Socket Transmit Queue Memory

The socket send buffer was increased by using ethtool to configure the wmem_default and wmem_max values. By default, both parameters were set as 212992 on our system. Testing was done for various values to see what kind of performance differences it may yield. The values were changed with the following commands respectively:

```
sysctl -w net.core.wmem_default=<VALUE>
sysctl -w net.core.wmem_max=<VALUE>
```

4.8.1.8 Generic Receive Offloading (GRO)

The default on the system that was used was to have Generic Receive Offloading (GRO) turned on. To disable the software offloading technique GRO and see what kind of impact it will have on performance will require ethtool to be used to manage the networking interface. To turn off GRO the following command were used:

```
ethtool -K <NETWORK INTERFACE> gro off
```

4.8.1.9 netdev_budget

The netdev_budget parameter was set to 300 as default on our system. Increasing the netdev_budget should allow for an increasing number of packets that the network interface can process at one time and thus, potentially increase performance. The netdev_budget parameter was set to double the size with the following command:

```
sysctl net.core.netdev_budget=1500
```

4.8.1.10 RX Flow Hash Indirection Table and Hash Function

To see how much impact tuning the indirection table can have on performance the most extreme case was tested, which was when the indirection table only maps to one ring. The default on the system was to distribute packets evenly between four RX rings. Through the following command, the indirection table will only map to one RX ring:

```
ethtool -X <NETWORK INTERFACE> equal 1
```

4.8.1.11 Hardware Buffer Queue

The default hardware setting for RX and TX ring buffers on the system was 512 bytes, but the maximum ring parameters were 4096 bytes. To maximize RX and TX ring buffers the following command was run:

```
ethtool --set-ring <NETWORK INTERFACE> rx 4096 tx 4096
```

4.8.2 Data Collection and Evaluation

The data collection and evaluation follow the same flow as was described in the methodology for research question 1.

5 Results

This chapter presents the results from applying the methodology and is divided into three main sections, each section representing the results of one of the research questions in the same order as previously shown.

The code that was used to investigate each research question is available in Appendix Code.

5.1 Benchmarking DPDK, io_uring and the Linux Network Stack

To gain a better understanding of how the different packet processing approaches compare to each other benchmarking was conducted measuring latency, throughput, packet rate, and packet loss rate. This section covers the results of each application's network performance related to research question 1.

5.1.1 Absolute Transmission Rate

To give a better understanding of the absolute packet transmission rate a sample was measured for packet size 1518 bytes and for all sending rates.

Transmission rate (%)	1	2	4	8	16	32	64	100
Transmission rate (Gbit/s)	0.4	0.8	1.6	3.2	6.4	12.8	25.6	39.5

Table 5.1: Transmission rate based on packet size

5.1.2 DPDK

This section will show the results for DPDK regarding research question 1.

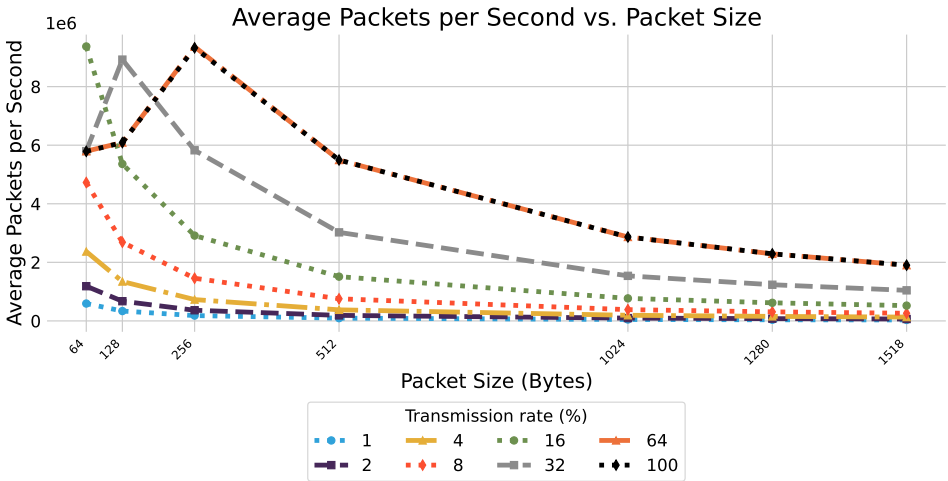

Figure 5.1: Packet rate of different rates and different packet sizes for DPDK

In Figure 5.1 the average packet rate decreases for large packet sizes. When packets are sent with a size of 256 bytes the packet rate spikes for transmission rate 64% and for transmission rate 100%. For transmission rate 32% the maximum packet rate happens for size 128. From rate 1 to rate 16 the trend looks the same where packet rates are higher for lower packet sizes, which then decrease as the packet size increases, and for rates 32%, 64%, and 100% the results differ from the rest. The overall trend is that larger transmission rates generally have a higher average packet rate and the average packet rate decreases as the packet size increases.

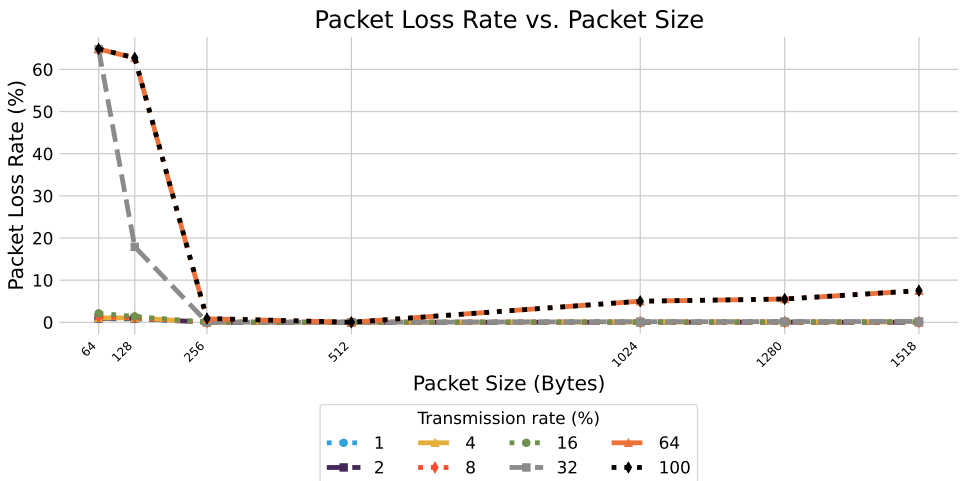

Figure 5.2: Packet loss for different rates and packet sizes for DPDK

Figure 5.2 shows the percentage of packets lost based on the transmission rate and packet size. For transmission rates of 16% and under the packet loss rate is around 0%. For transmission rates 32% and above, the packet loss is initially high, reaching up to 65%, but then sharply decreases to around 0% in packet loss. For the highest packet transmission rates 64% and higher the packet loss rate for packet sizes 64 and 128 bytes is around 65% but then quickly slopes down to 0%.

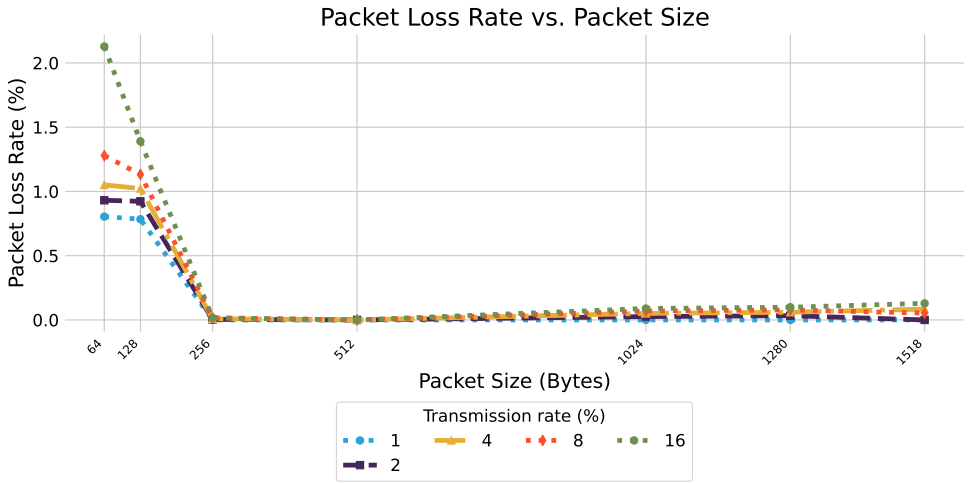

Figure 5.3: Packet loss excluding transmission rates 32, 64, and 100% for DPDK

Figure 5.3 is the same as Figure 5.2 but with transmission rates 32, 64, and 100% excluded. From the figure, it is clear that the packet loss rate is higher for higher rates, as a transmission rate of 16% has the most packet loss for smaller packet sizes. From packet size 256 bytes and larger, the packet loss rate is close to 0%. It should also be mentioned that the packet loss rate increases slightly for the higher packet transmission rates 4, 8, and 16% when reaching packet size 1518 bytes.

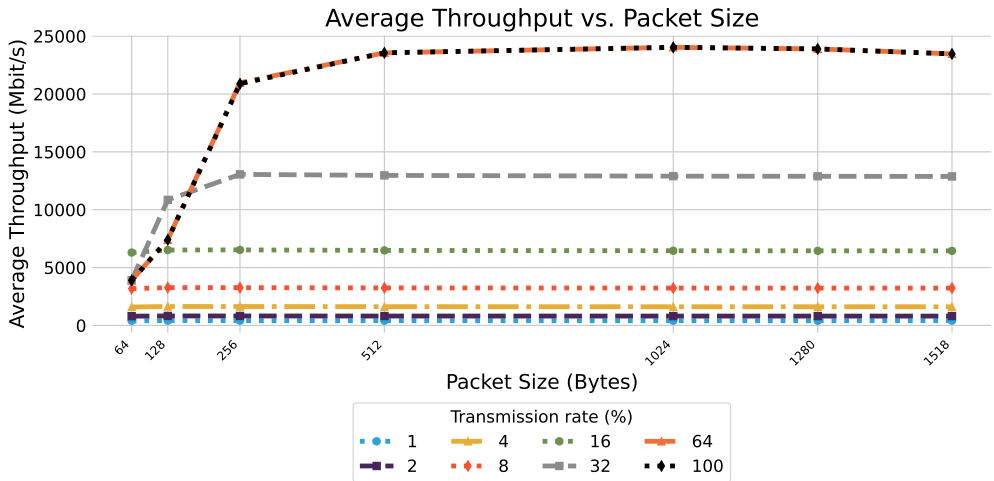

Figure 5.4: Throughput of different rates and different packet sizes for DPDK

In Figure 5.4, the trend is that for higher rates, the average throughput increases. The average throughput steadily increases until the transmission rate reaches 64%. The average throughput is the same for transmission rates of 64% and 100%. For packet sizes that are 512 bytes and larger the average throughput does not increase and stays the same even though the packet size increases. The throughput reaches a maximum of 25 Gbit/s.

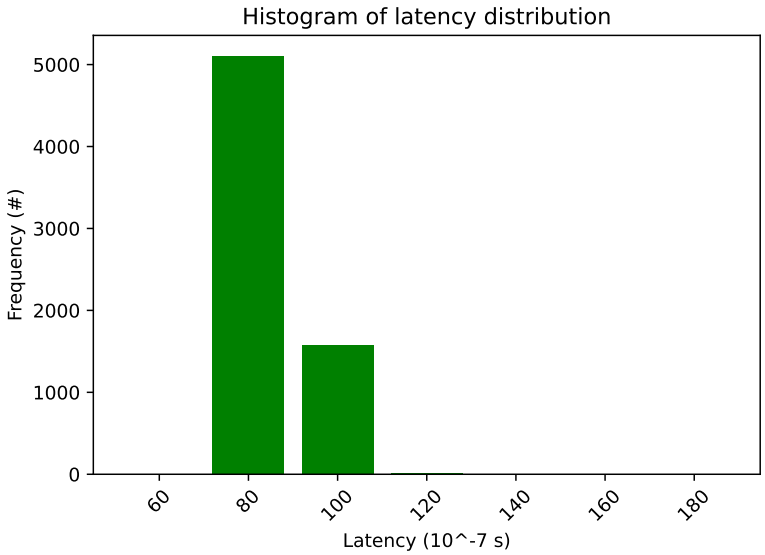

Figure 5.5: Latency histogram for packet size 1518 bytes at transmission rate 8% for DPDK

The latency histogram in Figure 5.5 shows that the majority of packets had a latency of 8μs, with some arriving with a latency of 10μs.

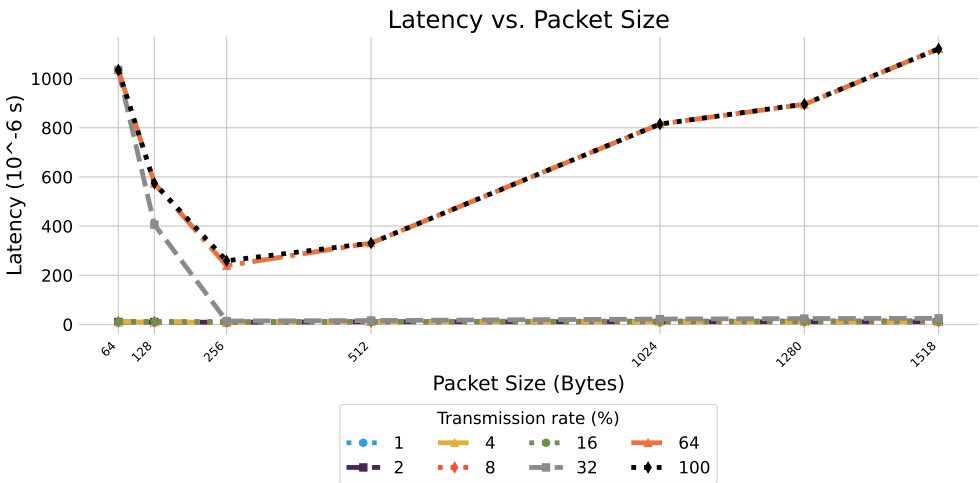

Figure 5.6: Latency for different rates and different packet sizes for DPDK

In Figure 5.6, the latency is generally low and is close to around 10μs even for larger packet sizes. For higher rates, the latency increases significantly. For a transmission rate of 32%, the latency reached a maximum of around 1 ms for a packet size of 64 bytes, but the latency quickly decreased to the same as for lower transmission rates. For transmission rates 64% and 100%, the latency started at around 1 ms and decreased to around 200 μs when reaching packet size 256 bytes and then increased steadily to over 1 ms when reaching packet size 1518 bytes.

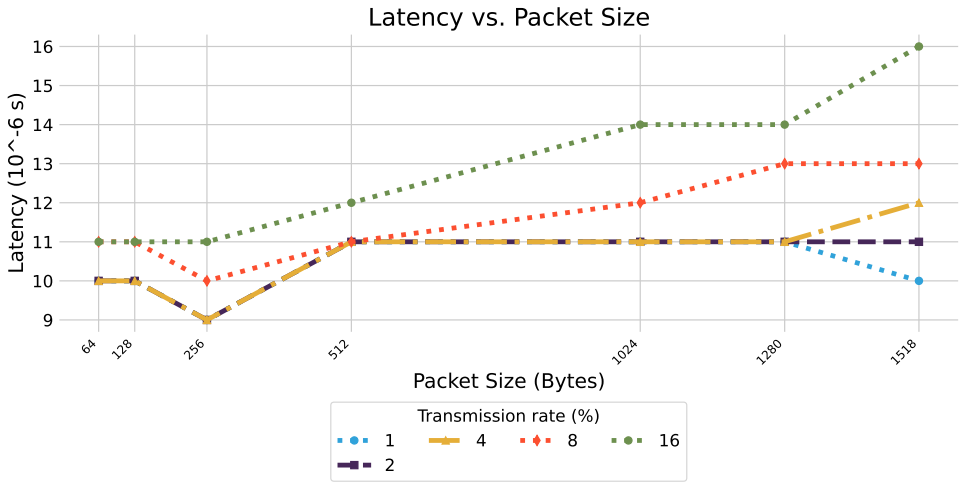

Figure 5.7: Latency excluding transmission rates 32, 64, and 100% for DPDK

Figure 5.7 shows the same values as Figure 5.6, but excludes the higher rates to make the results more clear on how the latency is acting on lower rates. Generally, the latency increases for higher rates as the highest latency is given by rate 16% and close thereafter transmission rate 8%. The overall transmission rates for these rates are revolving around 12 μs but reach a maximum transmission rate of 16% and packet size of 1518 bytes that reaches a latency of 16 μs.

5.1.3 io_uring

This section shows the results for io_uring regarding research question 1.

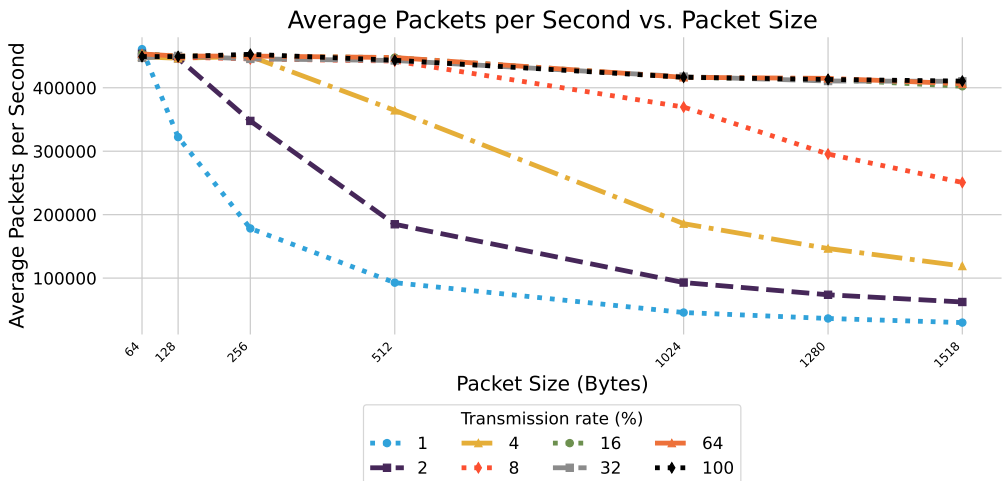

Figure 5.8: Packet rate of different rates and different packet sizes for io_uring

In Figure 5.8, the general behavior is that lower transmission rates and larger packet sizes yield lower average packet rates. From a transmission rate of 1% to 8%, the packet rate gets gradually lower, but for all transmission rates of 16% and over, the average packet rate does not differ much. The minimum packet rate is when the transmission rate is 1% and the packet size is 1518, bytes which is around 30,000 packets per second. The highest packet rate measured is around 450,000 packets per second.

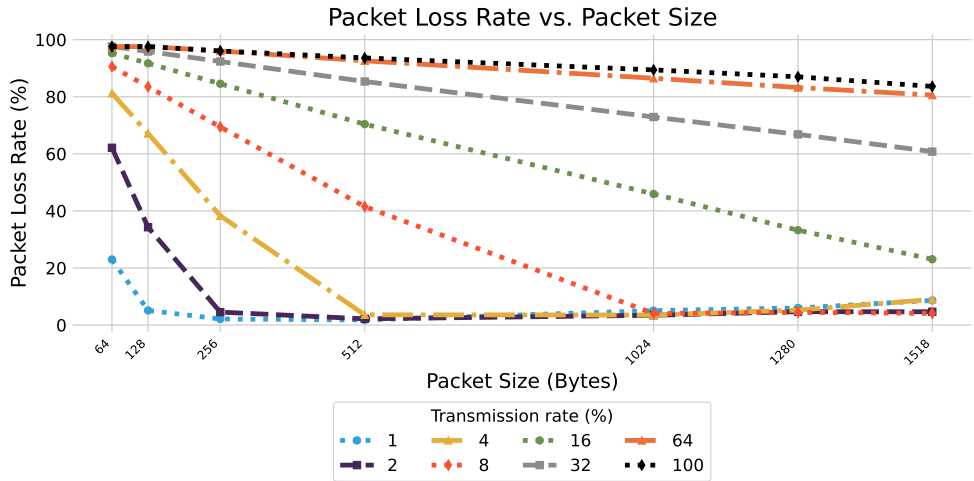

Figure 5.9: Packet loss of different rates and different packet sizes for io_uring

The result in Figure 5.9 shows that the highest packet loss rate is when using a high transmission rate and a small packet size. The packet loss overall is getting lower when the packet size increases. For packet transmission rates of 64% and higher, the packet loss differs from 100% down to 80%. For lower rates, the packet loss is still high for small packet sizes but does decrease significantly down to between 5% and 10% packet loss when reaching larger packet sizes.

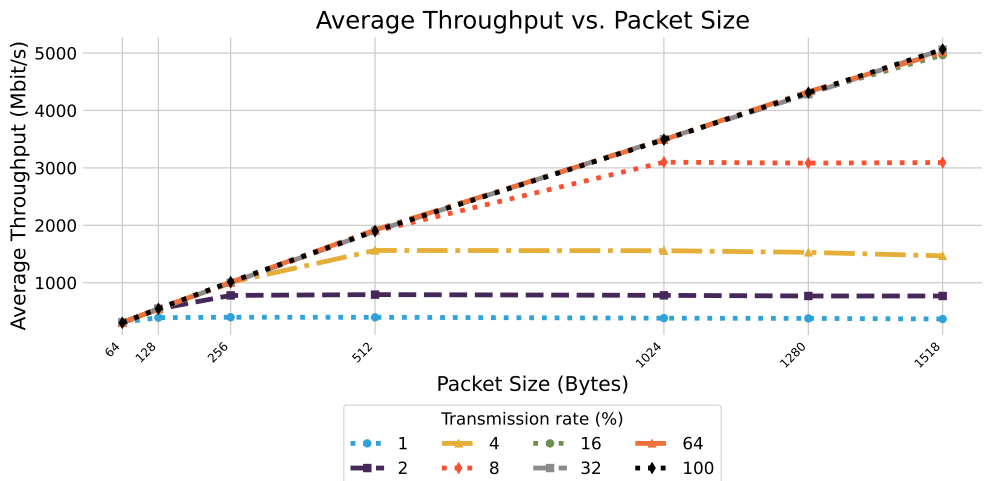

Figure 5.10: Throughput of different rates and different packet sizes for io_uring

Figure 5.10 shows how the average throughput changes when altering the transmission rate and the packet size. The throughput is higher for higher rates but does not increase any further after reaching a transmission rate of 16%. The highest throughput that is reached is 5.0 Gbit/s.

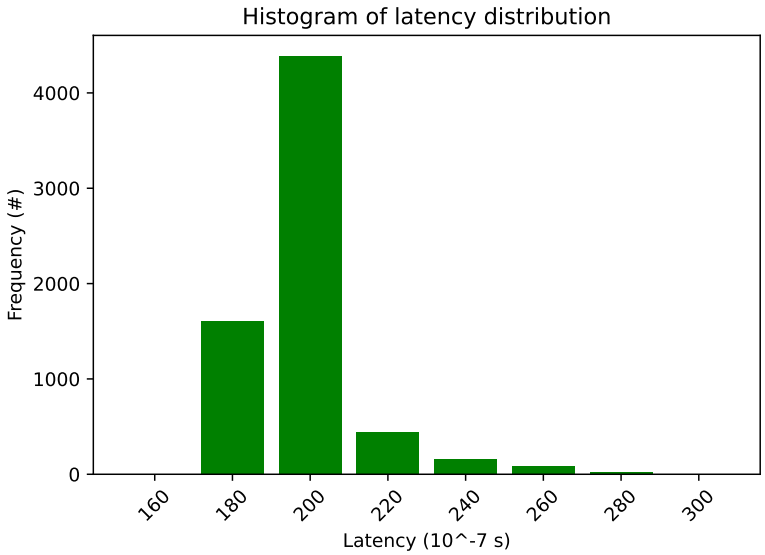

Figure 5.11: Latency histogram for packet size 1518 bytes at transmission rate 8% for io_uring

In Figure 5.11, the majority of packets had a latency of either 18, 20, or 22 μs.

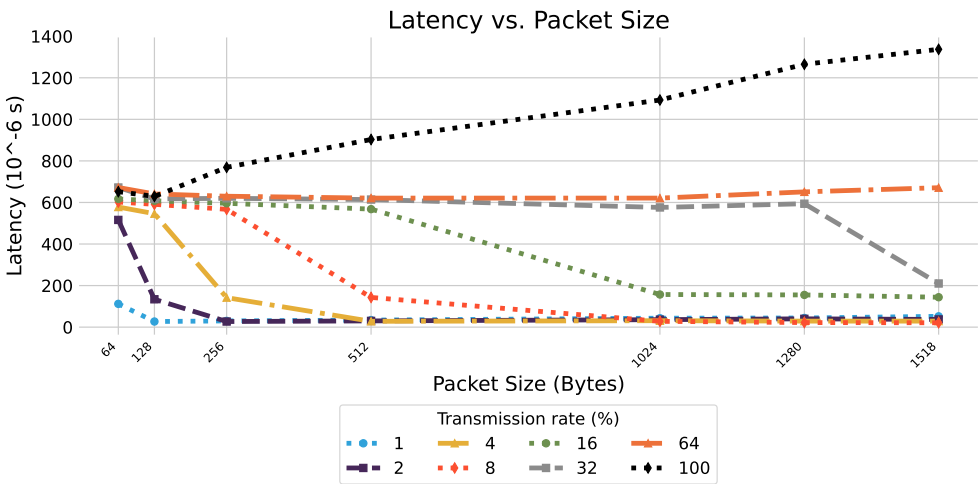

Figure 5.12: Latency for different rates and different packet sizes for io_uring

In Figure 5.12 the relationship between latency, transmission rate, and packet size is visualized. For the transmission rate of 100%, the latency does increase when the packet size increases but for the rest of the rates, the latency either decreases or plateaus. For the transmission rate of 64%, the latency revolves around 6 ms even when the size increases but for the lower transmission rates that are below the latency decreases when packet size increases and reaches a mean latency of around 1 ms.

5.1.4 Standard Linux Network Stack

This section shows the results for the standard Linux network stack regarding research question 1.

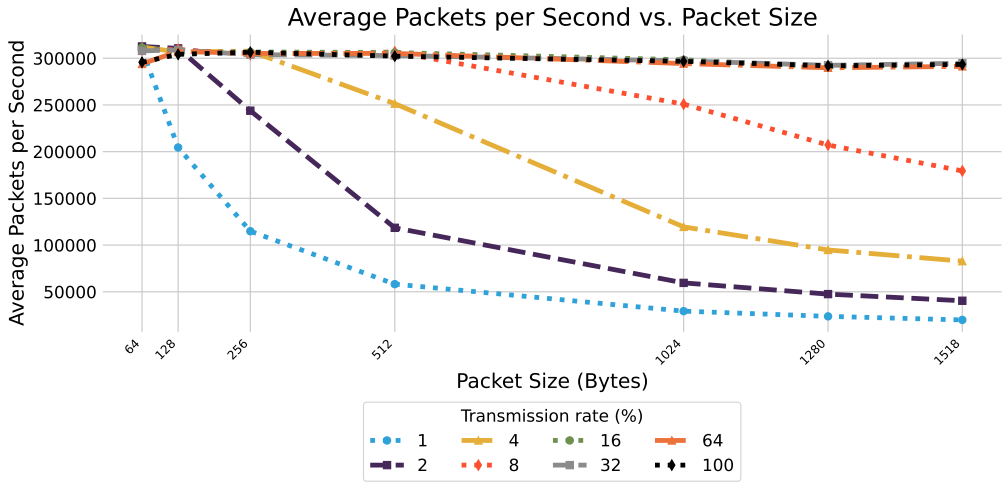

Figure 5.13: Packet rate of different transmission rates and different packet sizes for Linux network stack

In Figure 5.13, lower transmission rates and larger packet sizes yield a decrease in the average packet rate, with a minimum of around 50,000 packets per second. For transmission rates of 16% and higher, the average packet rate stays at around 300,000 packets per second for all packet sizes.

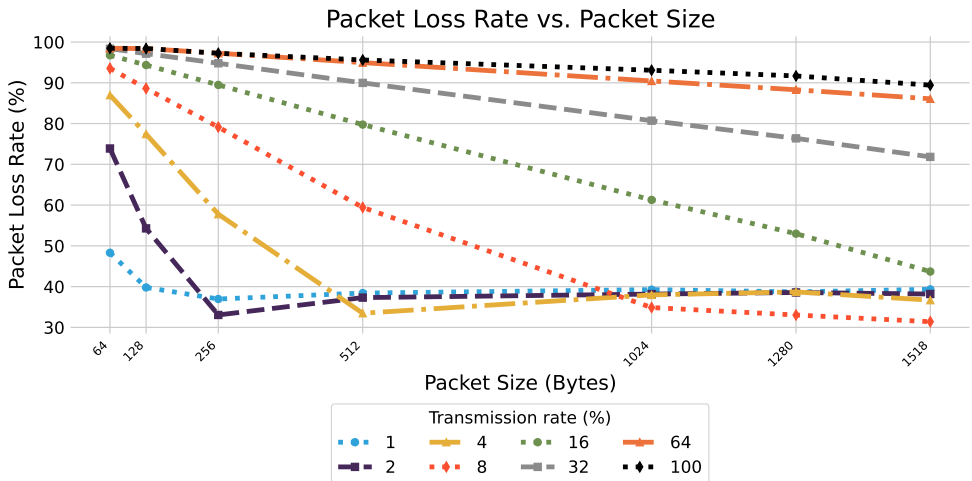

Figure 5.14: Packet loss of different transmission rates and different packet sizes for standard Linux network stack

In Figure 5.14, the packet loss generally decreases for all rates when the packet size increases. For transmission rates of 8% to 100%, the packet loss rate is around 97% for small packet sizes of 64 bytes and 128 bytes. The minimum packet loss for packets of size 64 bytes is 50% when the transmission rate is 1%. For transmission rates over 32%, the packet loss is over 80% for all packet sizes, but for the lower transmission rates, the packet loss decreases down to around 35% and 40% packet loss.

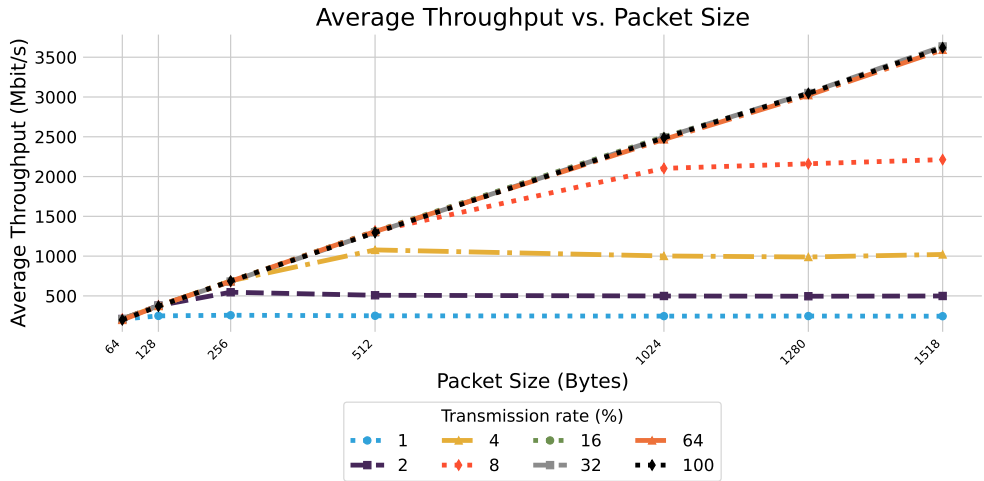

Figure 5.15: Throughput of different rates and different packet sizes for Linux network stack

In Figure 5.15, the throughput increases when the size and transmission rate increase. The throughput does not increase above a transmission rate of 16% and reaches a maximum throughput of around 3.5 Gbit/s. For transmission rates lower than 16%, the throughput decreases in steps until a minimum of around 200 Mbit/s.

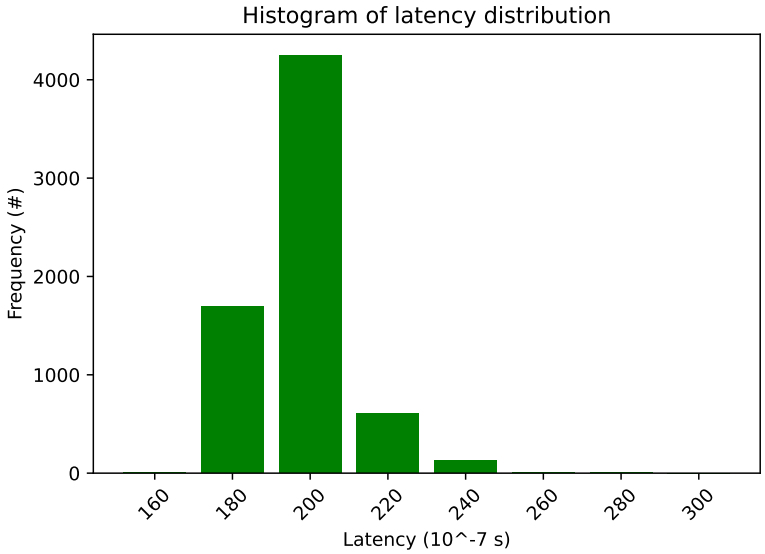

Figure 5.16: Latency histogram for packet size 1518 bytes at transmission rate 8% for Linux network stack

In Figure 5.16, the packet size is 1518 bytes and most of the packets had a latency of 20 μs but also had a spread that reached between 18 and 24 μs.

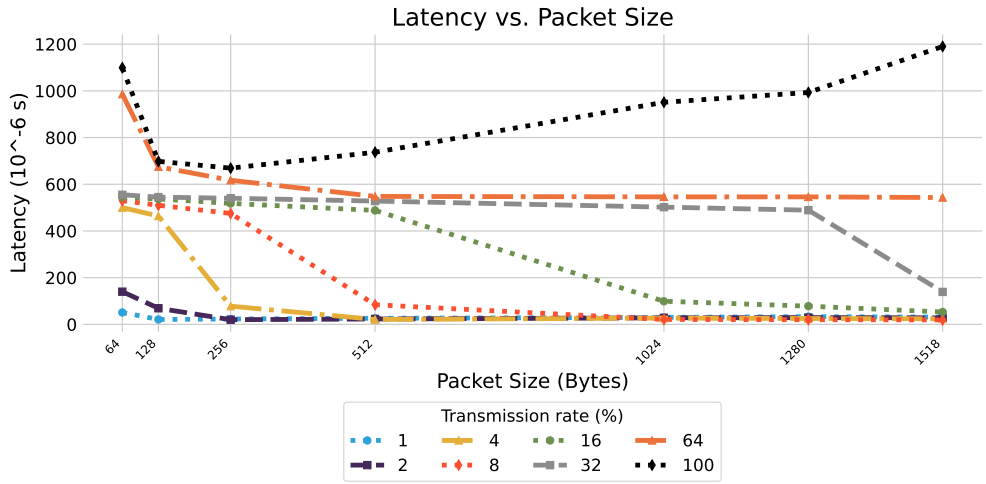

Figure 5.17: Latency of different transmission rates and different packet sizes for Linux network stack

In Figure 5.17, the latency increases for transmission rate 100% when the packet size increases, and reaches a maximum latency of 1200 ms. For all other rates other than transmission rate 100%, the latency decreases when packet size increases. The latency does decrease in most cases to a latency of about 10 ms when the packet size increases.

5.1.5 Summary of Benchmarking Results

Of the three different applications and frameworks, DPDK performs best in all aspects concerning throughput, packet loss, packet rate, and latency. The fastest throughput of DPDK was measured at about 25 Gbit/s and the highest packet rate was measured at about 9 million. The packet loss for DPDK stays under 10% most of the time, but for packet sizes 64 bytes and 128 bytes, and for transmission rates of 32% and over, the packet loss reaches a maximum of 60%. Latency stays at around 12 μs for all sizes and transmission rates under 32% and reaches a maximum latency of 1 ms for packets of size 1518 bytes with transmission rates of 64% and above.

Based on these results, it was determined that DPDK can optimally handle transmission rates up to around 64 bytes, above rate 64% performance increases are non-existent while packet loss and latency increase.

io_uring had a maximum throughput of 5.0 Gbit/s and was achieved at a transmission rate of 16% or higher when the packet size was 1518 bytes. The packet loss was significant, especially for transmission rates over 16%, and when packet size was below 1280 bytes. Generally, the packet loss decreased when packet sizes increased for all different transmission rates. The packet rate reached a maximum of approximately 460,000 packets per second. For higher transmission rates and for larger packet sizes, the packet rate decreased. This reached a minimum of around 40,000 packets per second for a transmission rate of 1%. The latency of io_uring is highest at size 1518 and transmission rate 100% with a latency of around 1.3 ms. For lower transmission rates under 64%, the latency decreases when packet size increase, reaching a minimum of around 20 to 30 μs.

The results of running io_uring at different transmission rates show that io_uring reaches its best performance on our system at around transmission rate 16%. Above rate 16% there are no improvements in performance and latency and packet loss increase.

For the Linux network stack that used a standard UDP socket for packet forwarding, the throughput reached a maximum value of 3.5 Gbit/s when the transmission rate was 16% or higher and the packet size was 1518 bytes. For packet rate, the general behavior is that for larger packet sizes the packet rate decreases or stays the same. The highest packet rate

is 300,000 packets per second and is reached when the packet transmission rate is 16% and over. Regarding packet loss, the minimum packet loss is around 40% and the maximum is 100%. When packet size increases, the packet loss decreases, but for transmission rates of 32% and over, the packet loss is still over 80%. The latency of the UDP socket generally decreases when the packet size decreases for all packet sizes but for transmission rate 100% the latency increases to approximately 1.2ms.

From the results of running a UDP socket with the standard Linux network stack, it was determined that it reaches its optimal performance on our system at a transmission rate of around 16%, and no increase in performance at higher transmission rates than 16%, and latency and packet loss increase at transmission rates above 16%.

5.2 Bursty Traffic with Switched Poisson Process

In this section, the results of simulating real-world data traffic, using a switched Poisson process to create bursty traffic, are shown. The results are related to research question 2. This includes how each application performed in terms of latency and packet loss.

The bursty traffic rates were based on two states, each state depends on the optimal performance of the application running. The optimal performance rate of each application was determined in Section 5.1.4, where the results showed optimal performance for io_uring and UDP socket at a transmission rate of around 16%, and 64% for DPDK.

5.2.1 Switched Poisson Process

The switched Poisson distribution is used for determining what rates to use in each state for each application.

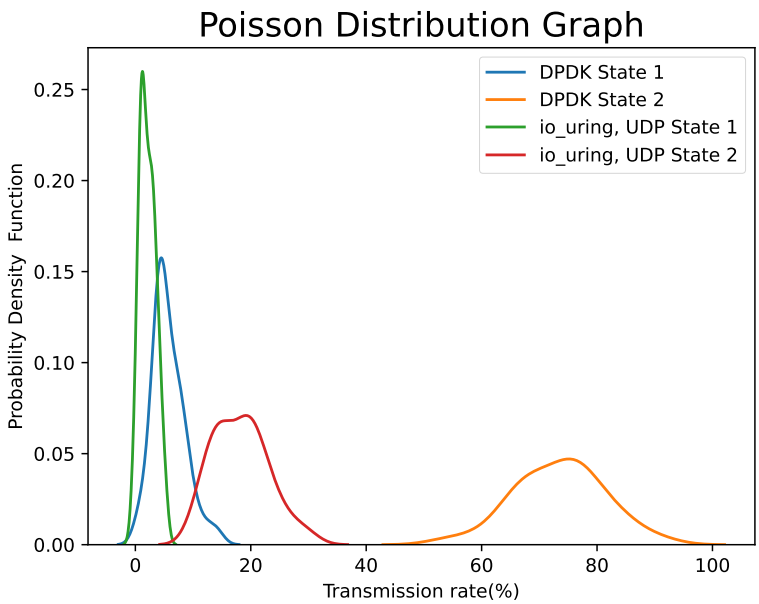

Figure 5.18: Switched Poisson process distribution

In Figure 5.18, the Switched Poisson process distribution used for choosing burst values of each state is shown. A higher mean value results in higher density and lower spread, whereas a low mean value results in lower density and larger spread.

DPDK state 1 in Figure 5.18 represents the low-traffic state where the mean value of the Poisson distribution is 10% of DPDK´s optimal transmission rate. DPDK state 2 represents

the high-traffic state with a Poisson distribution with a mean of 20% over the optimal transmission rate for DPDK.

io_uring and UDP State 1 in Figure 5.18 shows the low-traffic state with a mean that is 10% of the optimal transmission rate of io_uring and UDP socket using the standard Linux network stack. The high-traffic rate state is represented by io_uring, UDP State 2, and is using a Poisson distribution with a mean that is 20% over the optimal transmission rate for the applications.

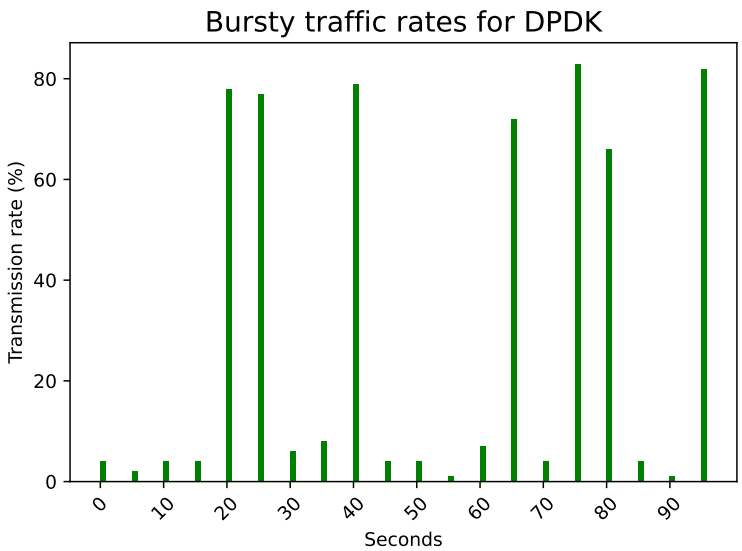

Figure 5.19: Rates used in Switched Poisson process for DPDK

Figure 5.18 shows which rates each burst should use when simulating bursty traffic for DPDK. The transmission rates were chosen through samples of the Poisson distributions for DPDK states 1 and 2 shown in Figure 5.18.

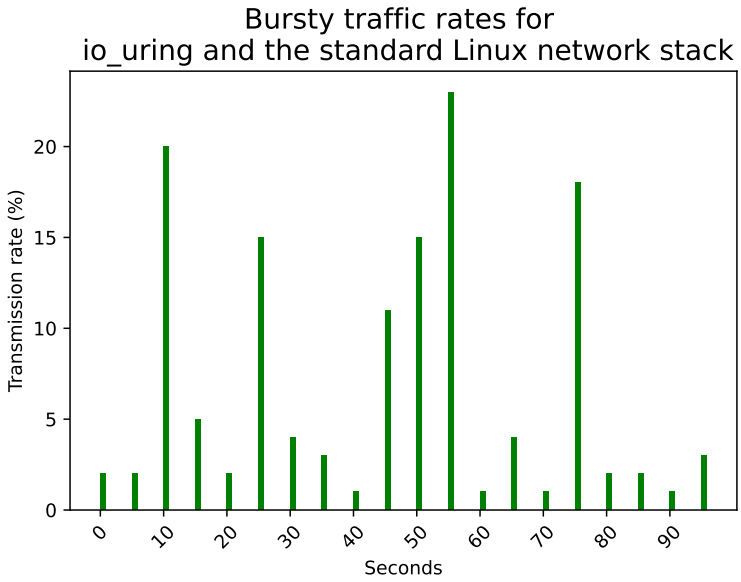

Figure 5.20: Rates used in Switched Poisson process for io_uring and the standard Linux network stack

In Figure 5.20, the rates used in simulating bursty traffic for io_uring and the UDP socket using the Linux network stack are shown.

5.2.2 DPDK

The results of DPDK handling bursty traffic in a total duration of 95 seconds are shown in this section.

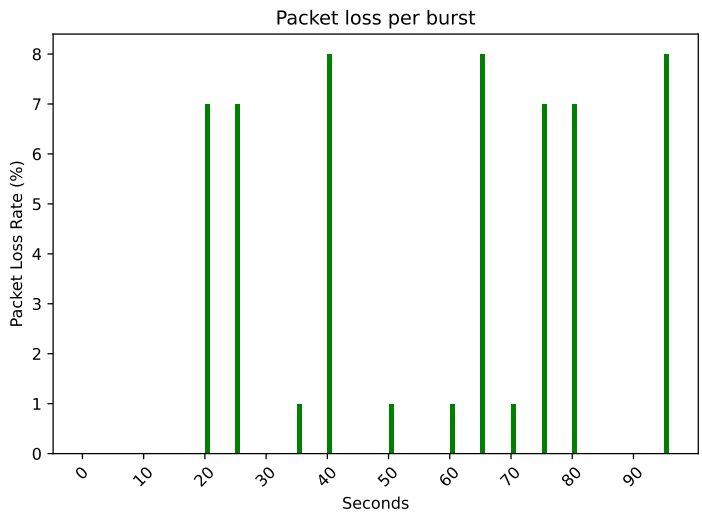

Figure 5.21: Packet loss for different rates with DPDK

In Figure 5.21 the packet loss during different transmission rates is shown. The packet loss reaches a maximum of around 8% and a minimum of 0% meaning no packet loss occurred during those bursts.

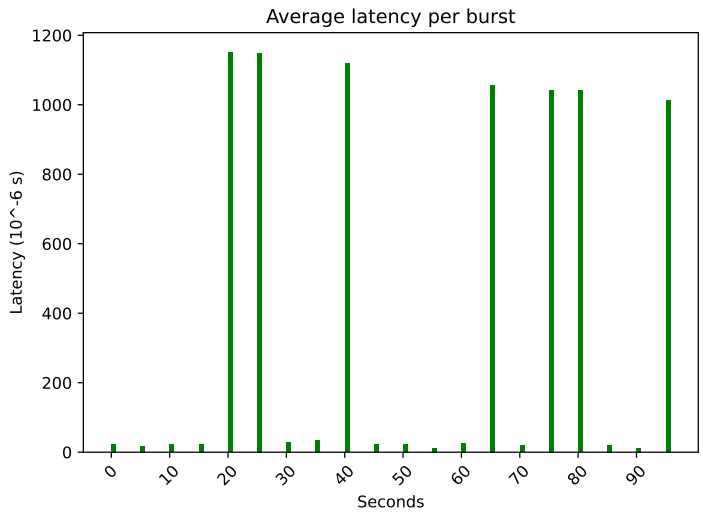

Figure 5.22: Latency for different rates with DPDK

The average latency during different bursts of traffic is shown in Figure 5.21. The maximum average latency that was recorded was around 1100 μs and the minimum is under 30 μs. The graph shows spikes in latency whenever the rate is increased to a higher transmission rate.

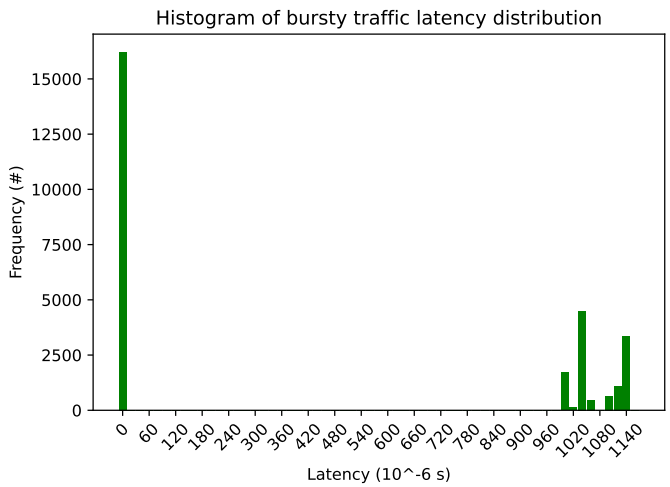

Figure 5.23: Latency histogram of latency during bursty traffic simulation with DPDK

The histogram in Figure 5.23 shows two intervals where latency occurs when sending bursty traffic data based on two states. One latency interval is between 0 and 30 μs and the other spans between 1000 and 1160 μs.

5.2.3 io_uring

The results of io_uring handling bursty traffic in a total duration of 95 seconds are shown in this section.

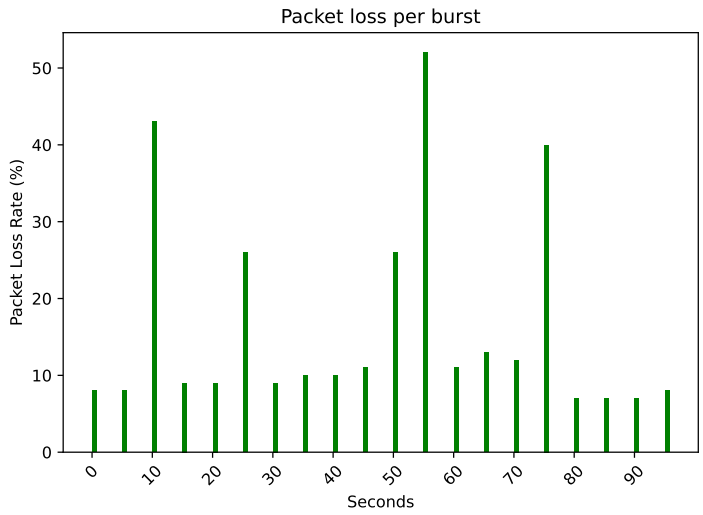

Figure 5.24: Packet loss during different rates with io_uring

In Figure 5.24, the packet loss for io_uring is shown when sending bursty traffic. The maximum packet loss is around 50% and occurs when 55 seconds have passed. The minimum packet loss rate is 8%.

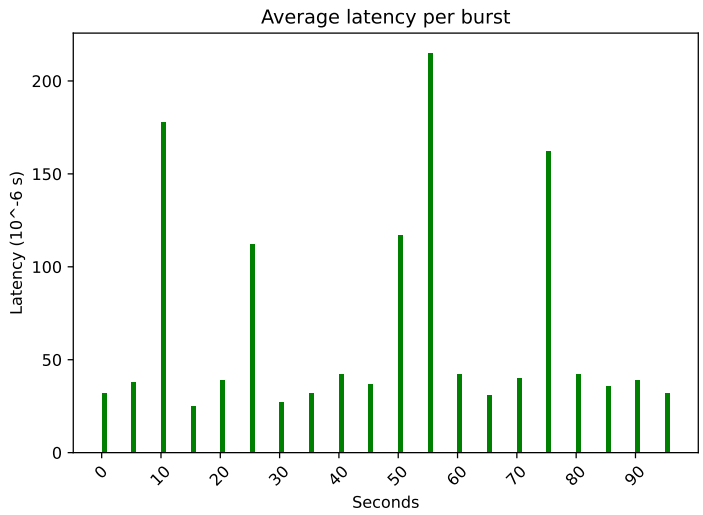

Figure 5.25: Average latency during different rates with io_uring

The average latency when sending bursty traffic via io_uring reaches a maximum of slightly over 200 μs, as shown in Figure 5.25. The minimum average latency that was recorded during the bursty traffic simulation was measured at 25 μs.

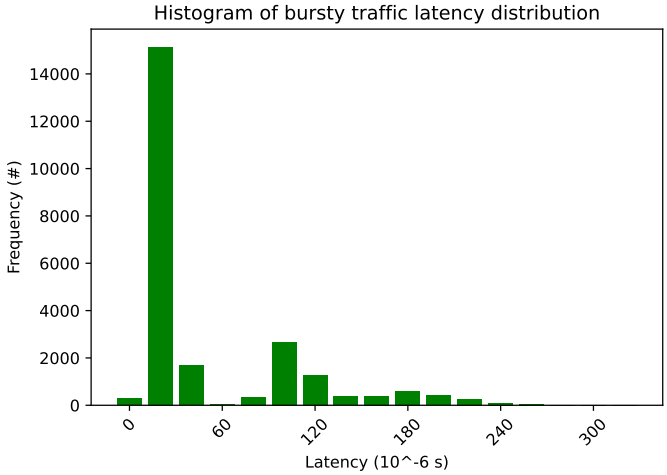

Figure 5.26: Histogram of latency during bursty traffic simulation with io_uring

The histogram of latency values recorded during the 95 seconds duration of bursty traffic is shown in Figure 5.26. The figure shows that the majority of received packets have a latency of between 10 and 30 μs, and the total spread is between 0 and 260 μs.

5.2.4 Standard Linux Network Stack

In this section, the results of sending bursty traffic via a UDP socket using the standard Linux network stack are shown.

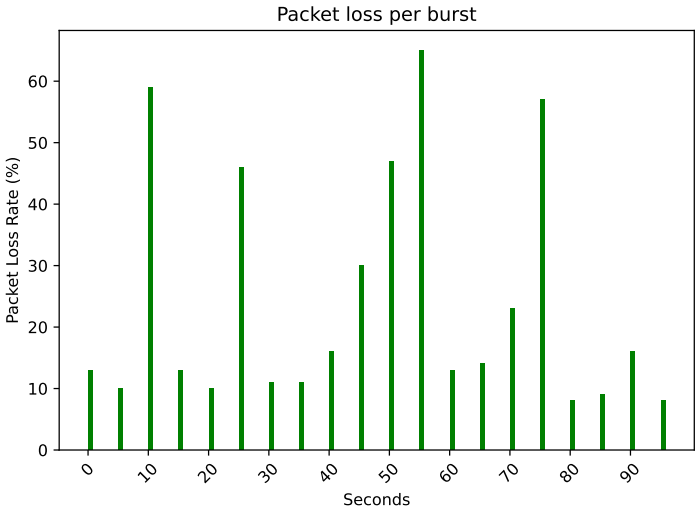

Figure 5.27: Packet loss during different rates with Linux network stack

The packet loss rate has a maximum of 65% after 55 seconds have passed and a minimum of 9% during the 95-second duration of the testing, as can be seen in Figure 5.27.

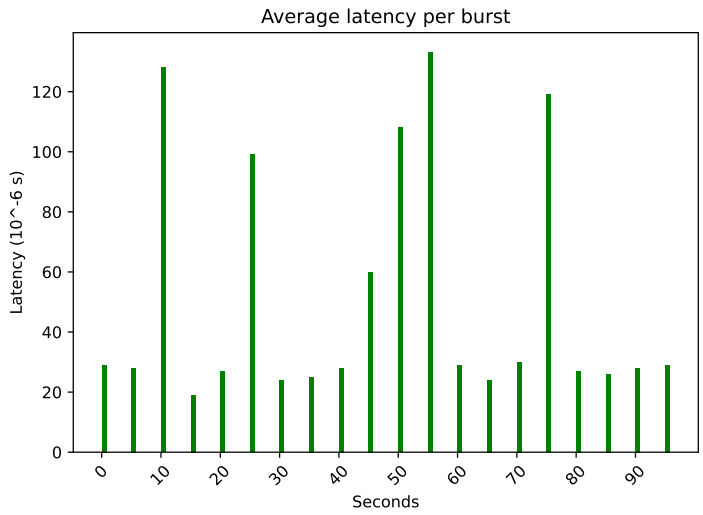

Figure 5.28: Average latency during different rates with Linux network stack

Measuring the average latency when sending bursty traffic via the UDP socket using the standard Linux network stack the results in Figure 5.28 showed a latency maximum of about 130 and minimum of about 20 μs.

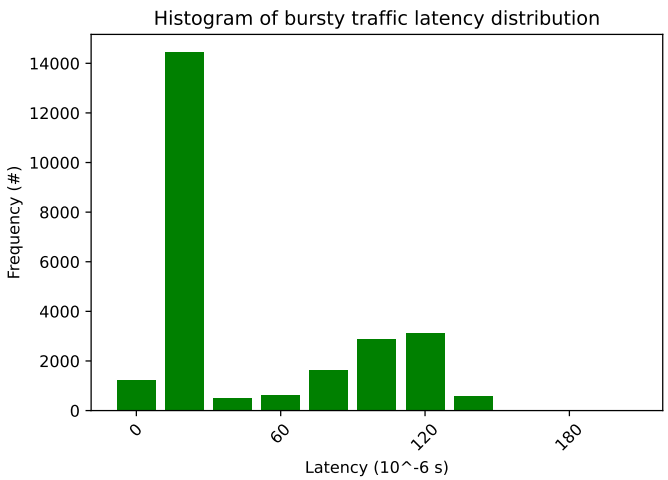

Figure 5.29: Histogram of latency during bursty traffic simulation with the standard Linux network stack

The latency histogram in Figure 5.29 shows that most of the packets arrive with a latency rate of between 10 and 30 μs and the spread of latency values is between 0 and 140 μs.

5.2.5 Summary of Bursty Traffic Results

The figures for average latency and packet loss rates for DPDK, io_uring, and the Linux network stack are strongly correlated with the transmission rates that are sent during different traffic bursts. The figures show that whenever the transmission rate is around or over the optimal performance of each application, the latency and packet loss increase drastically.

The maximum packet loss for DPDK was around 8%, while the minimum was 0%. DPDK has very high latency values, up to 1100 μs, whenever the transmission rate exceeds what DPDK is able to handle whenever the transmission rate is low, the latency decreases to values in the range of 10^{-7} seconds. This means that there are four orders of magnitude between the low and the high latency values. The histogram shows distinct latency intervals where one is very low relative to the other.

io_uring showed packet loss rates with a maximum of around 55% and a minimum of 8% when handling data coming in bursts. The latency values for handling bursty traffic with io_uring showed a maximum of over 200 μs and a minimum of around 25 μs which is one-fourth of the maximum latency value. The histogram containing latency values over the duration of the testing showed that io_uring had a spread of latency values between 0 and 260 μs. In contrast, most packets had a latency of around 20 μs.

The UDP socket using the standard Linux network stack had packet loss rates between 65% and 9%. The latency was measured at a maximum of 130 μs and a minimum of around 20 μs, meaning that the maximum was about six to seven times larger than the minimum latency values. The histogram containing latency values for the bursty traffic simulation with the standard Linux network stack showed a latency interval between 0 and 160 μs.

5.3 Result of Tuning Linux Kernel Parameters

For research question 3, several Linux kernel parameters were tuned. Not all parameters had the same magnitude impact as others, therefore only the most interesting and noteworthy metrics and graphs will be shown. The less intriguing results are available in Appendix Kernel tuning graphs.

5.3.1 Configurable Parameters

In this section, Linux kernel parameters that were configurable are described and their impacts on performance are shown.

5.3.1.1 RX Flow Hash Indirection Table and Hash Function

Changing the RX flow hash indirection table showed some interesting differences in latency and throughput, both in terms of packet rate and overall throughput. However, it was not possible for the Linux system to change the hash function of the network interface that was used for testing. The following results were obtained after changing the indirection table to only map to receive queue 0.

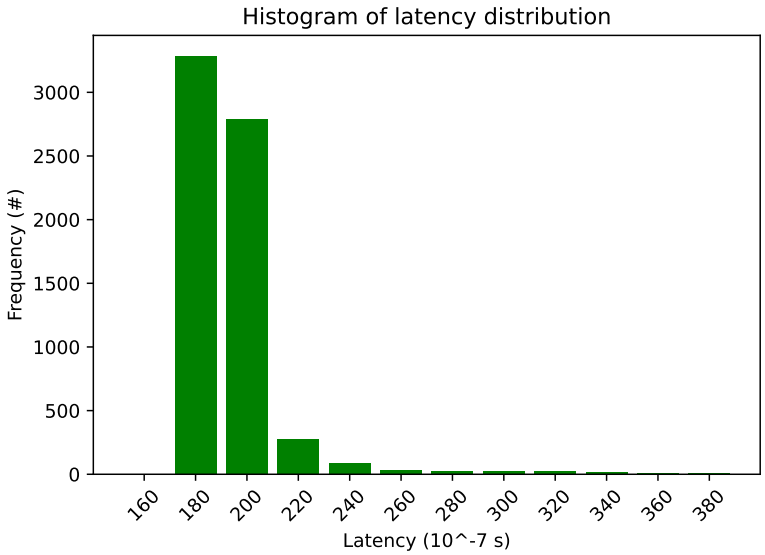

Figure 5.30: Latency histogram running io_uring with 1518 byte size packets at transmission rate 8% when RX flow hash indirection table only maps to receive queue 0

The difference that can be seen in Figure 5.30 compared to the previous histogram when running io_uring seen in Figure 5.11 is that the largest amount of incoming packets arrives with a latency around 18 μs instead of 20 μs seconds.

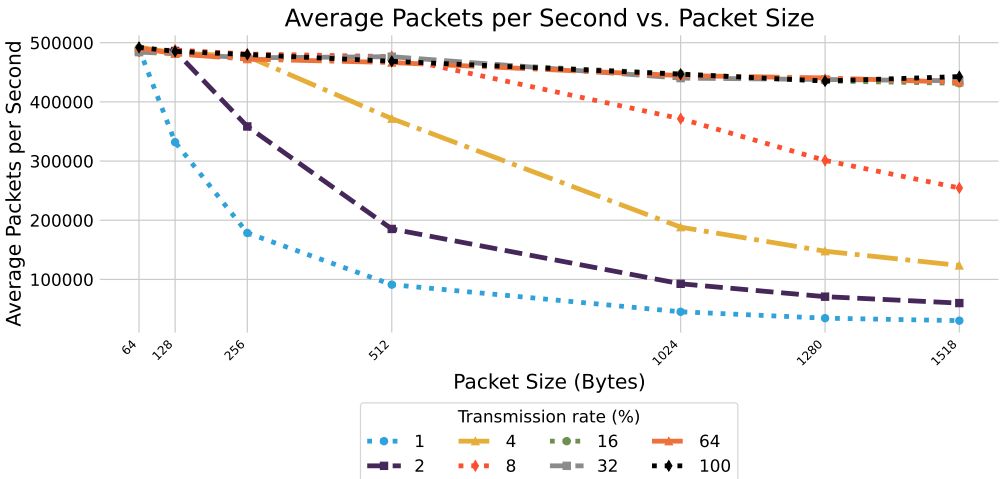

Figure 5.31: Packet rate of different transmission rates and different packet sizes for io_uring when RX flow hash indirection table only maps to receive queue 0

Figure 5.31 shows an increase in packet rate compared to 5.8. The increase is around 45,000 packets per second for all transmission rates at packet size 64 bytes, and for all packet sizes when transmission rates are 16% or larger.

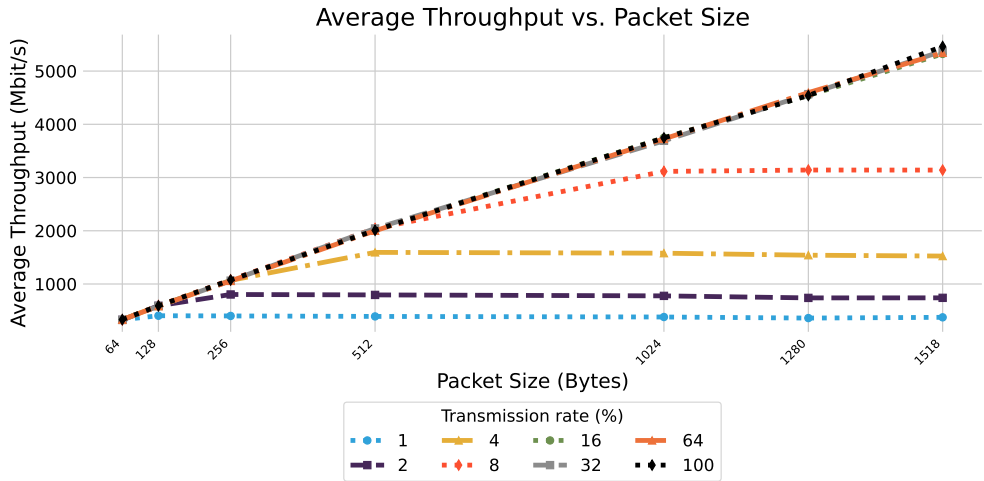

Figure 5.32: Throughput of different rates and different packet sizes for Linux network stack when RX flow hash indirection table only maps to receive queue 0

In Figure 5.32 there was an increase in throughput of about 200 Mbits/s for transmission rates 16% and above, for packet sizes of 1024 bytes or larger compared to Figure 5.10

5.3.1.2 Hardware Buffer Queue

The hardware ring buffer queues size was tested with two different values: 256 and 4096 bytes, which both showed some significant differences in latency from the original results with a default hardware buffer queue size of 512 bytes.

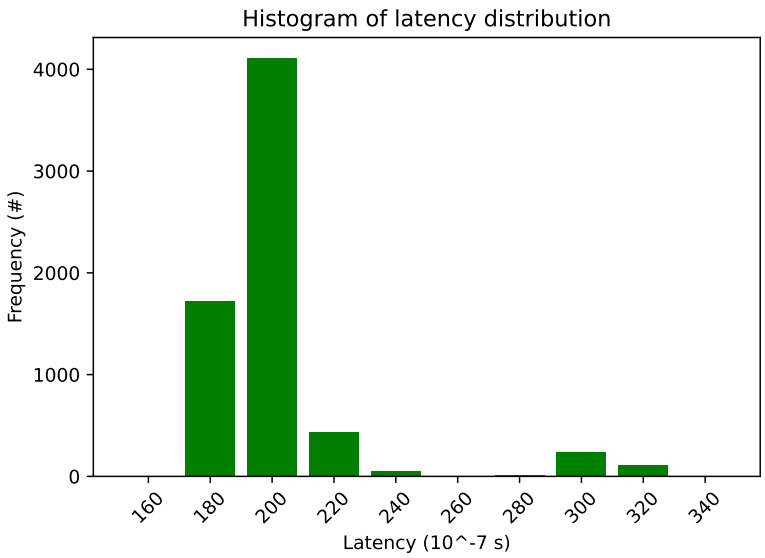

Figure 5.33: Latency histogram running io_uring with 1518 byte size packets at transmission rate 8% with ring buffer queue size set to 256 bytes

The latency histogram shown in Figure 5.33 shares some resemblance with Figure 5.11. However, it has some small peaks in latency values around 30 and 32 μs.

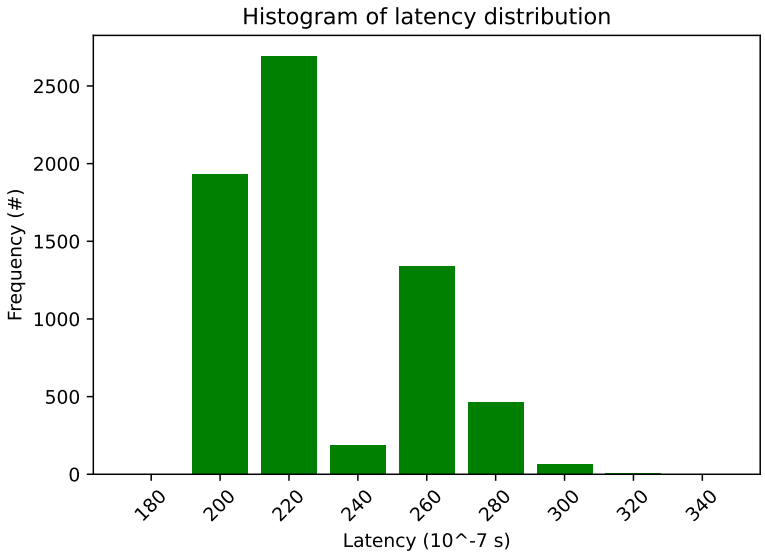

Figure 5.34: Latency histogram running io_uring with 1518 byte size packets at transmission rate 8% with ring buffer queue size set to 4096 bytes

Figure 5.34 shows that there were large amounts of packets with latency as high as 26 and 28 μs. Compared to Figure 5.11 the majority of recorded latency values were around 22 μs instead of 20 μs.

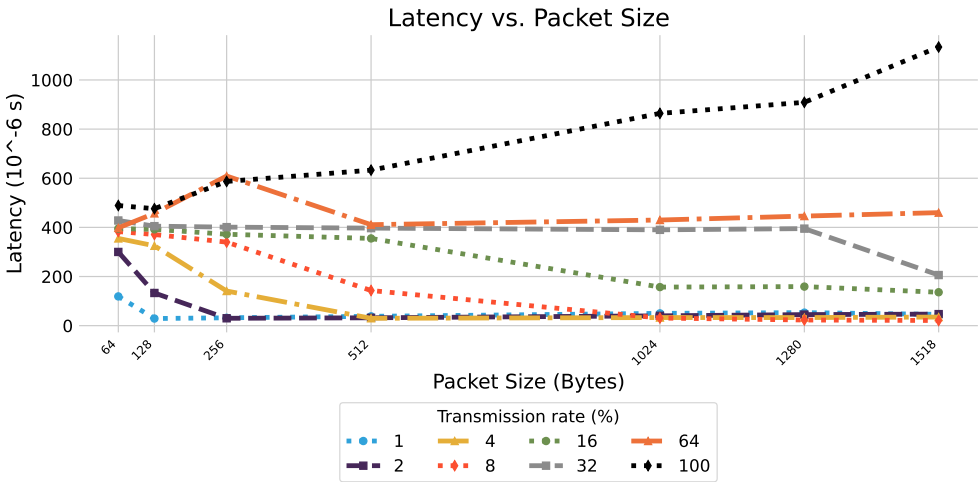

Figure 5.35: Latency of different rates and different packet sizes for io_uring with ring buffer queue size set to 256 bytes

For transmission rates between 4% and 100%, there was a clear decrease in latency of about 20 μs for smaller packet sizes. For larger packet sizes the latency decrease was still evident but only for transmission rates between 32% and 64% as can be seen in Figure 5.35.

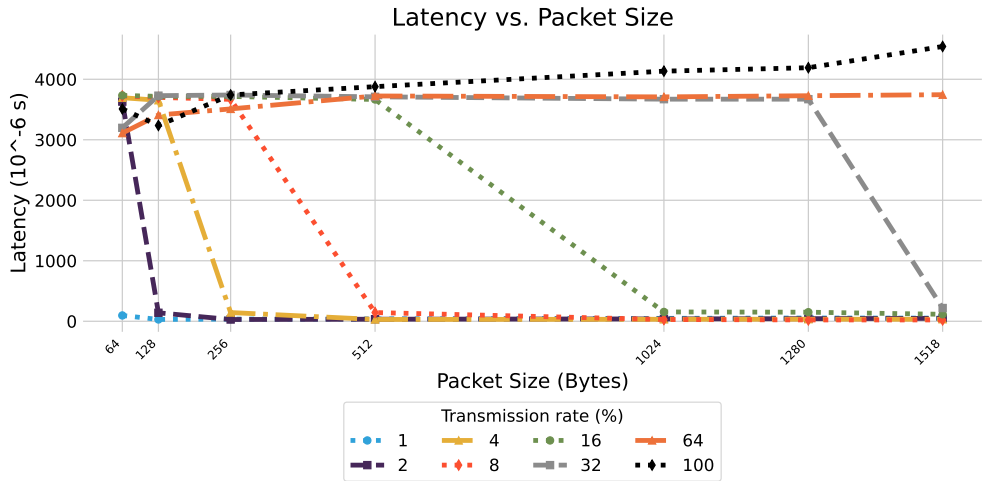

Figure 5.36: Latency of different rates and different packet sizes for io_uring with ring buffer queue size set to 4096 bytes

From Figure 5.36 it is clear that the larger ring buffer queue size also resulted in higher latency for almost all transmission rates where most transmission rates had latency values of around 350 μs seconds at some point during the measurement.

5.3.1.3 netdev_budget

Increasing the netdev_budget showed a small increase in latency values in the latency histogram.

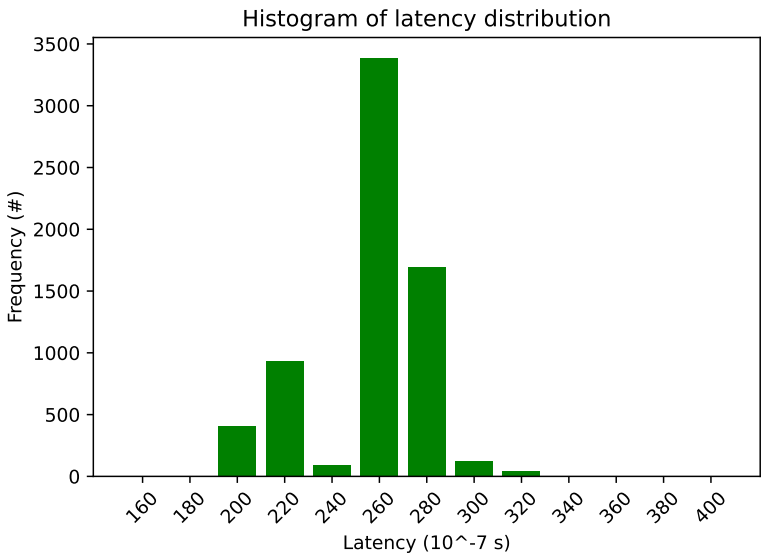

Figure 5.37: Latency histogram running io_uring with 1518 byte size packets at transmission rate 8% with netdev_budget set to 1500

After increasing netdev_budget up to 1500 bytes from its original value of 300 bytes a significant difference could be seen in the histogram in Figure 5.37, where the majority of packets show an increase in latency which is up to between 26 and 28 μs.

5.3.1.4 Socket Receive Queue Memory

Several sizes were tested for socket receive queue memory and some of the more interesting findings are shown in this section.

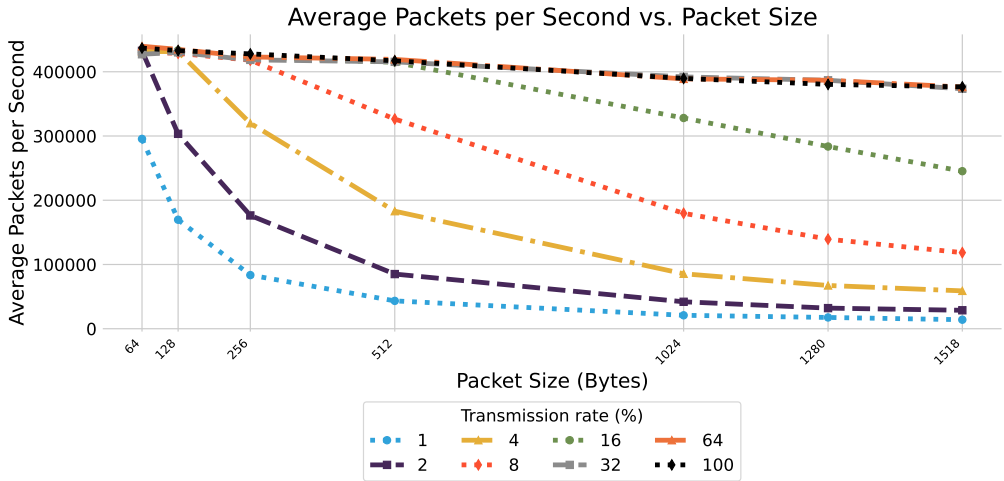

Figure 5.38: Packet rate of different rates and different packet sizes for io_uring with receive queue memory set to 26624 bytes

In Figure 5.38, the general trend is that for higher packet sizes and for higher transmission rates the average number of packets decreases. The maximum packet rate is around 430,000 packets per second for a packet size of 64 bytes and for a packet size of 1518 bytes, the maximum average packets per second rate reached is 380,000 packets per second.

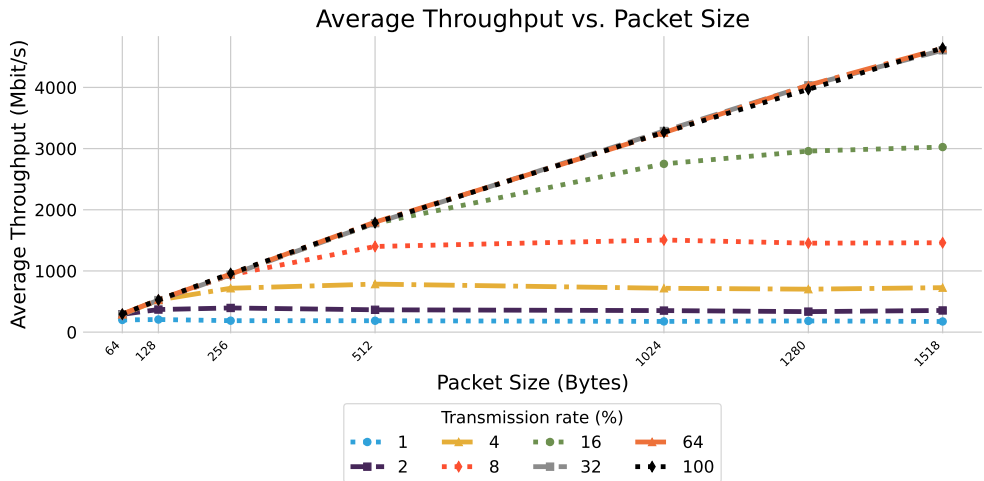

Figure 5.39: Throughput of different rates and different packet sizes for io_uring with receive queue memory set to 26624 bytes

Figure 5.39 shows that higher transmission rates and larger packet sizes lead to higher throughput. The maximum throughput is 4.6 Gbit/s and it is reached when the transmission rate is 32% or larger and when the packet size is 1518 bytes.

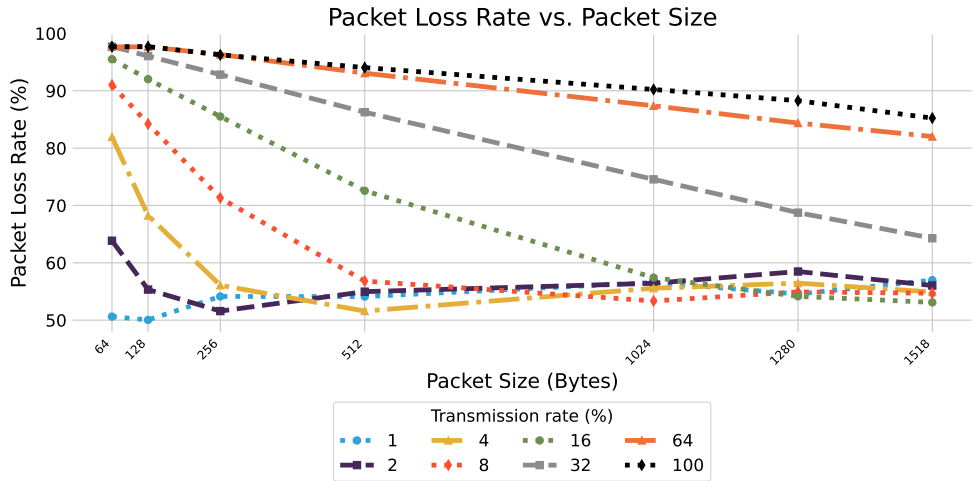

Figure 5.40: Packet loss of different rates and different packet sizes for io_uring with receive queue memory set to 26624

In Figure 5.40, there is a trend where higher transmission rates and smaller packet sizes yield more packet loss. For lower transmission rates and higher packet sizes, the packet loss is around 50% to 60%.

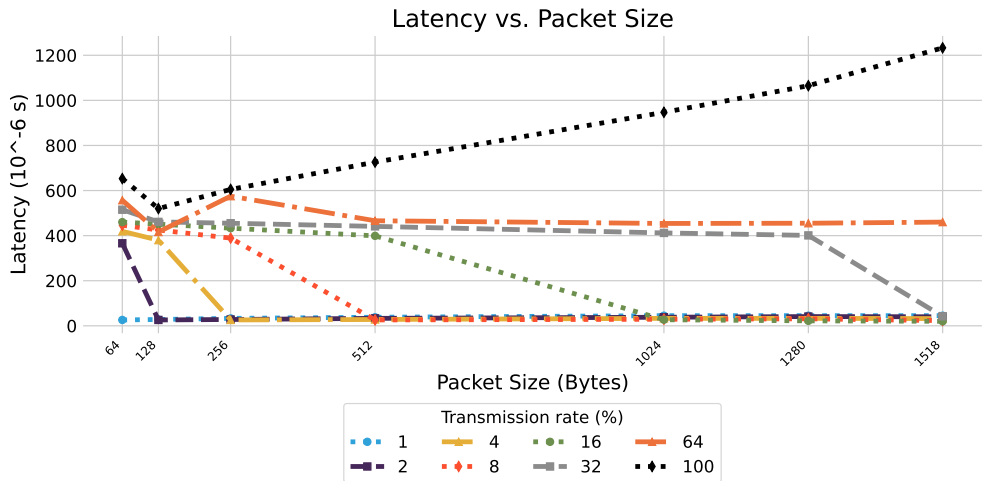

Figure 5.41: Latency of different rates and different packet sizes for io_uring with receive queue memory set to 26624

Figure 5.41 shows an overall latency decreases for transmission rates lower than 100%. For transmission rates 32% and 64%, the latency is around 400 μs which is 200 μs lower than what was shown in Figure 5.12.

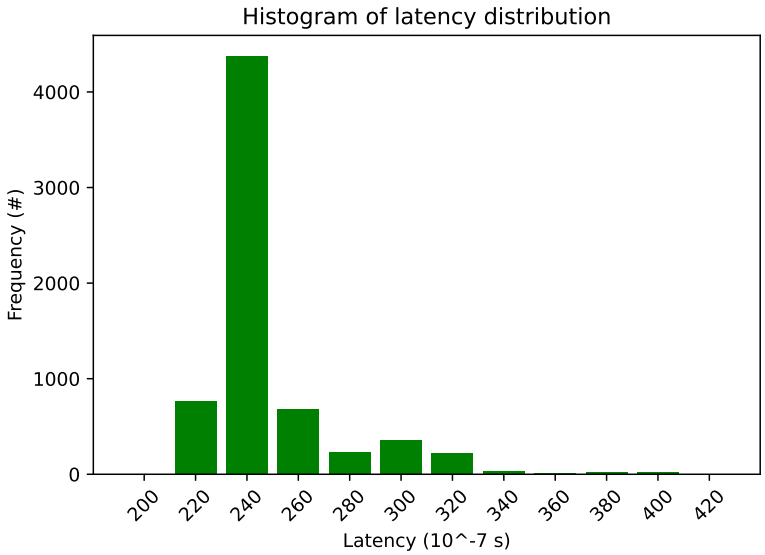

Figure 5.42: Latency histogram running io_uring with 1518 byte size packets at transmission rate 8% with receive queue memory set to 26624

The trend seen for packet sizes of 1518 bytes in Figure 5.42 when decreasing the socket receive memory size was an increase in latency of about 40 μs.

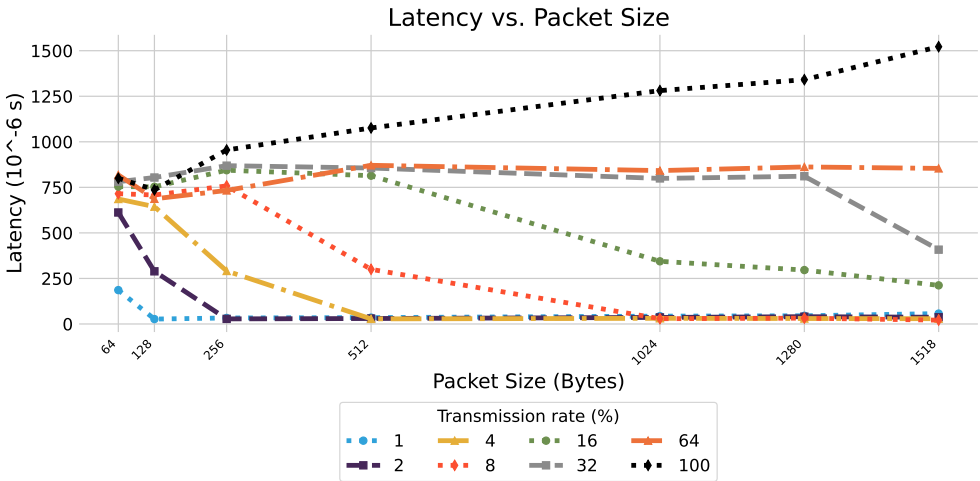

Figure 5.43: Latency of different rates and different packet sizes for io_uring with receive queue memory set to 425984 bytes

The latency shown in Figure 5.43 showed an average latency of about 800 μs for most packet sizes with transmission rates between 32% to 64%. The previous optimal transmission rate of 16% for io_uring reached a latency of around 200 μs which is significantly larger than what was previously recorded as can be seen in Figure 5.12.

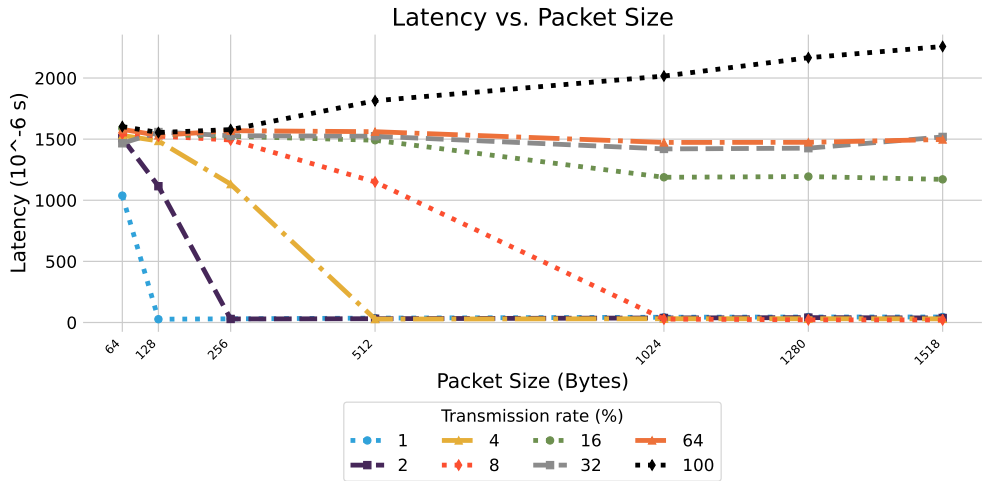

Figure 5.44: Latency of different rates and different packet sizes for io_uring with receive queue memory set to 1703936 bytes

With the socket receive queue memory set to 1703936 bytes, the average latency for transmission rates 16% up to 64% was around 1250 to 1500 μs, and a maximum latency of over 2000 μs for packet size 1518 bytes and transmission rate 100%, as can be seen in Figure 5.44.

5.3.1.5 Socket Transmit Queue Memory

When testing the socket transmit queue memory size it was not possible to configure it to be lower than its initial default value as the io_uring application that was used could not handle smaller socket transmit queue memory sizes.

When increasing the socket transmit queue memory, the same sizes were tested as for socket receive queue memory, that is 425984 and 1703936.

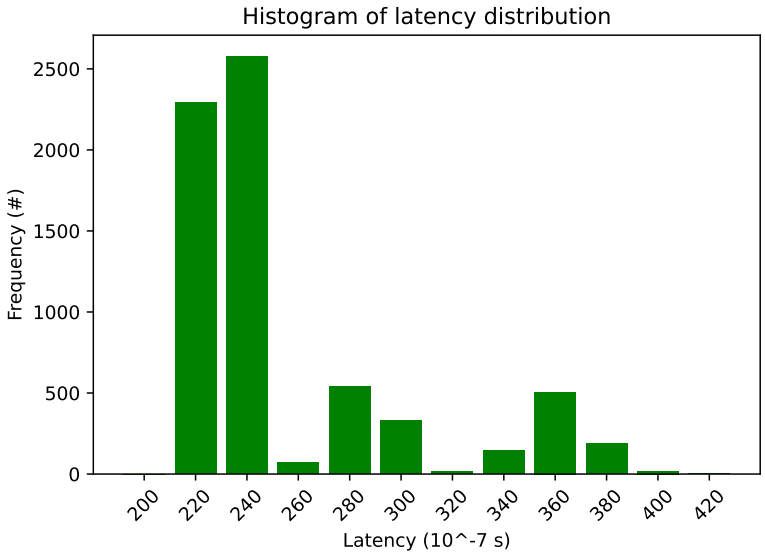

Figure 5.45: Latency histogram running io_uring with 1518 byte size packets at transmission rate 8% with socket receive queue memory set to 425984

When doubling the socket transmit queue memory, the histogram of latency values in Figure 5.45 showed an increase of about 20 μs to what it was previously and also showed significant amounts of packets with latency values between the 28 and 38 μs.

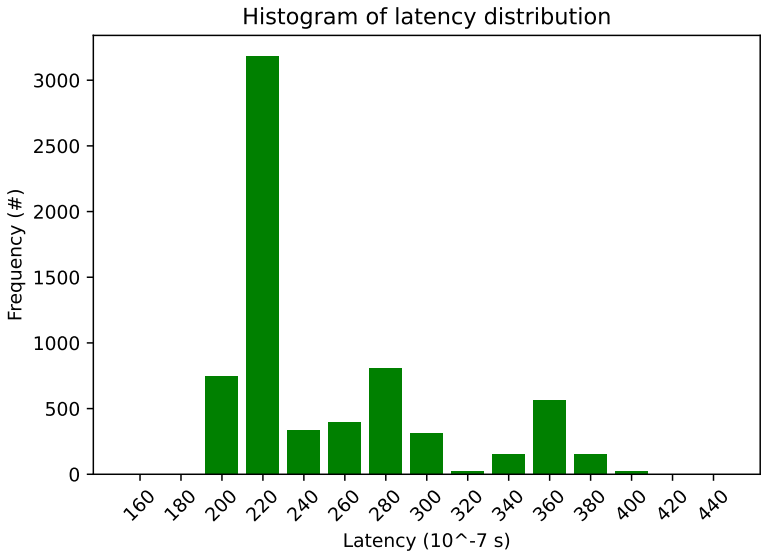

Figure 5.46: Latency histogram running io_uring with 1518 byte size packets at transmission rate 8% with socket receive queue memory set to 1703936

Increasing the socket transmit queue further did showed a small difference compared to Figure 5.45 which was that the spread of latency values was more centered around 220 μs as seen in Figure 5.46.

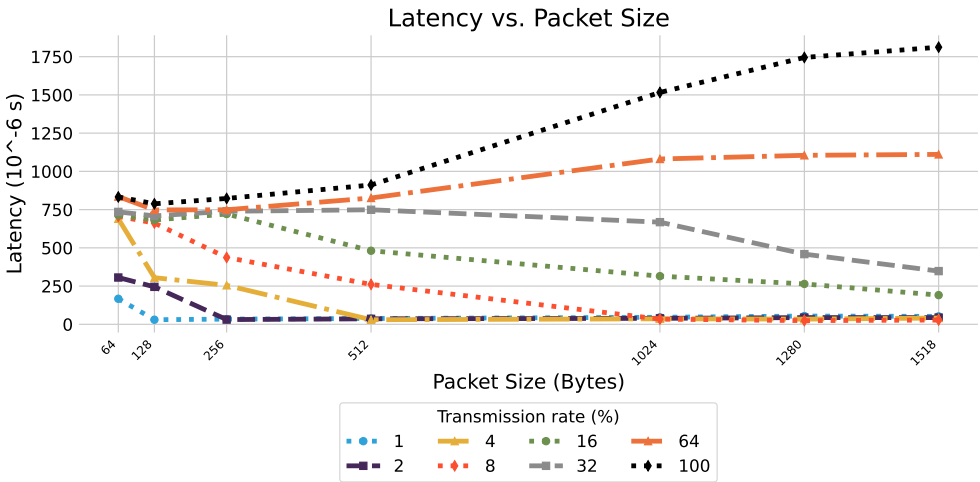

Figure 5.47: Latency of different rates and different packet sizes for io_uring with socket receive queue memory set to 425984

The increase in latency when increasing the socket transmit queue was also visible in Figure 5.47 where a majority of transmission rates and packet sizes showed an increase in latency. When increasing the socket transmit queue memory further to 1703936 the outcome was very closely matched to what was shown in Figure 5.47.

5.3.1.6 Receive Packet Steering (RPS)

Two configurations were tested for RPS. One was to delegate interrupt handling to a CPU on the same NUMA node as the CPU running the application. The other configuration was running the application and interrupt handling on the same CPU.

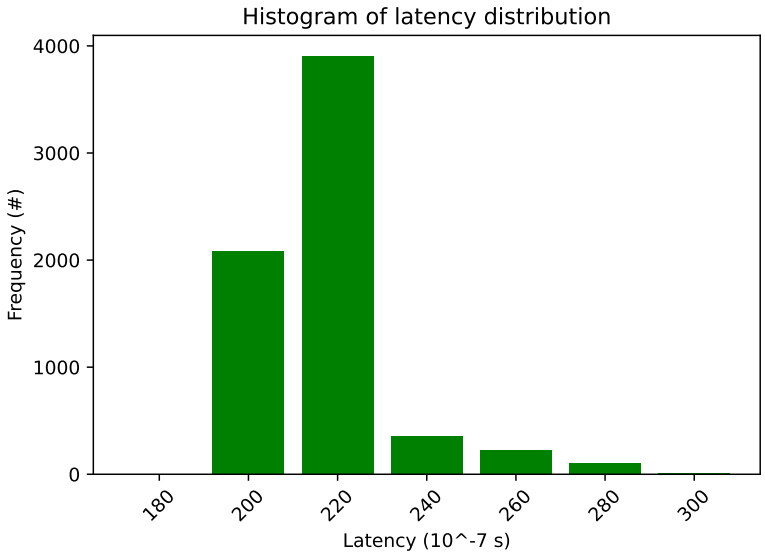

Figure 5.48: Latency histogram running io_uring with 1518 byte size packets at transmission rate 8% and delegating interrupts to another CPU

When delegating interrupts to another CPU, the latency increased slightly with around 20 μs, as can be seen in Figure 5.48.

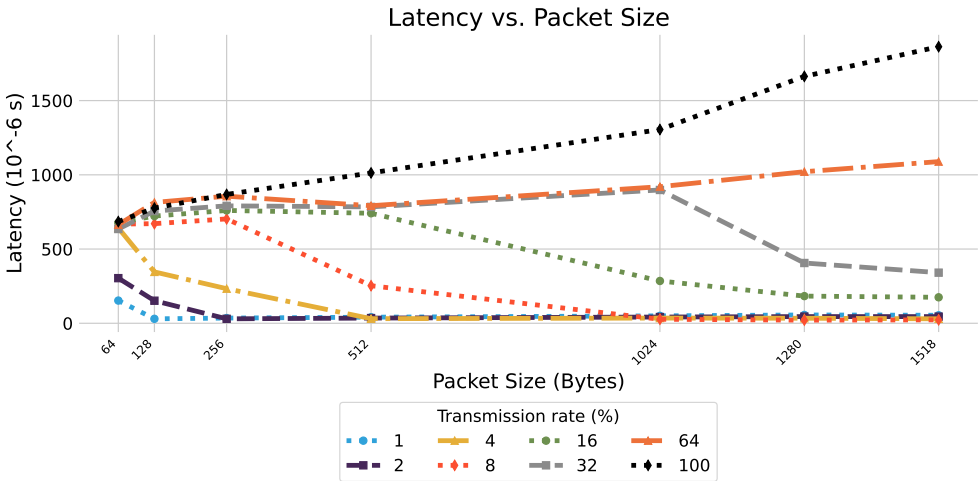

Figure 5.49: Latency of different rates and different packet sizes for io_uring and delegating interrupts to another CPU

When considering the overall latency values in Figure 5.49 there was a trend of increased latency for most transmission rates and packet sizes. The latency increases with higher transmission rates.

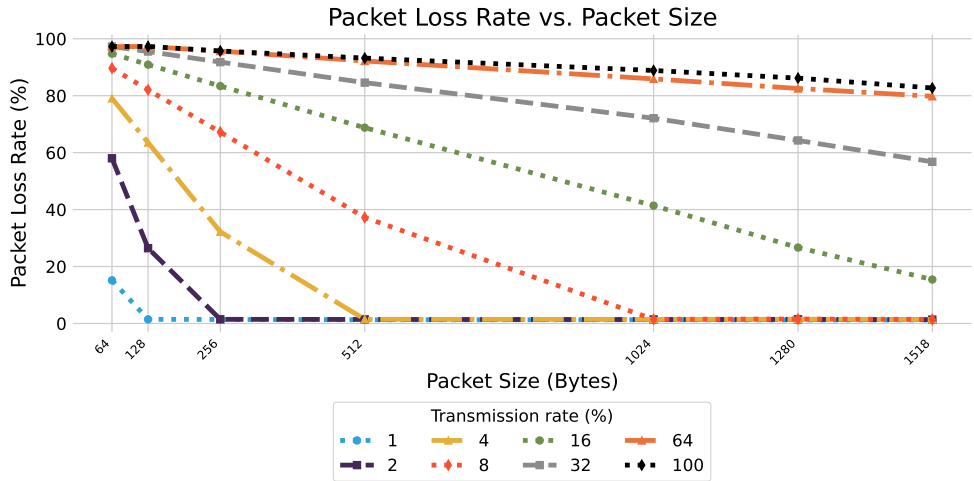

Figure 5.50: Packet loss during different rates with io_uring and delegating interrupts to another CPU

Although there was a decrease in packet loss for all transmission rates and packet sizes the decrease was largest for larger packet sizes. The decrease in packet loss was close to 10% for several transmission rates with packet size 1518 bytes, as seen in Figure 5.50.

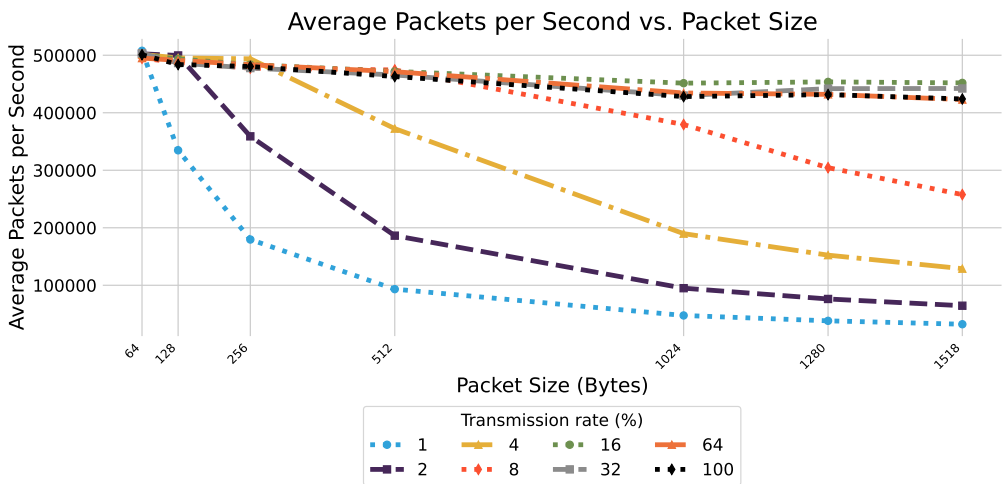

Figure 5.51: Packet rate of different rates and different packet sizes with io_uring and delegating interrupts to another CPU

Figure 5.51 showed an increase in packet rate. 500,000 packets per second were sent for all transmission rates when packet size was 64 bytes, which is a 50,000 increase in packets per second compared to earlier results in Figure 5.8. For transmission rates 16% and higher, the increase of 50,000 was apparent for all packet sizes, while for smaller packet sizes the difference became smaller as the packet size increased. A transmission rate of 16% showed the highest total packets per second.

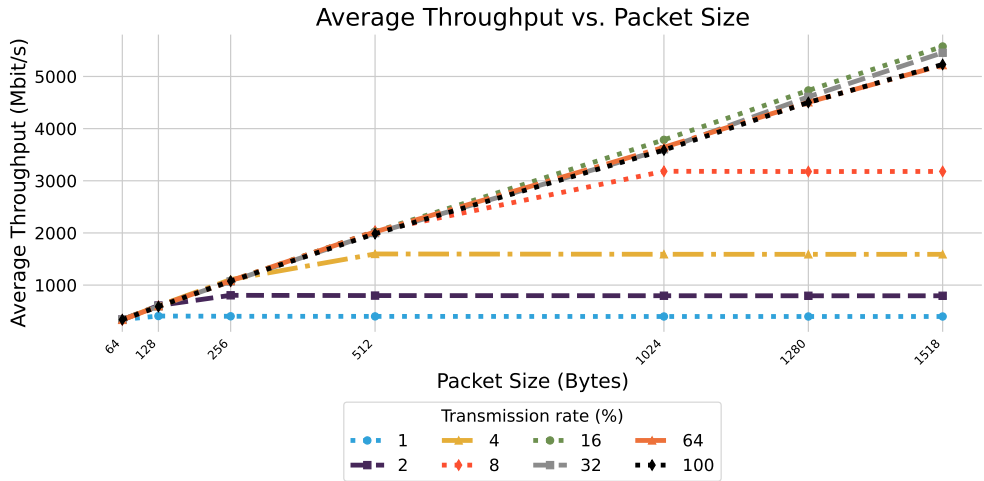

Figure 5.52: Throughput of different rates and different packet sizes with io_uring and delegating interrupts to another CPU

Figure 5.52 depicts a similar result as Figure 5.51 where the throughput rate is increased. The increase is largest for larger packet sizes and a transmission rate of 16% showing the overall highest throughput.

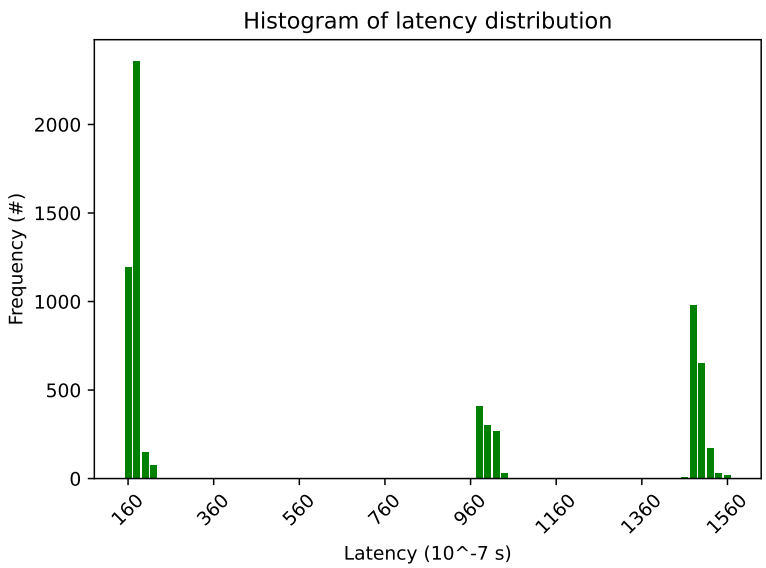

Figure 5.53: Latency histogram running io_uring with 1518 byte size packets at transmission rate 8% and the same CPU running the application handles interrupts

When configuring the same CPU that is running the application to also handle incoming interrupts the latency showed three spikes in latency values around 160, 1000, and 1569 μs, with the majority being around 160 μs.

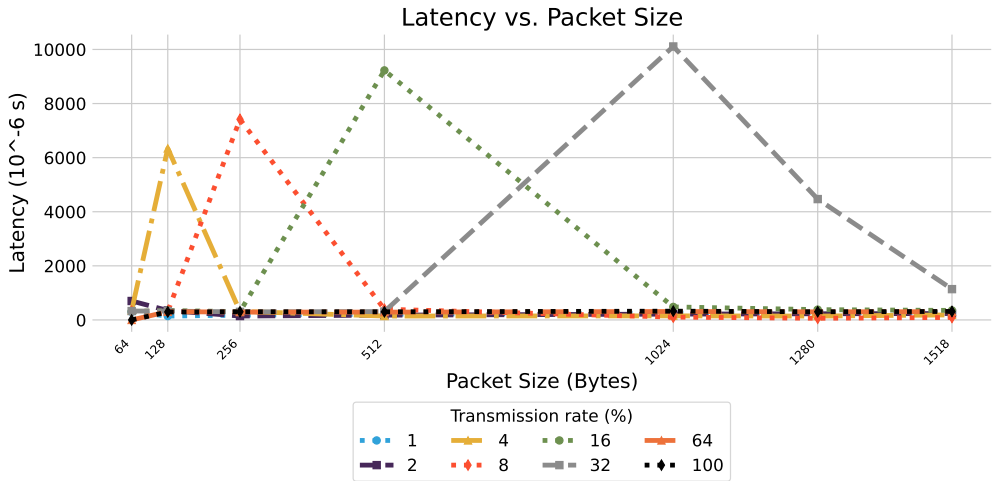

Figure 5.54: Latency of different rates and different packet sizes for io_uring and the same CPU running the application handles interrupts

Looking at latency there was a trend of certain packet sizes and transmission causing extremely large latency values with a maximum of 10,000 μs. When the latency values did not spike the latency was around 300 μs mostly which is relatively low compared to earlier results in Figure 5.12.

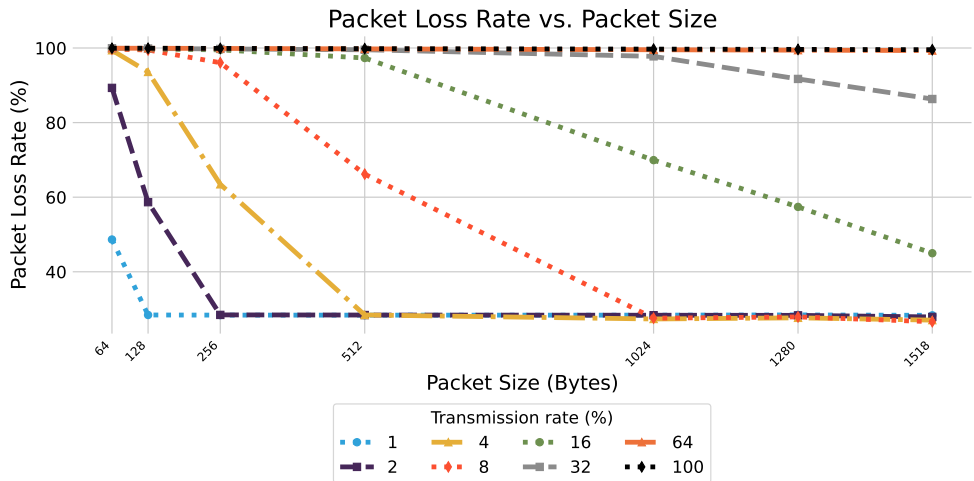

Figure 5.55: Packet loss during different rates with io_uring and the same CPU running the application handles interrupts

The packet loss was very high, especially for transmission rates 64% and 100% where it was 99% for all packet sizes as can be seen in Figure 5.55. The packet loss was higher than in earlier results, see Figure 5.9. The lowest packet loss rate was measured at slightly under 30% whereas the previous result showed packet loss rates of close to 0%.

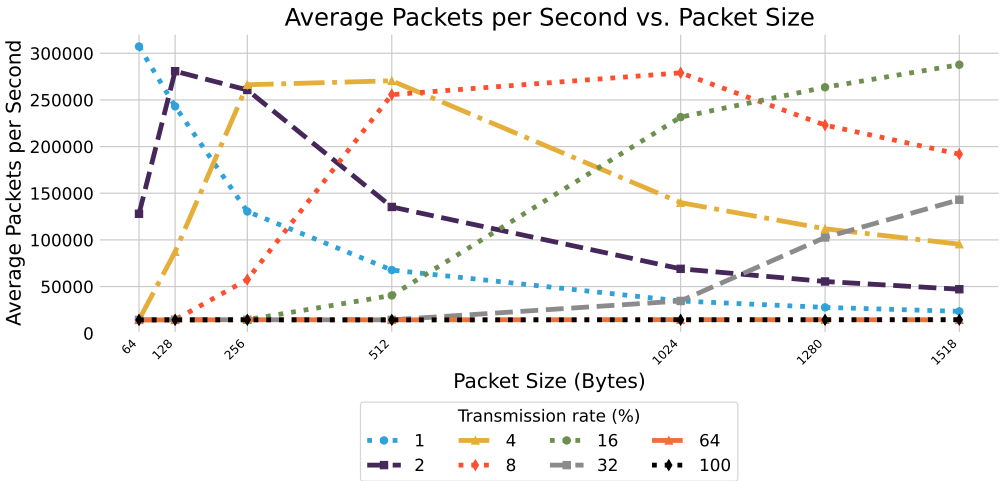

Figure 5.56: Packet rate of different rates and different packet sizes with io_uring and the same CPU running the application handles interrupts

Figure 5.56 shows a trend where the packet rate initially is increasing fast for each transmission rate and packet size until it reaches its maximum performance and then gradually starts to drop in packets per second.

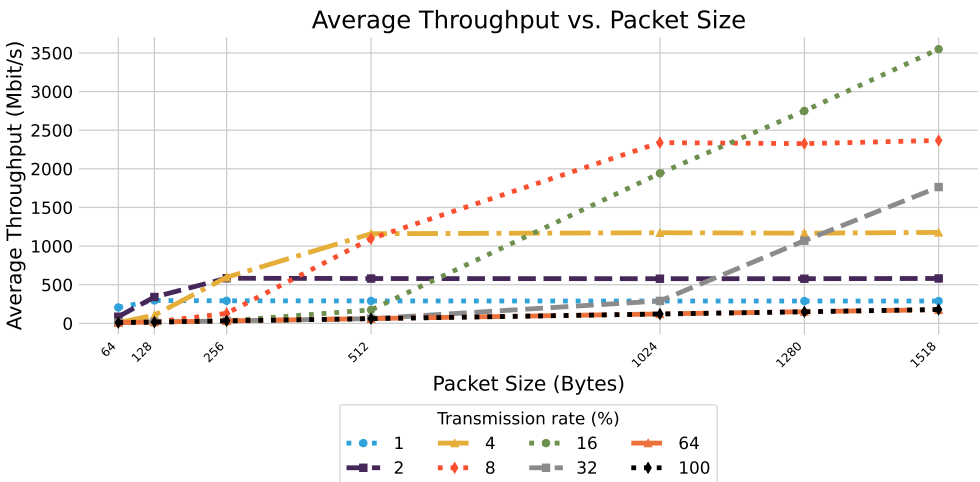

Figure 5.57: Throughput of different rates and different packet sizes with io_uring and the same CPU running the application handles interrupts

The trend in throughput shows a linear increase for each transmission rate and packet size then it plateaus once it has reached its maximum performance as can be seen in Figure 5.57. The maximum performance is reached at packet size 1518 bytes and transmission rate 16% where it is 3,500 Mbit/s. This is considerably lower than the 5000 Mbit/s that was shown in Figure 5.10. There are two outliers to the trend, these are transmission rate 64% and 100% which both have a very low throughput and only shows a very small linear increase in throughput as the packet size increases.

5.3.1.7 Generic Receive Offloading (GRO)

As a default, GRO is turned on. To consider a scenario where the GRO is disabled it was turned off. Having GRO turned off or on did not show any significant difference in networking performance.

5.3.1.8 netdev_max_backlog

There were no significant changes in any of the metrics when testing values between the default value 1000 and 5000 for netdev_max_backlog.

5.3.1.9 ip_early_demux

Enabling ip_early_demux did not have any impact on performance for any of the networking metrics.

5.3.2 Non-Configurable Parameters

This section includes parameters that could not be configured with explanations as to why they were not configurable on the system.

5.3.2.1 Accelerated Receive Flow Steering (aRFS)

As the kernel version that was used did not offer support for ntuple, and to make ntuple support enabled would require an entire reinstall of the kernel which was not an option as the results could differ if another kernel version was to be used.

5.3.3 Summary of Tuning Kernel Parameters

Not all parameters that were tuned for performance increase did actually have an impact. The metric that showed the largest changes was latency. The techniques and parameters that did have a significant impact on the performance were RPS, RX flow hash indirection table and hash function, netdev_budget, socket receive queue memory, hardware buffer queue, and socket transmit queue. RPS increased the network performance the most, and this was when one core was dedicated to running the application, and another core on the same NUMA node was dedicated to handling incoming interrupts.

6 Discussion

This chapter contains a discussion about the results of investigating the performance of DPDK, io_uring, and the standard Linux network stack implementations under different conditions. The impact of tuning the kernel for the network performance of io_uring is also discussed.

6.1 Results

In this section, the results of the study are explained. The discussion of results is divided into three parts, one part for each research question.

6.1.1 Benchmarking DPDK, io_uring and Linux Network Stack

This section will discuss the results that were collected for research question 1.

When comparing packet rate DPDK (figure 5.1) performed the best, reaching a maximum of around 9.0 million packets per second compared to io_uring (Figure 5.8), that reached a maximum of about 460,000 packets per second. The Linux network stack (Figure 5.13) performed the worst with a maximum of about 300,000 packets per second.

When considering the throughput, DPDK was also the best performing with a throughput of 25 Gbit/s compared to io_uring with 5.0 Gb/s and the standard Linux network stack implementation which performed the worst with a maximum of 3.5 Gbit/s.

The latency of the different applications was also in favor of DPDK as can be seen in the histogram shown in Figure 5.5, where the mean latency is about 10 μs lower than io_uring in Figure 5.11 and the standard Linux network stack in Figure 5.16. When comparing the latency as a function of packet size and transmission rate, DPDK has lower overall latency. DPDK has a better latency for all transmission rates except for a transmission rate of 64% where io_uring and the standard Linux network stack were performing better. io_uring and the standard Linux network stack performed close to equal in terms of latency and the results were highly dependent on transmission rates and packet sizes.

Packet loss rates were the lowest for DPDK. io_uring had higher packet loss rates than DPDK but for larger packet sizes and lower transmission rates, the packet loss rates became similar reaching close to 0%. The standard Linux network stack had a minimum of 30% in packet loss, meaning it performed the worst in regards to packet loss.

The results indicated that DPDK performs better than the kernel-based alternatives, io_uring, and the standard Linux network stack, in regard to all the metrics that were measured in this thesis. Between io_uring and the standard Linux network stack io_uring clearly outperformed the standard Linux network stack in terms of throughput, latency and packet loss. However, in terms of latency io_uring and the standard Linux network stack appear to perform equally.

The reason behind DPDK outperforming io_uring and the standard Linux network stack, likely is a result of DPDK using kernel bypassing, thus avoiding overheads caused by the Linux kernel such as copying and context switches. io_uring outperformed the standard Linux network stack, which likely is a result of it using shared buffer rings between the user space and kernel space. This means that when using io_uring overheads can be reduced as it allows for zero copying and system calls can be made more seldom, therefore less context switching is necessary. Based on the results, the features that io_uring provides appear to increase the performance of applications using the Linux kernel, however, the increase is not enough to reach the performance of DPDK.

6.1.2 Handling of Bursty Traffic

In this section, it is discussed how well DPDK, io_uring, and the Linux network stack are able to handle bursty traffic in terms of latency and packet loss. Each application was tested with bursty traffic relative to their maximum performance, where the low traffic rates had a mean of 10% of their maximum performance and the high traffic rates were 20% above their maximum performance. DPDK showed a maximum performance at a transmission rate of 64% while the kernel-based alternatives showed a maximum performance at a transmission rate of 16%. Therefore, the values that DPDK was tested with were on average four times higher than those of the kernel-based alternatives.

Overall, it was clear that out of the three applications, the application using DPDK was the best at handling the low-traffic state both in terms of latency and packet loss. Between the applications utilizing io_uring and the standard Linux network stack results for the low-traffic state differed. io_uring had better performance in terms of packet loss while the Linux network stack had slightly better results in terms of latency. The average packet loss rates and latency values during the low-traffic state are shown in Table 6.1.

Low-traffic state	**DPDK**	**io_uring**	**Standard Linux network stack**
Packet loss rate (%)	0	8	14
Latency (μs)	22	35	29

Table 6.1: Summary of average packet loss and latency during low-traffic bursts

When evaluating the high-traffic state of each application the outcome differs from the low-traffic state. io_uring and the Linux network stack outperformed DPDK in terms of latency, where DPDK had up to ten times higher latency for some high-traffic bursts. This means that even though the packet loss was considerably higher for the kernel-based packet processing solutions their result regarding latency was significantly worse than for DPDK during heavy traffic loads relative to their maximum performance. In table 6.2 the average packet loss and latency values for the high-traffic state are summarized.

High-traffic state	**DPDK**	**io_uring**	**Standard Linux network stack**
Packet loss rate (%)	7	37	55
Latency (μs)	1100	150	120

Table 6.2: Summary of average packet loss and latency during high-traffic bursts

Based on the results in Table 6.1.2 it is clear that DPDK is to be the preferred approach when dealing with packet processing at low data traffic rates relative to what the application and system are able to handle. However, if a user wants to utilize built-in functionalities and drivers in the kernel for handling certain protocols then DPDK is not an option and then the question becomes whether the Linux network stack or io_uring is better suited for dealing with low-traffic data flow rates. The results show only a small difference in latency and packet loss between io_uring and the Linux network stack, thus utilizing either of them should yield similar performance results. However if low packet loss is a priority io_uring should be used, and if it is important to minimize the latency the standard Linux network stack could be the better option.

At heavy traffic loads, around 20% over what the application is able to handle, DPDK showed good performance in terms of packet loss but the latency values were close to ten times larger than the kernel-based alternatives, potentially making it less useful for the heavy data traffic loads. io_uring and the standard Linux network stack were once again closely matched with io_uring showing lower packet loss rates and the standard Linux network stack was better in terms of latency.

Based on the simulation of heavy and low traffic loads with the switched Poisson process the results are ambiguous, and which application performance should be chosen is strongly associated with the incoming data traffic. If the bursts of incoming traffic tend to lean heavily towards low traffic in relation to what the application and system are able to handle, then it would make sense to choose DPDK as it showed the overall best performance of the three.

If the incoming burst of traffic has periods where the incoming traffic rates surpass what the application and system are able to handle a kernel-based solution would perhaps be more suitable if latency is considered important as DPDK can generate large latency values whenever it is experiencing packet loss. This does not mean that DPDK is less suitable for higher traffic rates than io_uring and the Linux network stack, as our study concludes that DPDK is able to handle higher traffic rates overall than the kernel-based alternatives before packet loss starts to occur. If it would be possible to reach the same kind of networking performance with io_uring or the standard Linux network stack, by either tuning the kernel or optimizing the application further, it would be better to use these kernel-based alternatives when considering latency. The reason is that they have lower latencies when handling traffic loads that are above their maximum performance and therefore offer greater opportunities in scalability in terms of latency. If packet loss is considered more important than latency, then DPDK is considerably better for handling heavy traffic loads than io_uring and DPDK.

Whenever packet loss starts to occur for DPDK the latency increases sharply which could be a result of congestion. TestPMD, the application used for testing DPDK, uses as a default the run-to-completion model described in section 2.5.3. This means it uses polling to access I/O devices and has no support from a scheduler. With the run-to-completion model, the one CPU handles both the polling and processing of packets. Although using polling in this way can increase performance by avoiding interrupt overheads it also allows for possible difficulties in scaling when the traffic loads exceed what it is capable of handling and congestion starts to occur. In contrast to DPDK, the kernel-based alternative has ways of dealing with congestion such as network buffers, and packet queuing and scheduling, which may therefore offer greater scalability out of the box than DPDK without any additions. The buffer sizes used by DPDK and the kernel are a possible factor as well in the result. Larger buffers tend to translate into less packet loss but at the cost of increased latency as fewer packets will be dropped and instead wait in the buffer longer until they are transmitted. Therefore, it is also possible that the high latency values generated by DPDK are a consequence of having large buffer sizes and would the buffer sizes have been decreased it could have resulted in increased packet loss and decreased latency for DPDK during heavy traffic loads.

6.1.3 Tuning the Linux Kernel Network Parameters

The results of tuning the kernel showed the most impact on network performance when configuring RPS, RX flow hash indirection table, hardware buffer queue, socket receive queue memory, socket transmit queue memory, and netdev_budget parameters. Their impact on performance showed relatively small increases and decreases in performance. In this section, there is an in-depth discussion about the outcome of each parameter and what the result could mean from a more technical viewpoint.

6.1.3.1 netdev_budget

When tuning the netdev_budget from 300 to 1500 most of the performance metrics did not change when running the performance tests again. The only clear difference was that the latency histogram did yield a worse result than before the change of the netdev_budget. Since netdev_budget is a parameter that defines how many packets get handled at every given interrupt, it could result in higher latency if each interrupt takes a longer time. This could therefore possibly make other parts of the system slower, thus giving it a slightly worse latency. For the throughput, packet loss, and packet rate the results do not show any significant difference. Some reasons that the netdev_budget does not increase performance could be that the CPU cores are already running at full capacity, such that it may not increase any performance if increasing netdev_budget. It could also be so that another bottleneck exists on another part of the system and therefore the change of the netdev_budget would not improve the results.

6.1.3.2 RX Flow Hash Indirection Table and Hash Function

Changing the RX flow hash indirection table of the network interface from using its maximum of four receive queues to only one had a positive impact on throughput, packet rate, and latency. This was an unexpected result as distributing packets to different receive queues should allow for more CPU cores to be able to handle the packets. One possible reason is that when distributing packets to one receive queue it allowed for better distribution of packets to CPU cores that were slightly better suited for processing the packets. The CPU cores receiving packets could possibly utilize memory resources and memory cache more effectively, thus leading to an increase in performance. The observed behavior is not an indication that this would be the case on another system in which the mapping and use of receive queues and CPU cores might be different.

6.1.3.3 Hardware Buffer Queue

Increasing the size of the hardware buffer queue to four times its original size showed a large increase in latency. Decreasing the hardware buffer queue decreased the latency for certain packet sizes and transmission rates. The increase seen in latency when the hardware buffer queue was increased could be a result of packets being queued for a longer time than before. The difference in packet loss was still negligible compared to before the tuning of the hardware buffer queue. The reason behind this could be that the hardware buffer queue was not a bottleneck so increasing it would not then yield any substantial results.

From the study, the results indicated that changing the size of the hardware buffer queue had an impact on performance. Therefore, if the goal is to find the optimal hardware buffer size for decreasing the latency of an application using io_uring then additional testing may be required to find a good balance between latency and packet loss. If the buffer queue size is too large the latency will increase, but if the size is too small overflows may happen to cause packet loss and high latency.

6.1.3.4 Socket Receive Queue Memory

Socket receive queue memory had some impacts on packet loss, packet rate, and above all latency. When decreasing the socket receive queue memory there was a decrease in latency, packet rate, and throughput where the difference became more prominent the higher the transmission rate was.

Increasing the socket receive queue memory increased the latency, but did not have any clear impact on other metrics.

These results could likely be related to when the size of the socket receive queue memory is reduced it is not able to hold as many incoming packets in its buffer space. This could mean that the received packets that actually make it through the application and are forwarded have to spend less time in the receive queue making it appear as if the latency is decreasing as only these packets are used to calculate the latency value. Therefore the latency will decrease while the packet loss increases as the receive queue memory buffer is smaller causing more packets to be dropped. Higher packet loss would also likely affect the packet rate and throughput metrics, which is what was seen in the result.

Increasing the socket receive queue memory would allow for storing more packets in the buffer before packet loss occurs. In the results, it was not obvious that the packet loss rate decreased. It was however apparent that the latency increased and this could be a result of more packets being stored in the buffer for longer time periods causing the latency to increase.

6.1.3.5 Socket Transmit Queue Memory

Increasing the socket transmit queue memory size also increased the latency and had little to no impact on the other network metrics. This result was not surprising as increasing the socket receive queue memory size also resulted in increased latency as described in section 6.1.3.4.

As described in section 6.1.3.4, one possible explanation could be that increasing the socket transmit queue memory means that more packets can be stored in the queue memory and thus packets may have to wait longer before being transmitted. What could strengthen this theory would be if the packet loss rate dropped, but this was not the case. One reason for this could be that socket receive queue memory was not increased simultaneously and therefore even though the transmit queue was larger the performance was bottlenecked by the receive queue making the packet loss appear the same as it was before.

6.1.3.6 Receive Packet Steering

Using RPS to ensure that one CPU core was running the application and another CPU core on the same NUMA node was handling the interrupt had a significant impact on the performance. Latency, packets per second, and Mbit/s increased while packet loss decreased. It was important that the CPU cores shared the same NUMA node as it allows for more shared memory resources leading to improved performance.

A reason why the performance increased when dedicating CPU cores to specific tasks could be that making each CPU core dedicated to one task could reduce context switches and reduce contention between cores. Removing overheads like these would potentially increase performance as it could result in reduced packet loss and higher throughput rates. The increase in latency could possibly be explained by an overwhelming amount of interrupts on the core dedicated to interrupts, which could then have caused the latency to increase as the core could not process them promptly.

When using RPS to instead ensure that the same CPU core that was running the application was also handling interrupts, it resulted in increased packet loss and a decrease in throughput as well as packets per second. In terms of latency, it decreased for low packet sizes but increased for larger packet sizes. High transmission rates, 64% and higher, were outliers in this case as they had lower latency for all packet sizes. Another interesting trend

for latency was that extremely high latency was recorded for certain packet sizes and transmission rates.

The decrease in performance when the same CPU that was running the application was also handling incoming interrupts can likely be explained by CPU core overload as it places a heavy load on a single CPU core. The application requires CPU cycles for executing the application logic while the interrupt handling requires immediate attention from the CPU. This could limit the core's ability to handle both tasks effectively as it will have to do context switching often causing large overheads, thus impacting the performance negatively. The trend of spikes in latency for certain transmission rates and packet sizes in figure 5.54 can possibly be attributed to system specifics such as queue management, buffer sizes and when context switching occurs, making the congestion increase drastically for certain combinations of packet sizes and transmission rate.

6.1.3.7 Generic Receive Offloading

Turning off the GRO setting did not have any significant impact on throughput, latency, packet loss, or number of packets per second. There could be multiple reasons for this outcome. There might have been a performance increase but since there is a possibility that the application is a bottleneck, the results will not show as clearly.

6.1.3.8 ip_early_demux

Enabling ip_early_demux did not yield any performance difference. It is believed to not have had any impact, or a very small impact. When considering the performance as mentioned at the beginning of section 6.1.3 the io_uring application is believed to have been a bottleneck, making a subtle difference in performance hard to notice. It is also possible that enabling ip_early_demux does not optimize the processing significantly and therefore would not have impacted the performance even if the application did not prove a bottleneck.

6.1.3.9 netdev_max_backlog

When tuning the netdev_max_backlog parameter to five times its original size it did not give any significant results. When the results do not either get worse or better it is important to consider bottlenecks that could be present that make it hard to see if the tuning yields a better result. Since it did not affect the result, one reason could be that netdev_max_backlog never was a bottleneck. In other words, this means that the default value of 1000 was never a problem and so when setting it to 5000 the result would be unchanged.

6.2 Evaluation of Methodologies

This section will discuss the method, its credibility, and what could have been done in a different way. First, a more general discussion of the method is conducted, and then a more specific discussion regarding each of the methodologies used for the research questions.

6.2.1 General Evaluation of Method

The method used to answer the research questions has some similarities in the setup and data collection. For example, the same base setup using Pktgen and an SR-IOV-enabled NIC with virtual functions was used. First of all, Pktgen could have a minor bias towards DPDK since it is built by the same team as DPDK. This could yield better results for DPDK since it could be optimized for DPDK. Another reason is that Pktgen is still much under development and there could exist bugs that could give misleading results or features that are not completed yet.

Pktgen is still being developed and the public GitHub repository received several updates during the time period when the study was conducted. After finding bugs within Pktgen when conducting the study, the authors of this study were able to message the developer of Pktgen about the bugs.

Each performance test also takes a very long time to run. It also needs much manual supervision. The reason for this is that Pktgen may crash unexpectedly whenever too many configurations have been done or it has run for a long period of time. This could affect the result in a negative way since less time to do tests could yield less accurate results. If more time would have been available more test runs could have been executed to be able to see a normal distribution of the result between different runs. This could have been useful for finding out how much the result could differ between runs.

The method does also evaluate the results using the same hardware for all tests. To get a more accurate performance result from each of the applications, different hardware setups would have been a good alternative. This would make the result more versatile and could give more edge cases for when different applications are a better alternative.

6.2.2 Evaluation of Benchmarking Methodology

The method consisted of implementing three different setups that could test each of the applications. Pktgen was used for all setups. For DPDK, TestPMD was the forwarding application used to do the performance test. The io_uring and the standard Linux network stack applications were written by the authors, who are two computer science students with limited experience in computer networking compared to professionals. As TestPMD was written by professionals and maintainers of DPDK, that also has been developed for several years, it is highly optimized for showing the high performance of DPDK. This could be a cause for an unfair comparison between the three alternatives. The comparison between io_uring and the standard Linux network stack should be relatively fair, but not all io_uring features were tested and thus it is possible that some features that could improve its performance further were missed.

6.2.3 Evaluation Methodology for Simulating Bursty Traffic

The bursty traffic generated for each application was based on the applications' maximum performance. The reasoning behind using bursty traffic relative to the applications' performance instead of using the same transmission rate for all, was to create a fair comparison of how the applications are able to perform relative to their maximum network performance.

Although this works for creating a relatively fair comparison between the applications, it may also result in some less fair outcomes. In the results, it was clear that whenever the packet loss of DPDK started to increase the latency spiked extremely high, from around 0 μs to about 1,200 μs. This could be an indication of DPDK not being able to handle packet loss as well, but it could also be a result of the mean transmission rate used for testing high bursty traffic rates on DPDK being four times higher than it was for io_uring and Linux network stack. As the transmission rate used for DPDK's high traffic rate state was relatively high this could potentially mean that whenever packet loss occurs the congestion builds up very quickly thus delaying the packets being forwarded back to their source. Had DPDK been tested on the same transmission rate as io_uring and the standard Linux network stack of 16% the results would likely have been different and the average latency value of DPDK during the simulation of heavy traffic loads would have been lower.

Even though the method used using relative performance clearly has its flaws, it remains one of the few ways of simulating heavy traffic loads with switched Poisson processes and without using real traffic data. A relative measurement approach of DPDK maximum network performance seems to be the best possible solution for testing how it performs under

both low and heavy data traffic loads. The same conclusion can be drawn for measuring for io_uring and Linux network stack.

Had an absolute performance measurement been used the outcome would have been different depending on what mean values were chosen. If a mean of 64% would have been used for all applications, then it would have resulted in good measurement for DPDK, but for the kernel-based alternatives this would have meant that both the low and high-traffic states would have been regarded as relatively heavy loads resulting in an unfair comparison. The same goes for 16%, but then the results would instead have been less representative for DPDK as both the low and high-traffic states would have been regarded as low-traffic for the DPDK application.

TestPMD uses as default the run-to-completion model which could, as explained in section 6.1.2, be a reason for the latency increase whenever the traffic load was above its maximum capacity and packet loss started to occur. Another approach to using DPDK could have been to configure it to use the pipeline model instead of the run-to-completion model. Using the pipeline model would enable the use of packet processing on multiple cores and could therefore help increase throughput. Even though this could have helped increase some network performance metrics it would likely not have changed the final results as the pipeline is not a congestion mechanism and therefore eventually when packet loss occurs the results would have been the same and latency would still see a large increase.

6.2.4 Evaluation of Methodology Used for Tuning the Kernel

The parameters that were chosen to tune network kernel parameters were based on available online resources and Linux kernel documentation. The Linux kernel contains many parameters that can be tuned and it is possible that parameters that could have had a significant impact on performance were missed, as there are no clear guidelines for how the kernel should be tuned for optimal network performance.

Tuning kernel parameters and evaluating their impact on the network performance indicates that the io_uring application is a possible bottleneck, as some parameters that were expected to result in a performance difference did not impact any of the metrics that were measured. What this means is that even though some parameters may have had a potential impact on the kernel, the differences were too subtle to be noticed as the impact by the bottleneck was too large.

6.3 The Work in a Wider Context

In a wider context, this can help clarify in which systems and setups the different applications do perform best. This could help organizations and users in choosing the correct software for its purpose which could save computer resources and increase networking performance. By using the best application and setup for network traffic, companies can increase the usability of their product and increase the overall quality of experience. From an ethical perspective, only open-source software was investigated to contribute to the open-source community. By using the same hardware and just altering the software, it is possible to achieve better performance which makes better performance accessible to more people. From a social perspective, the thesis contributes to the privacy and safety of users since more efficient networks are more prone to withstand data loss and delays in the data being transferred.

7 Conclusion

In this chapter the research questions each has a dedicated section explaining the purpose of the research question, to what extent the aim of the question was achieved, and whether the conclusion is trustworthy. At the end of the chapter future works are discussed.

7.1 Benchmarking DPDK, io_uring and the standard Linux Network Stack

The first research question was related to benchmarking the network performance of DPDK, io_uring, and the Linux network stack. It was defined as follows:

- How does the network performance of io_uring and DPDK compare to the standard Linux network stack in terms of latency, packet loss, throughput (bps), and packet rate (pps)?

io_uring takes a new approach to dealing with packet processing while still utilizing the Linux kernel, and there has been interest in seeing how well its performance compares to other more well-established packet processing mechanisms and techniques such as DPDK and the Linux network stack. Research question 1 was designed to investigate whether io_uring is able to perform near the performance of DPDK that uses kernel-bypass to increase performance, and how well it outperforms the standard Linux network stack.

The results showed that DPDK performed the best in all metrics and was able to handle transmission rates of 64% of what the NIC was able to produce. io_uring and the Linux network stack were able to handle transmission rates up to 16%. DPDK was able to generate throughput values that were around five times higher than io_uring, and the Linux network stack performed the worst with more than seven times lower throughput than DPDK. In terms of latency, DPDK also outperformed io_uring and the Linux network stack, in some cases with values 100 times lower. The Linux network stack and io_uring performed very close to each other in terms of latency and no clear second place could be decided. DPDK had close to 0% packet loss for most of the transmission rates and packet sizes that were tested while io_uring and the standard Linux network stack showed higher packet loss rates with varying results. Comparing io_uring to the Linux network stack in terms of packet loss rates, io_uring outperformed the Linux network stack where it for some specific packet sizes

and transmission rates was close to DPDK's performance of close to 0% packet loss, while the lower packet loss rates achieved by the standard Linux network stack were between 30% to 40%.

DPDK was able to outperform both io_uring and the Linux network stack as it uses a kernel bypass technique to avoid the Linux network stack resulting in a mitigation of packet processing overheads caused by the Linux kernel. Between io_uring and the Linux network stack, io_uring was able to outperform the Linux network stack in terms of throughput and packet loss, while the difference in latency was determined to be similar between the two. Thus, it was concluded that io_uring does outperform the Linux network stack, but is not able to reach the same high performance as DPDK. The reason for io_uring being able to outperform the standard Linux network stack is likely related to several factors. One of the key factors is believed to be that io_uring uses ring buffers to make expensive system calls less frequent.

When doing the benchmarking an already existing and optimized application for high performance was used for DPDK, while for io_uring and the Linux network stack an application had to be created for each. The traffic generator that was used for the testing was powered by the DPDK framework and this could potentially cause a bias in favor of DPDK as it was developed for DPDK. Whether this bias exists or not would require some additional investigation, but it is important to consider when looking at the results. As these factors could have impacted the final results. However, the overall difference between DPDK and the kernel-based alternatives was large enough that it should be reasonable to make the assumption that the conclusions would have been similar regardless if these factors existed or not.

7.2 Handling of Bursty Traffic

Research question two was defined as follows:

- How well do io_uring, DPDK, and the Linux network stack handle bursty traffic data considering latency and packet loss?

The purpose of the research question was to simulate a traffic flow that acted more like real-world flows of data to see how well the different ways of handling packet processing would handle it. It was decided that the best way of doing this was to use a Switched Poisson distribution with a Markov chain consisting of a low-traffic state and a high-traffic state, where the low-traffic state was more commonly occurring. The low-traffic state was determined to be 10% of each application's maximum performance and the high-traffic state was chosen as 20% above it.

The results showed that DPDK was best at handling the low-traffic state out of the three methods, where it had both the lowest latency and packet loss. Amongst the kernel-based alternatives, io_uring and the Linux network stack, io_uring had the edge in terms of packet loss and the standard Linux network stack showed a small edge in terms of latency. For the high-traffic rate state, DPDK experienced some packet loss and very high latency values, making it the least useful in terms of latency for the high-traffic state when the transmission rates were over the application's maximum performance. The kernel-based alternatives showed the same pattern as they did for the low-traffic state, but in this case, both had lower latency during the high-traffic bursts than DPDK. One possible conclusion that could be drawn from this is that DPDK, in its original state, has a lower potential for scalability as the latency sharply increases whenever packet loss occurs. As io_uring and the Linux network stack both have built-in support within the kernel to handle congestion, they did not show as severe a decline in performance as DPDK when experiencing heavy traffic loads relative to their maximum performance.

One thing to take into consideration is that these results are based on testing bursts of traffic relative to their own maximum performance. What this means is that even though DPDK seemed to perform worse in terms of latency during the heavy traffic load it was also experiencing bursts of traffic that were around four times higher than the ones used for io_uring and the Linux network stack. Had the measurements been done on the same burst of traffic the final results could have been different, but then it would not have been possible to test the applications both during high and low traffic loads.

7.3 Tuning Linux Kernel Parameters for Network Performance

The third research question was about how tuning Linux kernel parameters may impact the network performance of applications that utilizes io_uring. The research question was defined as:

- What impact does tuning network parameters in the Linux kernel have on applications utilizing io_uring?

The reason for investigating how parameters in the Linux kernel can be tuned to increase network performance was to investigate how much performance can be increased for applications using io_uring and if the tuning would bring io_uring close to the performance of DPDK.

From tuning a total of nine parameters in the kernel that are related to computer networking, it was possible to determine that tuning these variables can have an impact on the network performance for applications using io_uring. Although many of the variables had an impact on network performance the results were not large enough to bring io_uring close the the performance of DPDK. The most significant results were seen through tuning the RPS, RX flow hash indirection table, hardware buffer queue, socket receive queue memory, socket transmit queue memory, and netdev_budget, where RPS had the largest positive impact on network performance.

There were two limiting factors that impacted the final result. The io_uring application was a potential bottleneck making it so that tuning parameters in the kernel had less of a significant impact as the application's performance could not be improved over a certain point. This could have also caused some more subtle changes after tuning kernel parameters to go unnoticed as these were bottlenecked by the application. Another limitation was the test environment setup, where all the used CPU cores were isolated. Although this allows for more reliable results, as no other tasks are scheduled on isolated CPU cores, it would also be interesting to test on CPU cores that were not isolated.

7.4 Future Work

There are multiple paths that could be taken to further investigate the performance differences of DPDK, io_uring, and the standard Linux network stack. The first could be to evaluate factors like power consumption. This thesis has strictly focused on stand-alone performance whilst an important factor to take into consideration is the cost of running the different alternatives. This could give rise to cases where applications have a lower performance but also low power consumption, thus making it a better choice for specific use cases.

Another future work aspect would be to evaluate the scalability of the different applications and the setups. This thesis has been running everything on a single machine, and therefore it would be interesting to also evaluate the performance on a larger scale. This could include using multiple machines in a data center for example, where scalability and large-scale deployments are important. It could also be of interest to test the same setup but with different hardware that is more powerful, like for example a 100 Gbit/s NIC or a more

powerful CPU. A reason that this is a good path for future work is that DPDK and io_uring could be more beneficial for large-scale setups or different hardware.

Investigating how decreasing DPDK buffer sizes would affect its ability to handle bursty traffic above its maximum performance would also be an interesting future study. The results of this kind of study would help in determining if DPDK has a worse ability to scale in terms of latency than kernel-based alternatives, or if the high latency values seen when DPDK experiences packet loss are simply a result of large buffer sizes.

More future work could be to do performance testing with different kinds of data transmissions. To give a better evaluation of the individual performance of the applications real-time video, large files, and a larger variety of bursty traffic could be examined. This would give a good picture of when the different applications are performing at their best and make it easier to choose the best application for the given task.

An area that was neither discussed nor measured in this thesis is how DPDK, io_uring, and the standard Linux network stack handles data and how secure they are. Since the thesis revolves around sending data, one can not ignore that private data or secure information could be sent. Therefore a very important facet would be to test how data is handled and how secure the transmission is for the three different setups.

TCP is also relevant since a large portion of the internet is using this protocol instead of UDP. It would therefore be a good field for research to compare how DPDK, io_uring, and the standard Linux network stack handles TCP in contrast to UDP. This could create new possibilities since the outcome might not be the same as the one displayed in this thesis.

Lastly, it is important to notice that the Linux kernel, DPDK, and io_uring are under development. Therefore, it would be relevant to do similar measurements, as done in this thesis, in the future.

Bibliography

[1] Soumya Mahalakshmi A, Amulya B S, and Minal Moharir. "A study of tools to develop a traffic generator for L4 – L7 layers". In: *2016 International Conference on Wireless Communications, Signal Processing and Networking (WiSPNET)*. 2016, pp. 114–118. DOI: 10.1109/WiSPNET.2016.7566102.

[2] Åsa Arvidsson and Richard Harris. "Analysis of the Accuracy of Bursty Traffic Models". In: *Proceedings of the First International Conference On Telecommunication Systems Modelling and Analysis*. 1993. URL: https://www.diva-portal.org/smash/record.jsf?pid=diva2%5C%3A838438&dswid=-1130.

[3] AvidThink and Converge! Network Digest. *Myth-busting DPDK in 2020*. Tech. rep. Intel Corporation, 2020. URL: https://nextgeninfra.io/wp-content/uploads/2020/07/AvidThink-Linux-Foundation-Myth-busting-DPDK-in-2020-Research-Brief-REV-B.pdf.

[4] Jens Axboe. *IO-Uring UDP echo server example*. 2022. URL: https://github.com/axboe/liburing/blob/master/examples/io_uring-udp.c (visited on 04/16/2023).

[5] Jens Axboe. *io_uring and networking in 2023*. URL: https://github.com/axboe/liburing/wiki/io_uring-and-networking-in-2023 (visited on 04/08/2023).

[6] Jens Axboe. *Welcome to FIO's documentation!* 2017. URL: https://fio.readthedocs.io/en/latest/index.html (visited on 11/29/2022).

[7] Prof. Shashi Banzal. *Data and Computer Network Communication, Third Edition*. Laxmi Publications © 2022, 2022.

[8] Thomas Begin, Bruno Baynat, Guillaume Artero Gallardo, and Vincent Jardin. "An Accurate and Efficient Modeling Framework for the Performance Evaluation of DPDK-Based Virtual Switches". In: *IEEE Transactions on Network and Service Management* 15.4 (2018), pp. 1407–1421. DOI: 10.1109/TNSM.2018.2874476.

[9] Christian Benvenuti. *Understanding Linux Network Internals*. 1st ed. O'Reilly Media, Inc., 2005. ISBN: 9780596002558.

[10] Daniel P. Bovet and Marco Cesati. *Understanding the Linux Kernel*. 3rd ed. O'Reilly Media, Inc., 2005. ISBN: 9780596005658.

[11] Terrehon Bowden, Bodo Bauer, Jorge Nerin, and Shen Feng. *Documentation for /proc/sys/net*. URL: https://github.com/torvalds/linux/blob/master/Documentation/admin-guide/sysctl/net.rst (visited on 04/22/2023).

[12] *Build Ultra High-Performance Storage Applications with the Storage Performance Development Kit*. URL: https://spdk.io (visited on 05/10/2023).

[13] Qizhe Cai, Shubham Chaudhary, Midhul Vuppalapati, Jaehyun Hwang, and Rachit Agarwal. "Understanding Host Network Stack Overheads". In: *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. SIGCOMM '21. Virtual Event, USA: Association for Computing Machinery, 2021, pp. 65–77. ISBN: 9781450383837. DOI: 10.1145/3452296.3472888. URL: https://doi-org.e.bibl.liu.se/10.1145/3452296.3472888.

[14] Mary K. Stahl Carol A. Ward Elizabeth J. Feinler and Ole J. Jacobsen. *DDN (Defense Data Network) Protocol Handbook. Volume 1. DoD Military Standard Protocols*. Tech. rep. 1985. URL: https://apps.dtic.mil/sti/citations/ADA166324.

[15] Sung Nok Chiu, Dietrich Stoyan, Wilfrid S. Kendall, and Joseph Mecke. *Stochastic Geometry and Its Applications*. John Wiley & Sons, 2013. ISBN: 978-1-118-65825-3.

[16] Jonathan Corbet and Mike Rapoport. *Concepts overview*. URL: https://github.com/torvalds/linux/blob/master/Documentation/admin-guide/mm/concepts.rst (visited on 04/28/2023).

[17] IBM Corporation. *Network stack settings*. URL: https://www.ibm.com/docs/en/linux-on-systems?topic=tuning-network-stack-settings (visited on 04/22/2023).

[18] Diego Didona, Jonas Pfefferle, Nikolas Ioannou, Bernard Metzler, and Animesh Trivedi. "Understanding Modern Storage APIs: A Systematic Study of Libaio, SPDK, and Io_uring". In: *Proceedings of the 15th ACM International Conference on Systems and Storage*. SYSTOR '22. Haifa, Israel: Association for Computing Machinery, 2022, pp. 120–127. ISBN: 9781450393805. DOI: 10.1145/3534056.3534945. URL: https://doi.org/10.1145/3534056.3534945.

[19] doc.dpdk.org. *Enviroment Abstraction Layer*. 2017. URL: https://doc.dpdk.org/guides/prog_guide/env_abstraction_layer.html#environment-abstraction-layer (visited on 02/26/2023).

[20] Shen Feng. *Documentation for /proc/sys/net/*. URL: https://github.com/torvalds/linux/blob/master/Documentation/admin-guide/sysctl/net.rst (visited on 04/22/2023).

[21] Behrouz A. Forouzan. *Data Communications and Networking*. 4th ed. McGraw-Hill, 2007.

[22] Sebastian Gallenmüller, Paul Emmerich, Florian Wohlfart, Daniel Raumer, and Georg Carle. "Comparison of frameworks for high-performance packet IO". In: *2015 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2015, pp. 29–38. DOI: 10.1109/ANCS.2015.7110118.

[23] Pablo De Lara Guarch. *Testing DPDK Performance and Features with TestPMD*. 2017-03-21. URL: https://www.intel.com/content/www/us/en/developer/articles/technical/testing-dpdk-performance-and-features-with-testpmd.html (visited on 02/09/2023).

[24] Sangjin Han. "System Design for Software Packet Processing". PhD thesis. EECS Department, University of California, Berkeley, Aug. 2019. URL: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2019/EECS-2019-112.html.

[25] Tom Herbert Herbert and Willem de Bruijn. *Scaling in the Linux Networking Stack*. URL: https://github.com/torvalds/linux/blob/master/Documentation/networking/scaling.rst (visited on 04/21/2023).

[26] Craig Hunt. *TCP/IP Network Administration*. 3rd ed. O'Reilly Media, Inc., 2002. ISBN: 9780596002978.

[27] Shuveb Hussain. *What is io_uring?* 2020. URL: https://unixism.net/loti/what_is_io_uring.html (visited on 11/29/2022).

[28] Shuveb Hussain. *What is io_uring?* URL: https://unixism.net/loti/what_is_io_uring.html (visited on 01/30/2023).

[29] Jon Maxwell Jamie Bainbridge. *Red Hat Enterprise Linux Network Performance Tuning Guide*. 2015. URL: https://access.redhat.com/sites/default/files/attachments/20150325_network_performance_tuning.pdf (visited on 05/04/2023).

[30] Bharathi Jayathirtha. *HugePages*. 2017. URL: https://docs.oracle.com/database/121/UNXAR/appi_vlm.htm#UNXAR391 (visited on 03/10/2023).

[31] Mengxiao Jin, Canshuai Wang, Peng Li, and Zhijie Han. "Survey of Load Balancing Method Based on DPDK". In: *2018 IEEE 4th International Conference on Big Data Security on Cloud (BigDataSecurity), IEEE International Conference on High Performance and Smart Computing, (HPSC) and IEEE International Conference on Intelligent Data and Security (IDS)*. 2018, pp. 222–224. DOI: 10.1109/BDS/HPSC/IDS18.2018.00054.

[32] kernel.dk. *Efficient IO with io_uring*. 2019. URL: https://kernel.dk/io_uring.pdf (visited on 11/29/2022).

[33] James F. Kurose and Keith W. Ross. *Computer Networking: A Top-Down Approach (6th Edition)*. 6th. Pearson, 2012. ISBN: 0132856204.

[34] Leonardo Lai, Gabriele Ara, Tommaso Cucinotta, Koteswararao Kondepu, and Luca Valcarenghi. "Ultra-low Latency NFV Services Using DPDK". In: *2021 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*. 2021, pp. 8–14. DOI: 10.1109/NFV-SDN53031.2021.9665131.

[35] Xuesong Li, Wenxue Cheng, Tong Zhang, Jing Xie, Fengyuan Ren, and Bailong Yang. "Power Efficient High Performance Packet I/O". In: *Proceedings of the 47th International Conference on Parallel Processing*. ICPP 2018. Eugene, OR, USA: Association for Computing Machinery, 2018. ISBN: 9781450365109. DOI: 10.1145/3225058.3225129. URL: https://doi-org.e.bibl.liu.se/10.1145/3225058.3225129.

[36] David S. Miller. *IP Sysctl*. URL: https://github.com/torvalds/linux/blob/master/Documentation/networking/ip-sysctl.rst (visited on 04/22/2023).

[37] Jeff Moyer. *libaio*. URL: https://pagure.io/libaio (visited on 05/10/2023).

[38] Milan Navrátil, Laura Bailey, and Charlie Boyle. *Red Hat Enterprise Linux 7 Performance Tuning Guide*. 2018. URL: https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/7/pdf/performance_tuning_guide/red_hat_enterprise_linux-7-performance_tuning_guide-en-us.pdf (visited on 05/06/2023).

[39] Ntop. *PF_RING ZC (Zero Copy)*. 2023. URL: https://www.ntop.org/products/packet-capture/pf_ring/pf_ring-zc-zero-copy/ (visited on 01/30/2023).

[40] David A Patterson and John L Hennessy. *Computer organization and design: the hardware/-software interface*. 3rd. Morgan Kaufmann Publishers, 2005.

[41] Adam Paul. *All-in-One CCIE Data Center V2.1 400-151 Written Exam Cert Guide, 3rd Edition*. CCIEin8Weeks © 2019, 2019.

[42] Larry L. Peterson and Bruce S. Davie. *Computer Networks: A Systems Approach*. 5th. San Francisco, CA, USA: Morgan Kaufmann, 2011. ISBN: 9780123850591.

[43] DPDK Project. *ABOUT DPDK*. 2022. URL: https://www.dpdk.org/about/ (visited on 11/15/2022).

[44] DPDK Project. *Compiling the DPDK Target from Source*. 2023. URL: http://doc.dpdk.org/guides-16.04/linux_gsg/build_dpdk.html#loading-modules-to-enable-userspace-io-for-dpdk (visited on 03/08/2023).

[45] DPDK Project. *Generic Receive Offload Library*. 2023. URL: https://doc.dpdk.org/guides/prog_guide/generic_receive_offload_lib.html (visited on 04/24/2023).

[46] DPDK Project. *Introduction*. 2023. URL: https://dpdk.readthedocs.io/en/v17.11/testpmd_app_ug/intro.html (visited on 03/08/2023).

[47] DPDK Project. *Overview*. 2023. URL: https://doc.dpdk.org/guides/prog_guide/overview.html (visited on 03/07/2023).

[48] DPDK Project. *Poll Mode Driver*. 2023. URL: https://dpdk.readthedocs.io/en/v17.11/prog_guide/poll_mode_drv.html (visited on 03/07/2023).

[49] DPDK Project. *QUICK START GUIDE*. 2023. URL: https://core.dpdk.org/doc/quick-start/ (visited on 03/10/2023).

[50] Luigi Rizzo. *netmap - the fast packet I/O framework*. 2023. URL: http://info.iet.unipi.it/~luigi/netmap/ (visited on 01/30/2023).

[51] Lee Schermerhorn and Linus Torvalds. *What is NUMA?* 2010. URL: https://github.com/torvalds/linux/blob/master/Documentation/mm/numa.rst (visited on 04/20/2023).

[52] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. *Operating system concepts*. 10th ed. Hoboken, NJ: Wiley, 2018. ISBN: 9781119320913.

[53] Jaspar Subhlok and Gary Vondran. "Optimal Latency-Throughput Tradeoffs for Data Parallel Pipelines". In: *Proceedings of the Eighth Annual ACM Symposium on Parallel Algorithms and Architectures*. SPAA '96. Padua, Italy: Association for Computing Machinery, 1996, pp. 62–71. ISBN: 0897918096. DOI: 10.1145/237502.237508. URL: https://doi.org/10.1145/237502.237508.

[54] Kun Tian, Yaozu Dong, Xiang Mi, and Haibing Guan. "sEBP: Event Based Polling for Efficient I/O Virtualization". In: *2012 IEEE International Conference on Cluster Computing*. 2012, pp. 135–143. DOI: 10.1109/CLUSTER.2012.50.

[55] Keith Wiles. *Burst traffic during a short time after startup*. 2023. URL: https://github.com/pktgen/Pktgen-DPDK/issues/145 (visited on 04/16/2023).

[56] Keith Wiles. *The Pktgen Application*. URL: https://pktgen-dpdk.readthedocs.io/en/latest/ (visited on 02/09/2023).

[57] Wenjun Zhu, Peng Li, Baozhou Luo, He Xu, and Yujie Zhang. "Research and Implementation of High Performance Traffic Processing Based on Intel DPDK". In: *2018 9th International Symposium on Parallel Architectures, Algorithms and Programming (PAAP)*. 2018, pp. 62–68. DOI: 10.1109/PAAP.2018.00018.

A Kernel tuning graphs

The results of tuning Linux kernel parameters that did not show a significant difference are shown in this chapter.

A.1 ip_early_demux

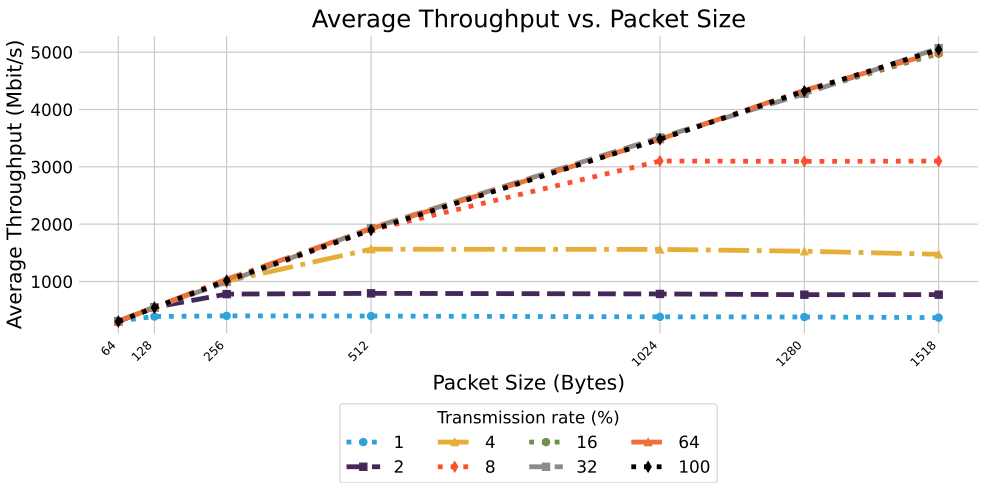

Figure A.1: Throughput of different rates and different packet sizes for io_uring with ip_early_demux turned on

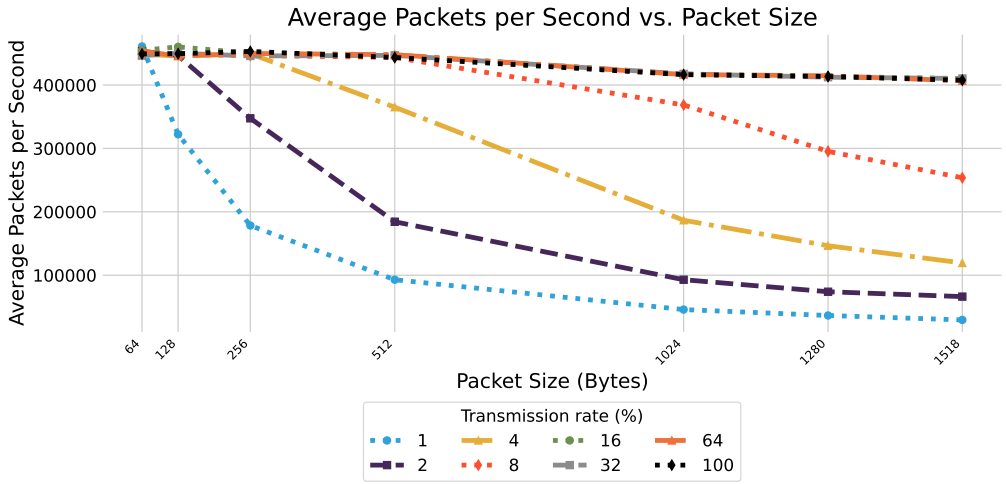

Figure A.2: Packet rate of different rates and different packet sizes for io_uring with ip_early_demux turned on

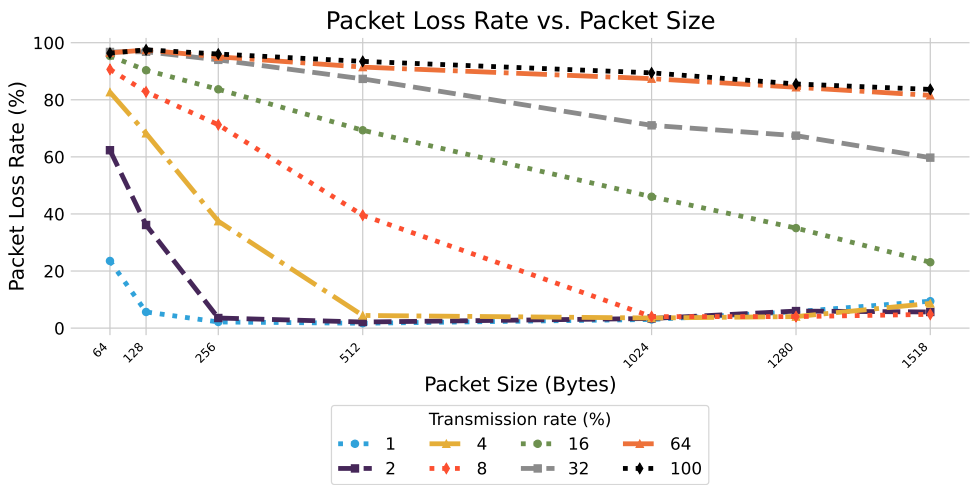

Figure A.3: Packet loss rate of different rates and different packet sizes for io_uring with ip_early_demux turned on

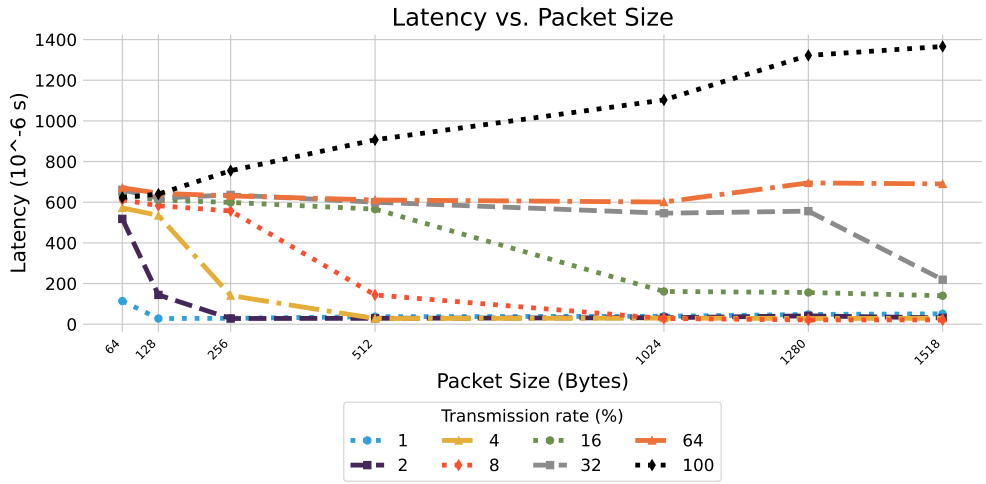

Figure A.4: Latency of different rates and different packet sizes for io_uring with ip_early_demux turned on

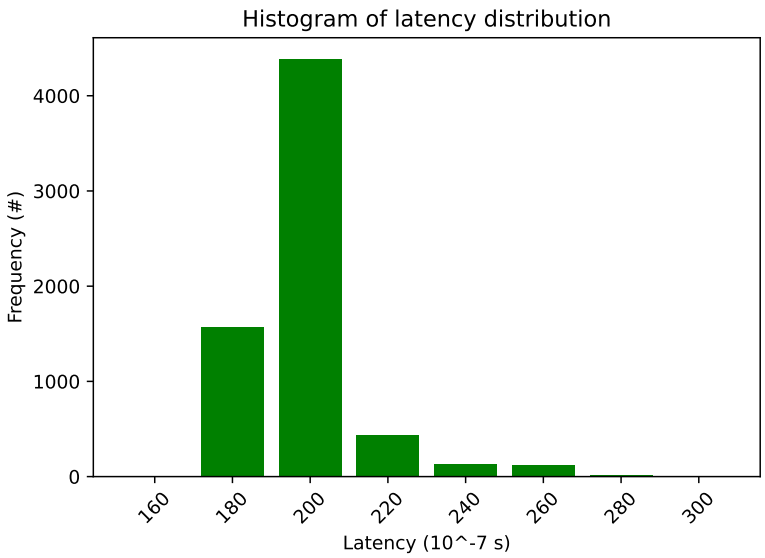

Figure A.5: Latency histogram of different rates and different packet sizes for io_uring with ip_early_demux turned on

A.2 RX flow indirection table and hash function

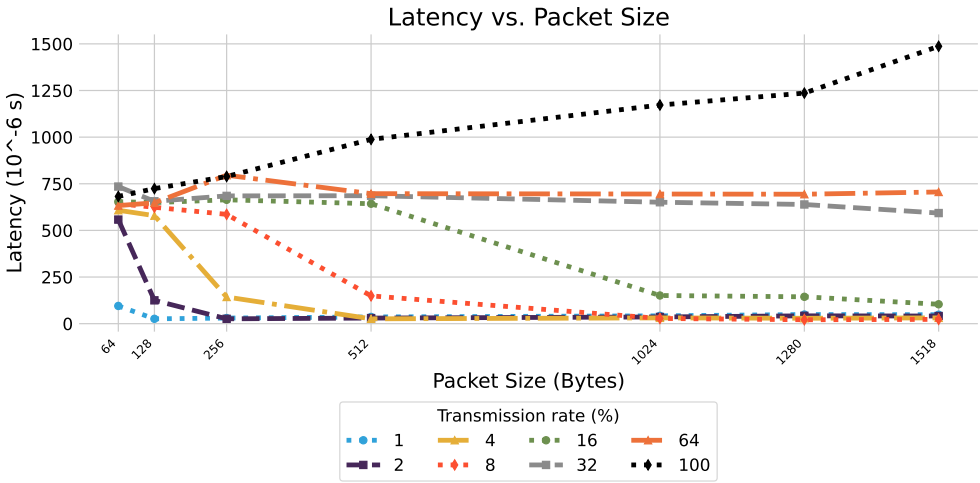

Figure A.6: Latency rates of different rates and different packet sizes for io_uring with tuned RX flow indirection table

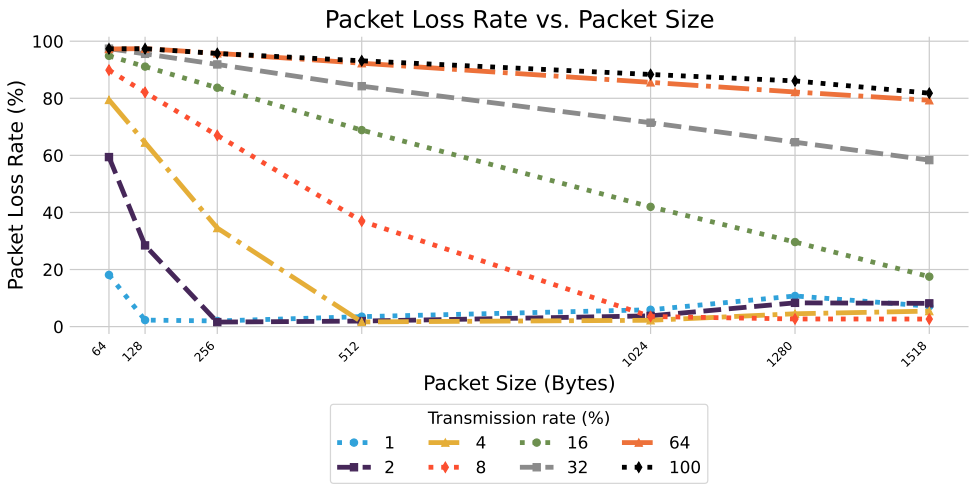

Figure A.7: Packet loss of different rates and different packet sizes for io_uring with tuned RX flow indirection table

A.3 Hardware buffer queue

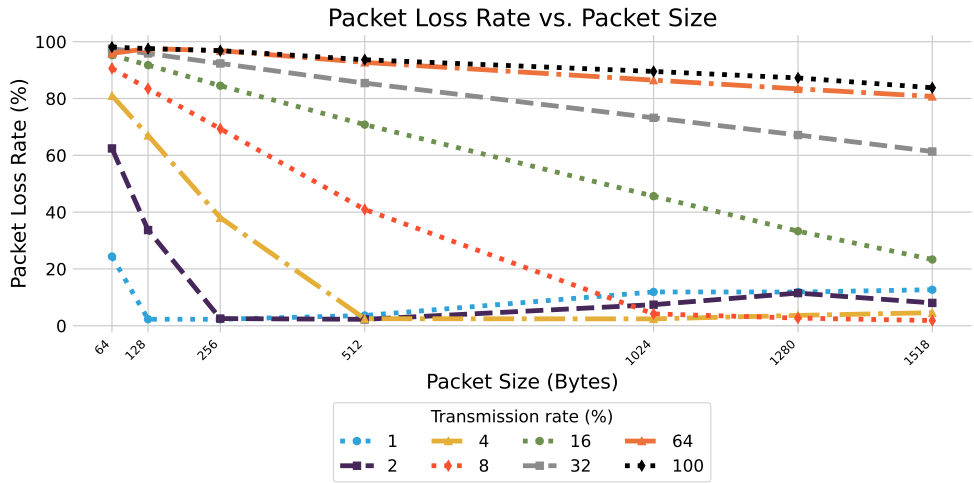

Figure A.8: Packet loss of different rates and different packet sizes for io_uring with ring buffer size set to 256 Bytes

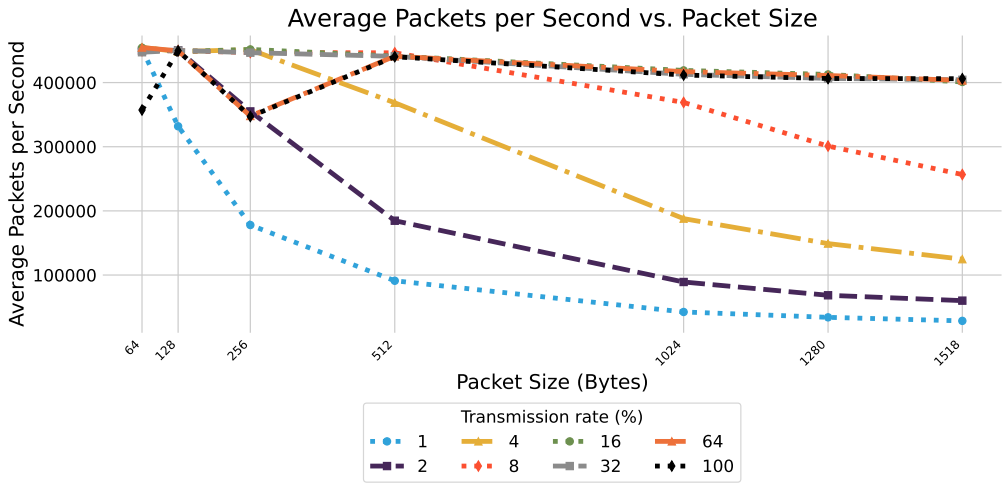

Figure A.9: Packets rate of different rates and different packet sizes for io_uring with ring buffer size set to 256 Bytes

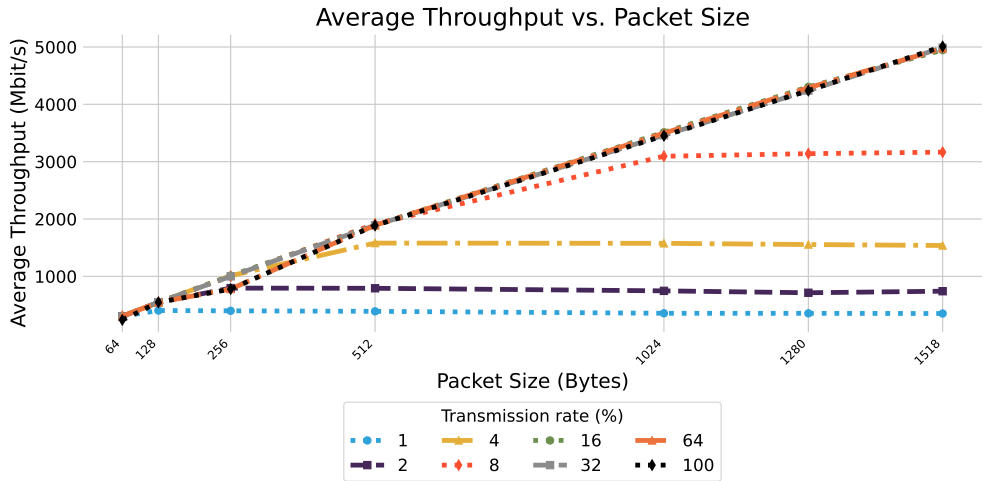

Figure A.10: Throughput of different rates and different packet sizes for io_uring with ring buffer size set to 256 Bytes

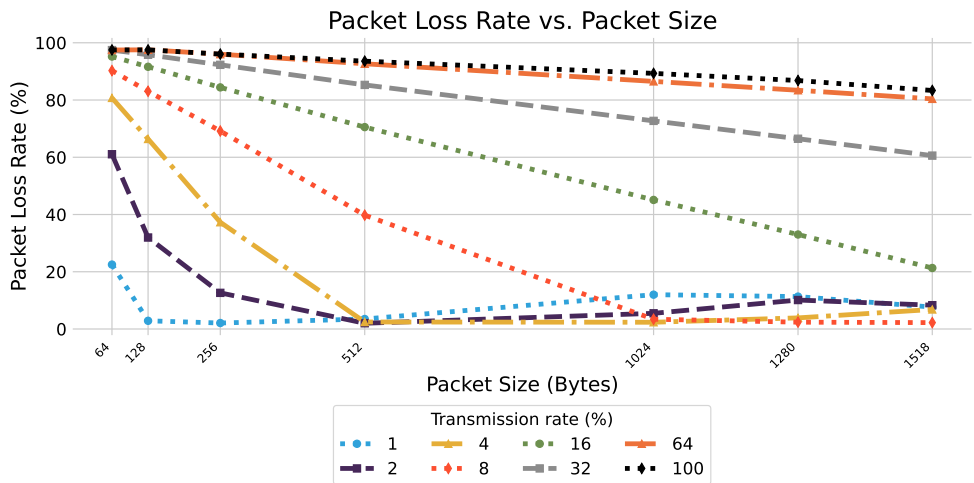

Figure A.11: Packet loss of different rates and different packet sizes for io_uring with ring buffer size set to 4096 Bytes

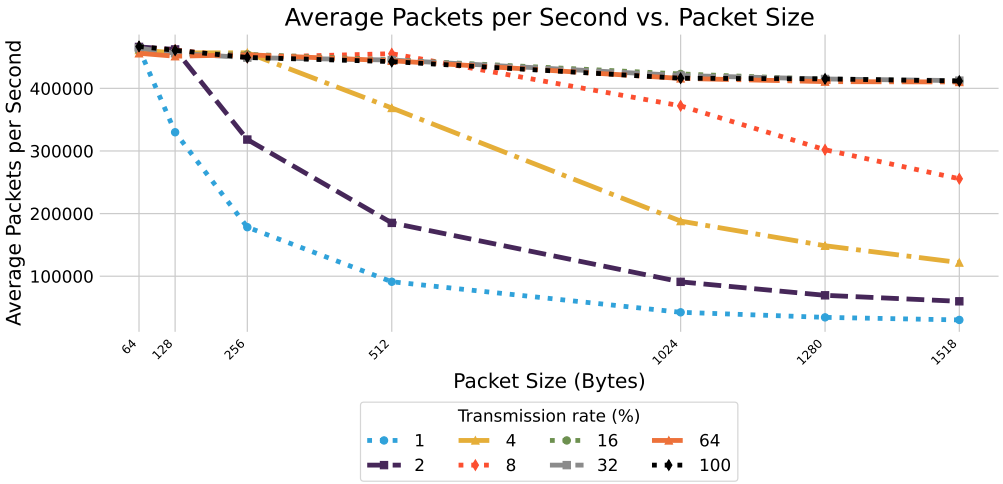

Figure A.12: Packets rate of different rates and different packet sizes for io_uring with a ring buffer size set to 4096 Bytes

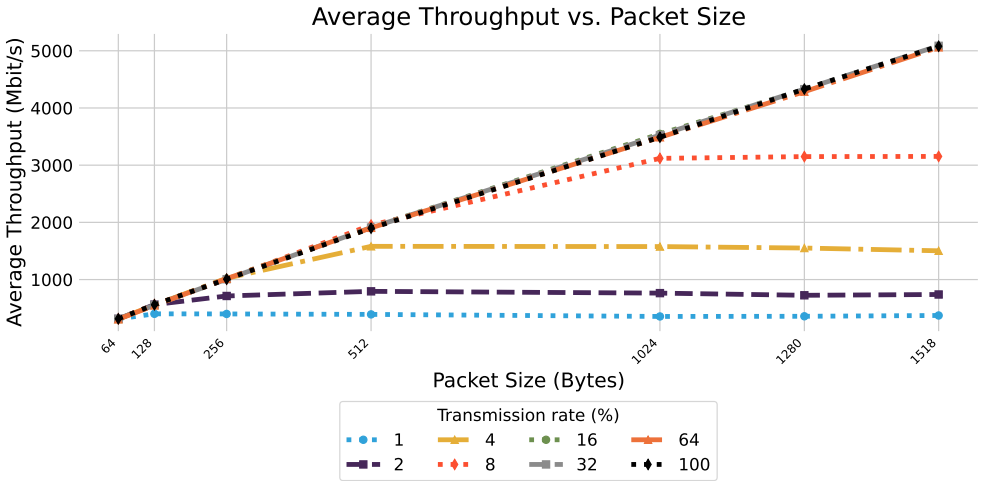

Figure A.13: Throughput of different rates and different packet sizes for io_uring with ring buffer size set to 4096 Bytes

A.4 netdev_budget

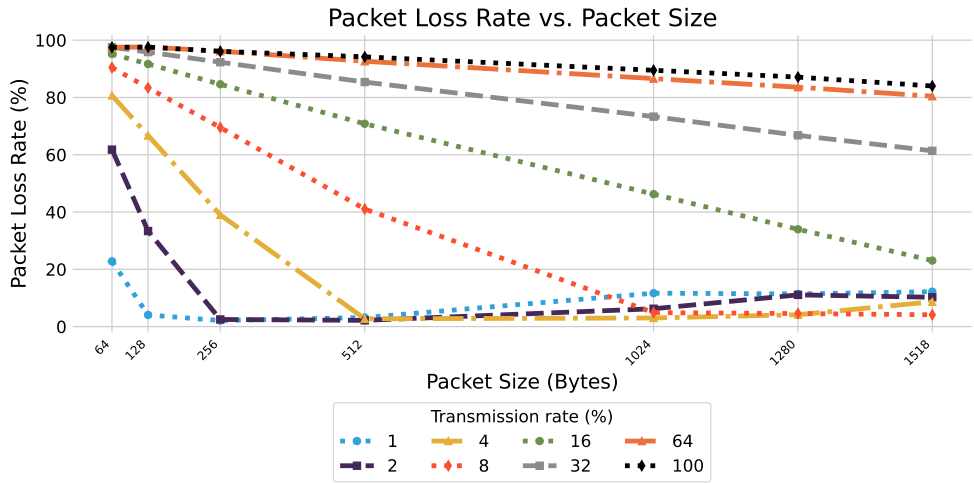

Figure A.14: Packet loss of different rates and different packet sizes for io_uring with increased netdev_budget

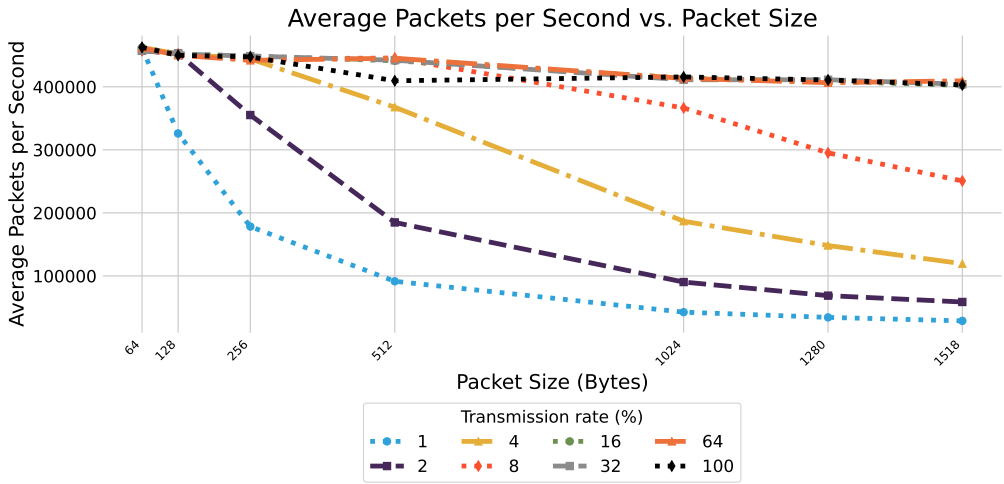

Figure A.15: Packets rate of different rates and different packet sizes for io_uring with increased netdev_budget

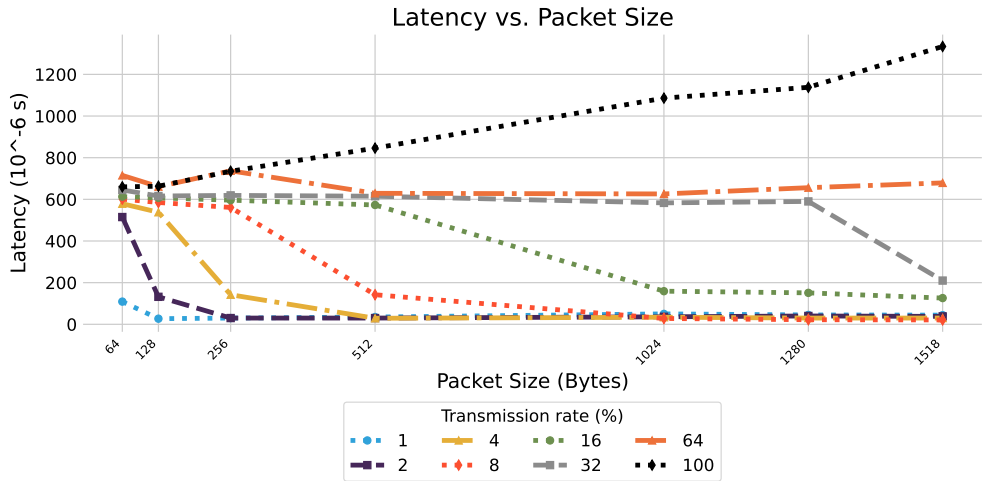

Figure A.16: Latency rate of different rates and different packet sizes for io_uring with increased netdev_budget

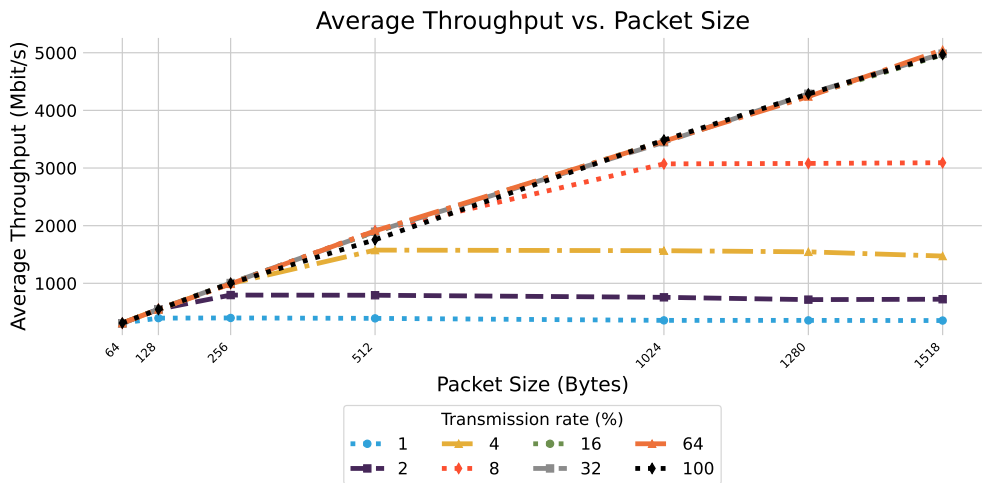

Figure A.17: Throughput of different rates and different packet sizes for io_uring with increased netdev_budget

A.5 Socket receive queue memory

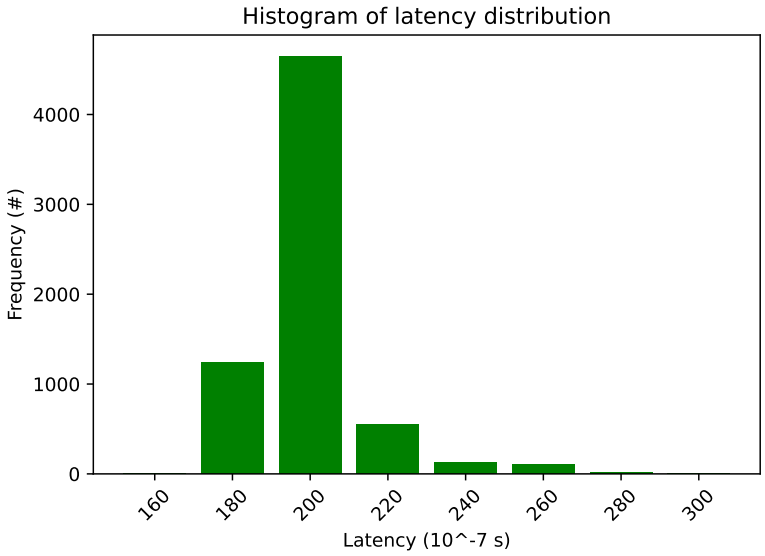

Figure A.18: Latency histogram for sending rate 8% and seize 1518 Bytes for io_uring with socket receive queue memory set to 425984 Bytes

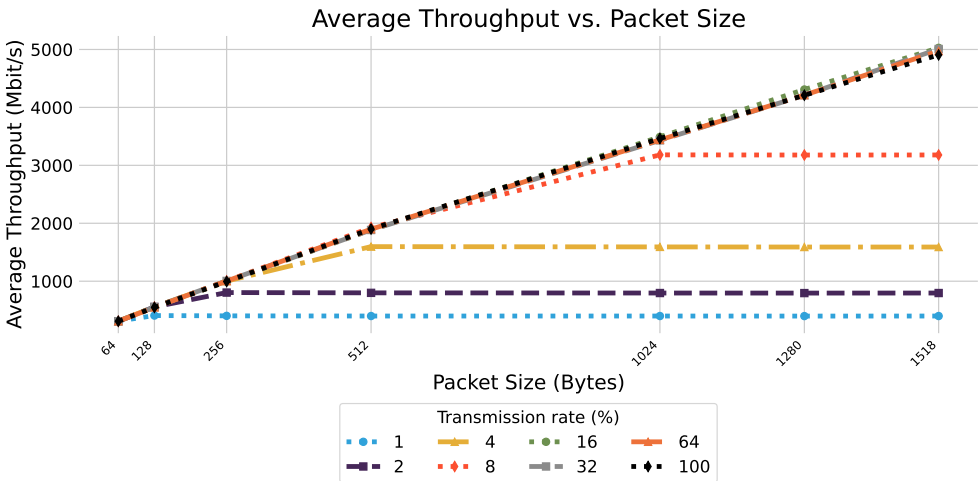

Figure A.19: Throughput of different rates and different packet sizes for io_uring with socket receive queue memory set to 425984 Bytes

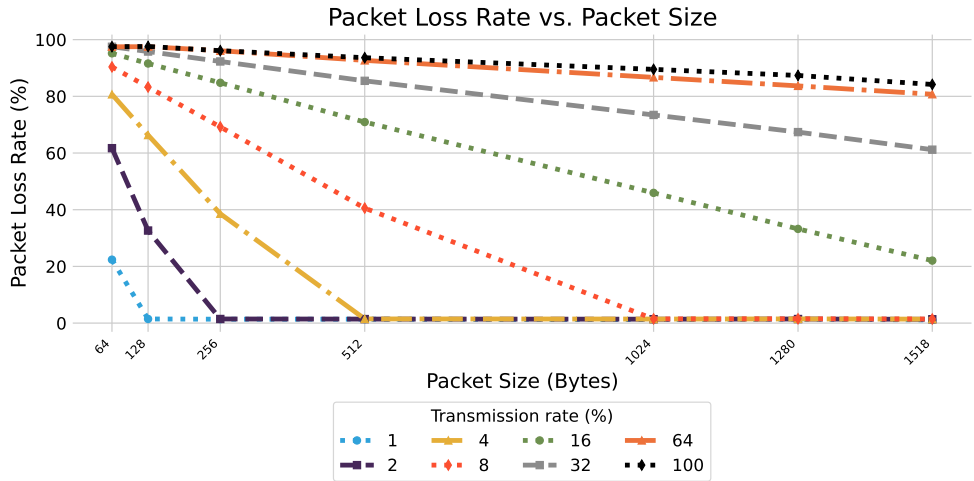

Figure A.20: Packet loss rate of different rates and different packet sizes for io_uring with socket receive queue memory set to 425984 Bytes

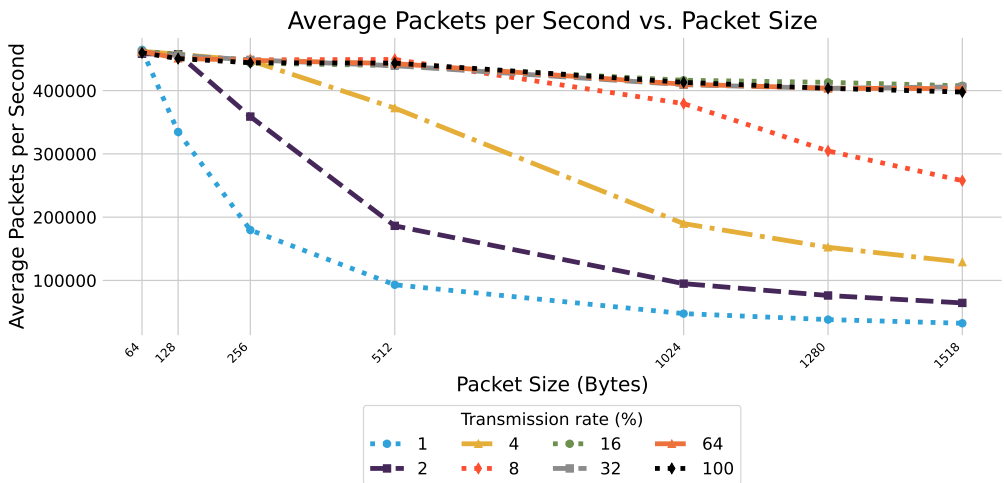

Figure A.21: Packet rate of different rates and different packet sizes for io_uring with socket receive queue memory set to 425984 Bytes

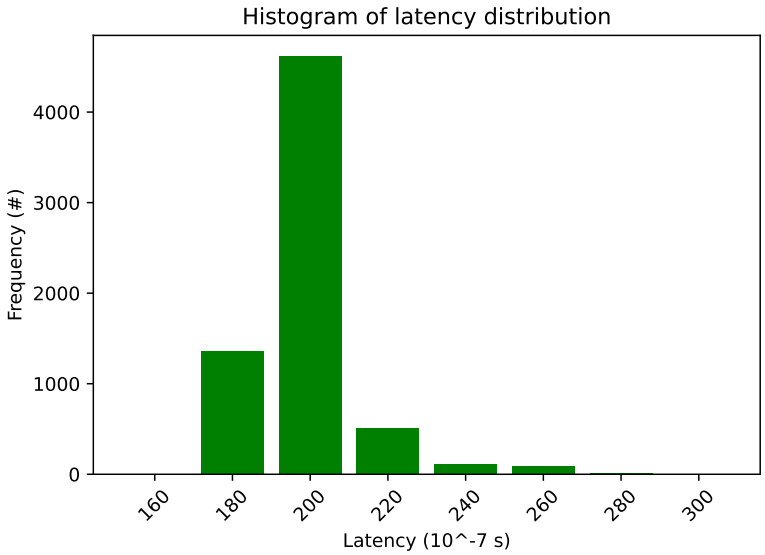

Figure A.22: Latency histogram for sending rate 8% and seize 1518 Bytes for io_uring with socket receive queue memory set to 1703936 Bytes

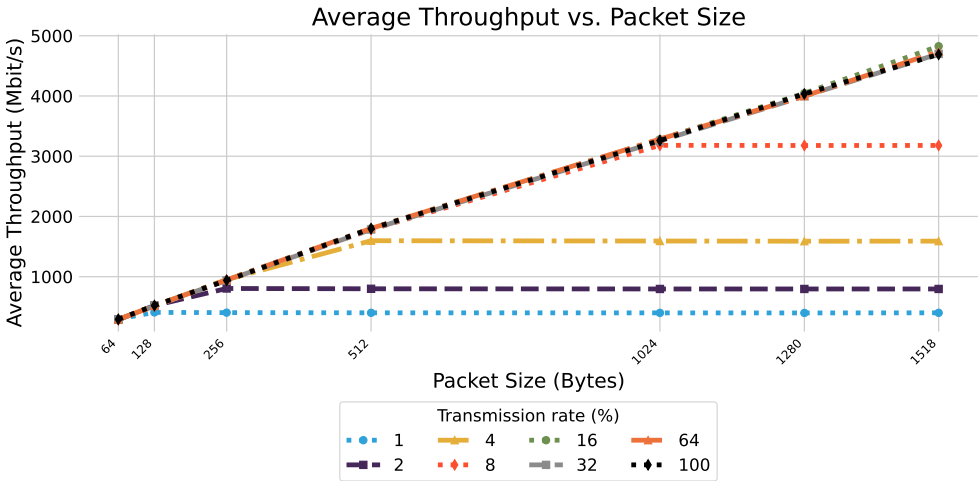

Figure A.23: Throughput of different rates and different packet sizes for io_uring with socket receive queue memory set to 1703936 Bytes

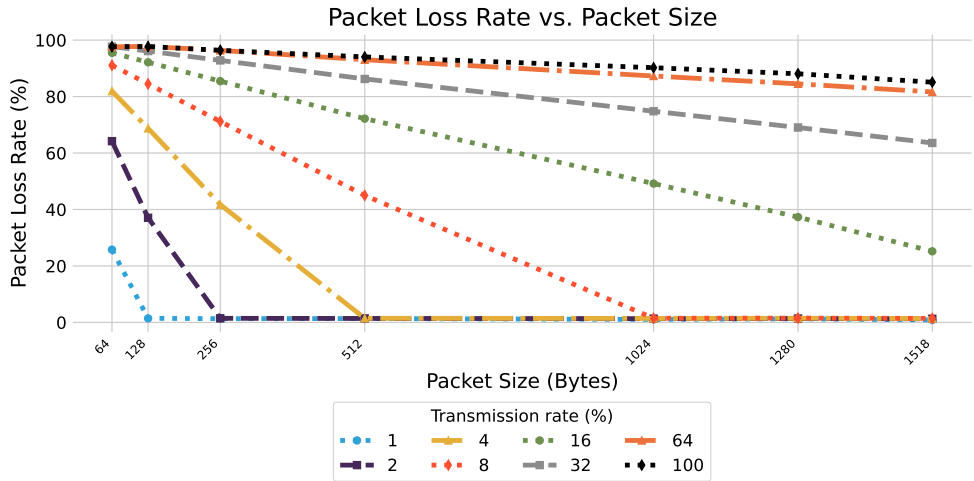

Figure A.24: Packet loss rate of different rates and different packet sizes for io_uring with socket receive queue memory set to 1703936 Bytes

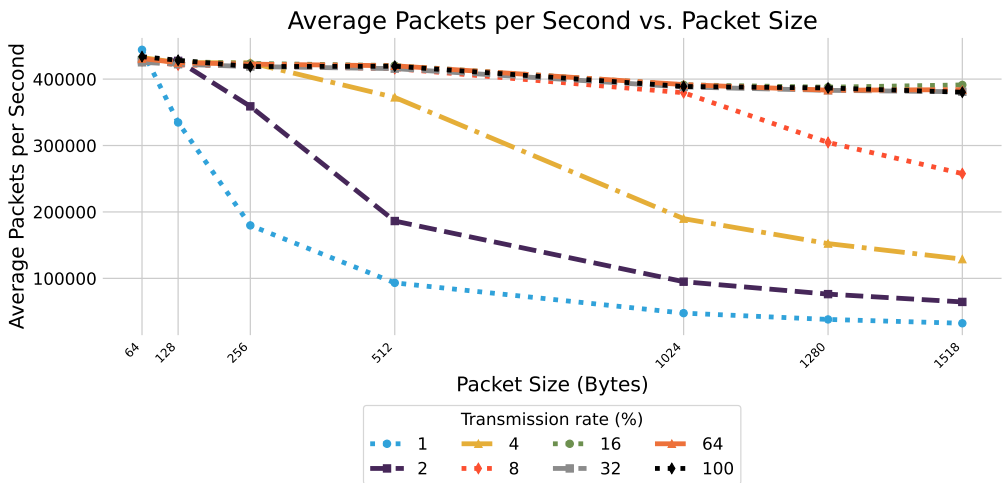

Figure A.25: Packet rate of different rates and different packet sizes for io_uring with socket receive queue memory set to 1703936 Bytes

A.6 Socket transmit queue memory

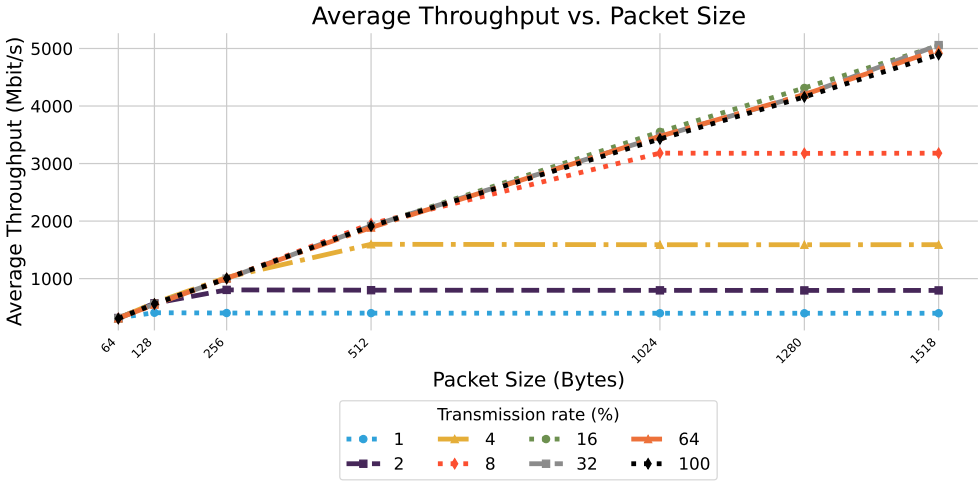

Figure A.26: Throughput of different rates and different packet sizes for io_uring with socket transmit queue memory set to 425984 Bytes

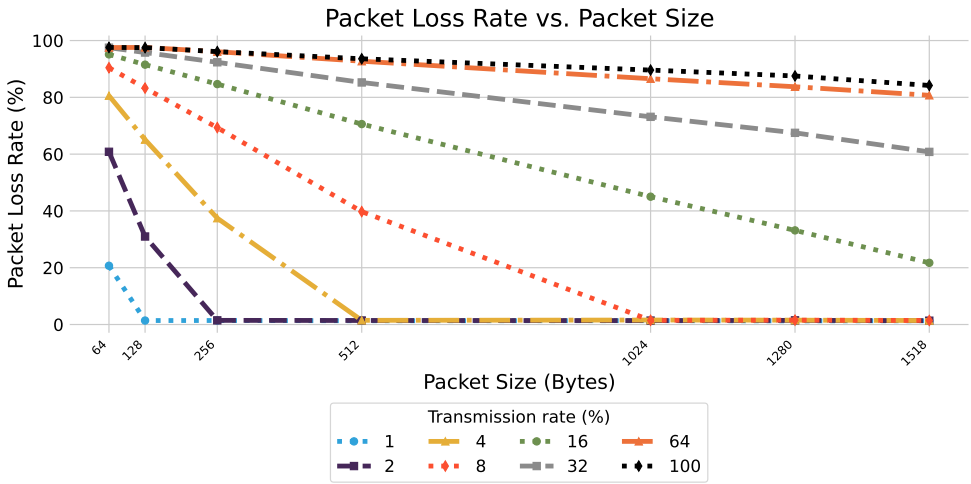

Figure A.27: Packet loss rate of different rates and different packet sizes for io_uring with socket transmit queue memory set to 425984 Bytes

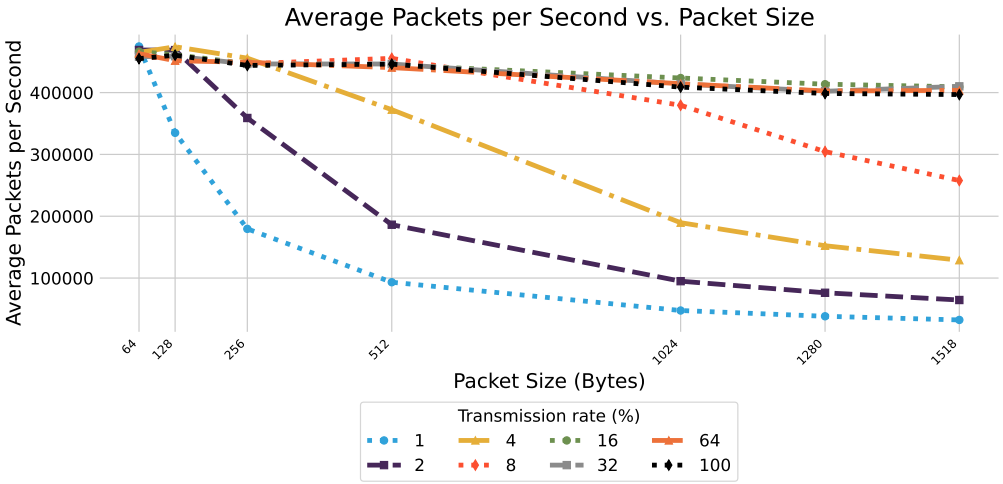

Figure A.28: Packet rate of different rates and different packet sizes for io_uring with socket transmit queue memory set to 425984 Bytes

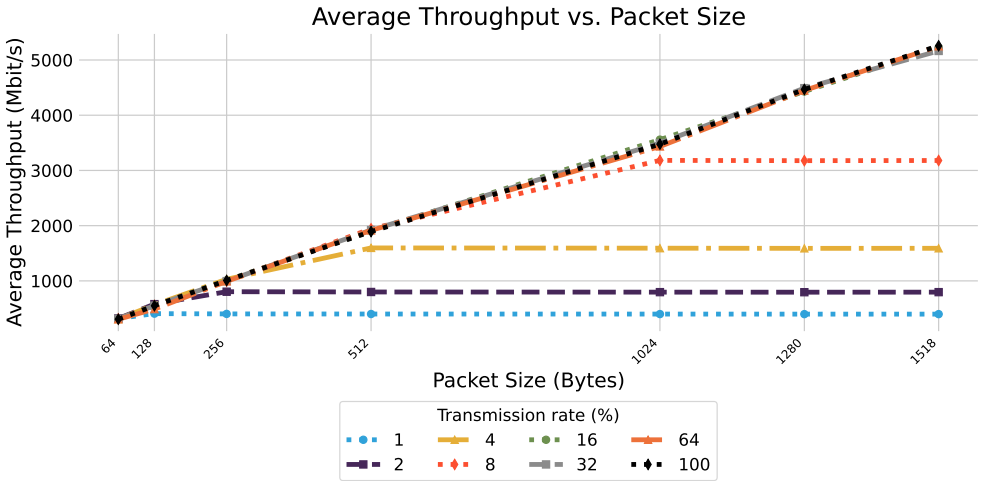

Figure A.29: Throughput of different rates and different packet sizes for io_uring with socket transmit queue memory set to 1703936 Bytes

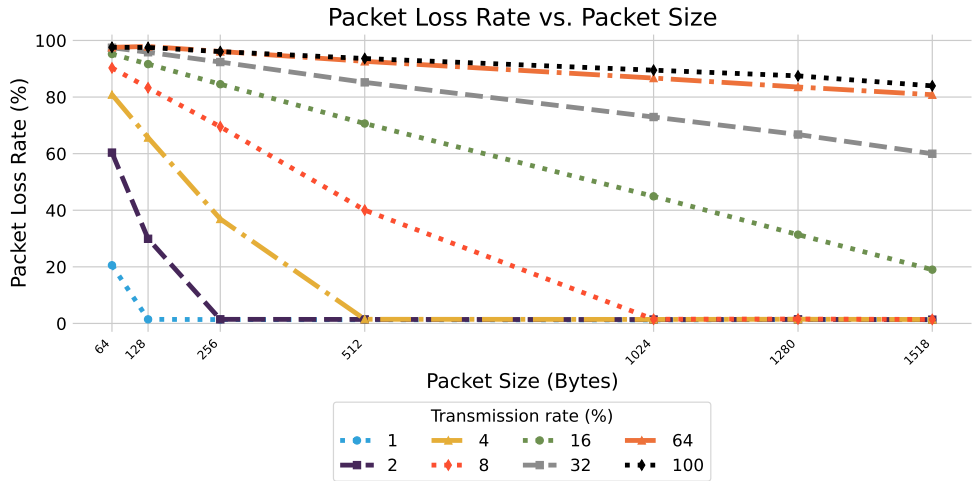

Figure A.30: Packet loss rate of different rates and different packet sizes for io_uring with socket transmit queue memory set to 1703936 Bytes

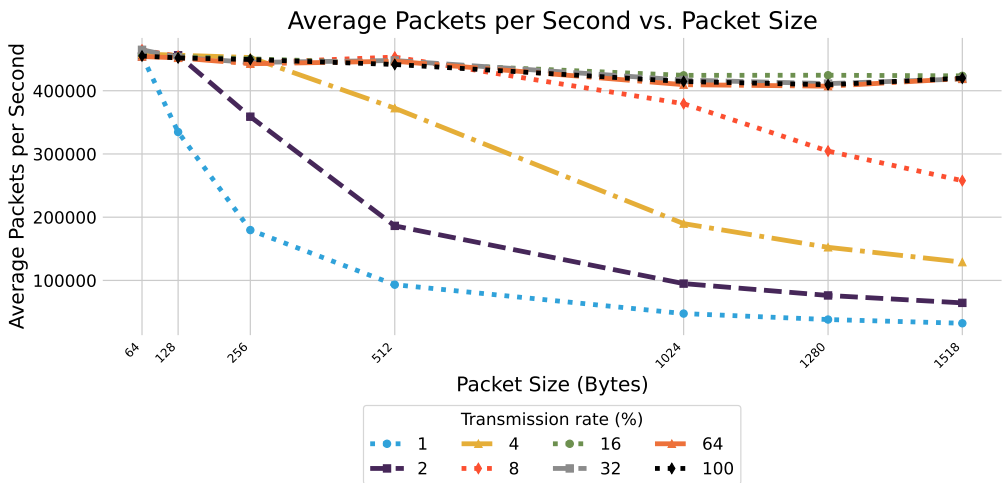

Figure A.31: Packet per second rate of different rates and different packet sizes for io_uring with socket transmit queue memory set to 1703936 Bytes

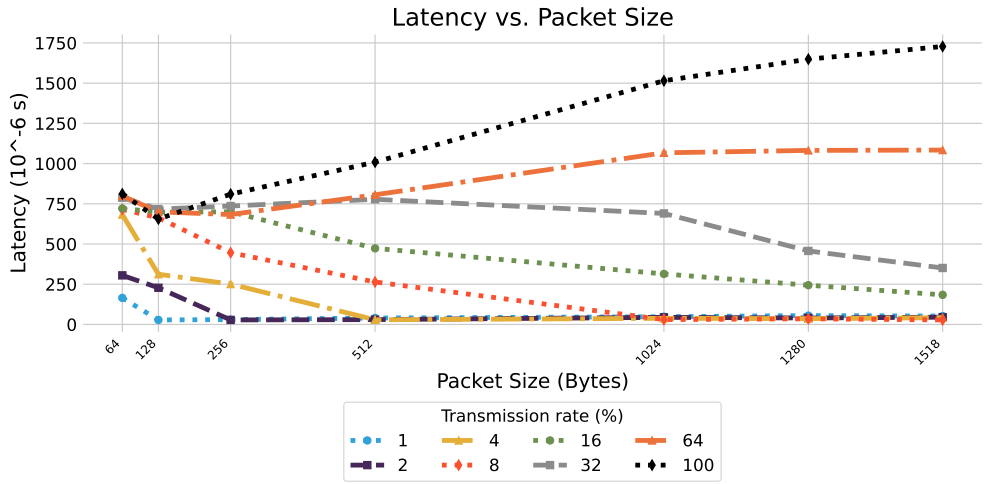

Figure A.32: Latency rate of different rates and different packet sizes for io_uring with socket transmit queue memory set to 1703936 Bytes

A.7 Generic Receive Offloading (GRO)

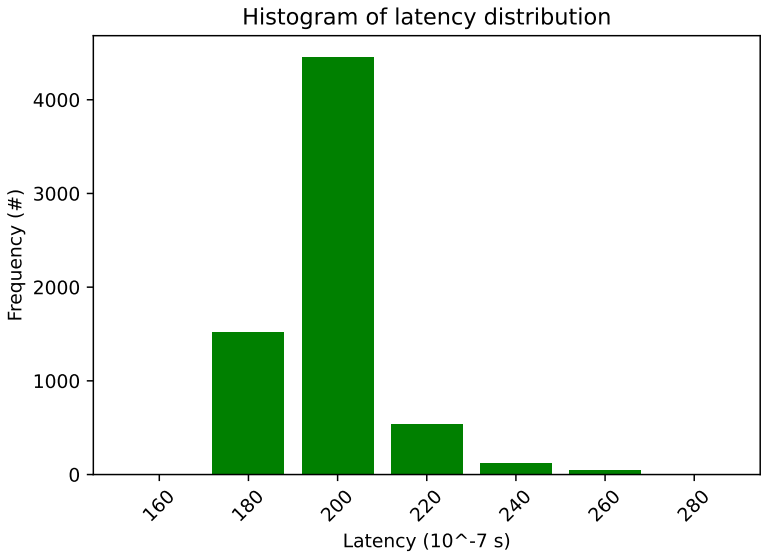

Figure A.33: Latency histogram of different rates and different packet sizes for io_uring with GRO turned off

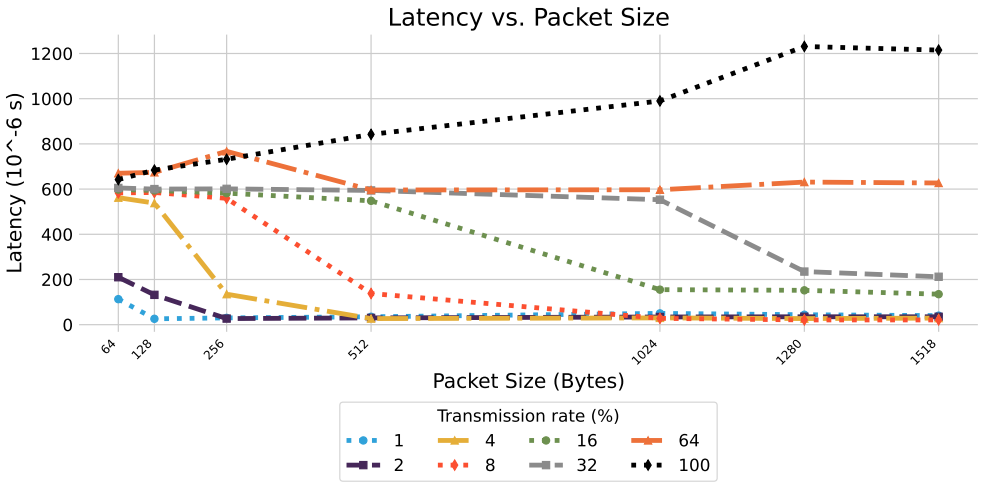

Figure A.34: Latency rate of different rates and different packet sizes for io_uring with GRO turned off

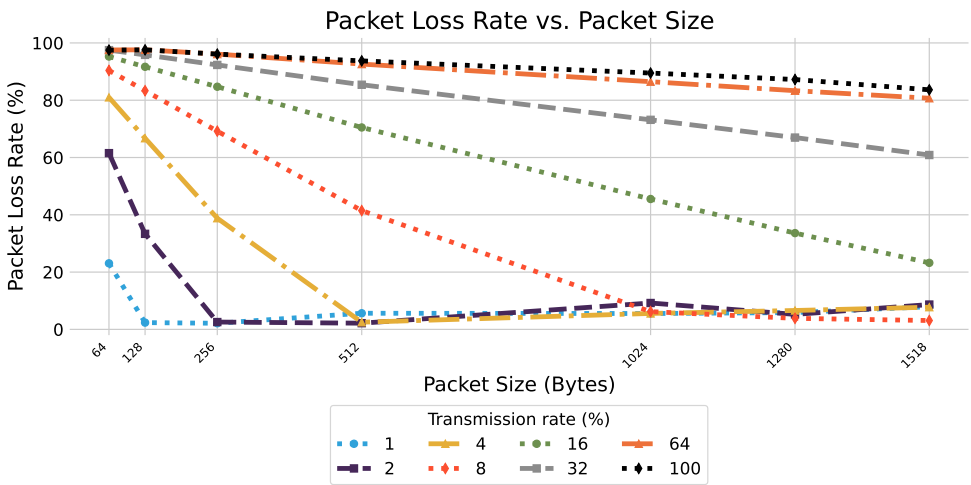

Figure A.35: Packet loss rate of different rates and different packet sizes for io_uring with GRO turned off

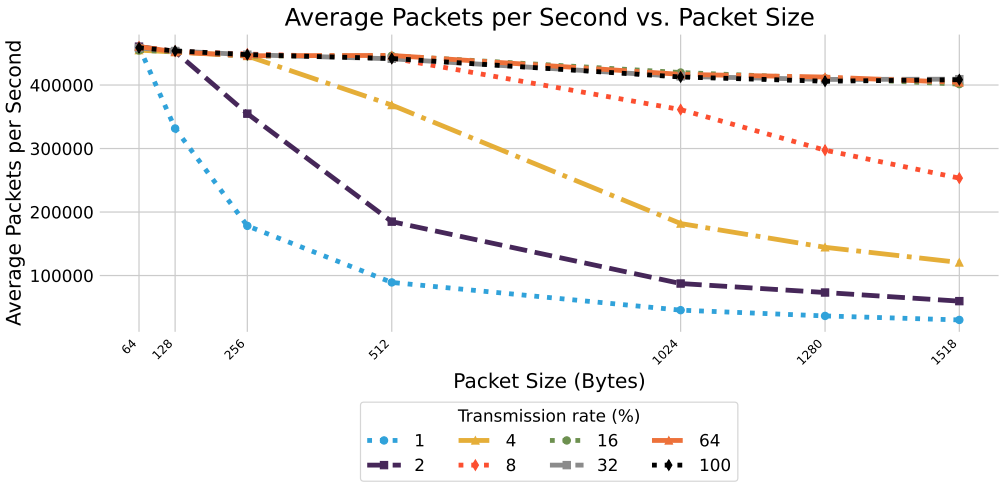

Figure A.36: Packet rate of different rates and different packet sizes for io_uring with GRO turned off

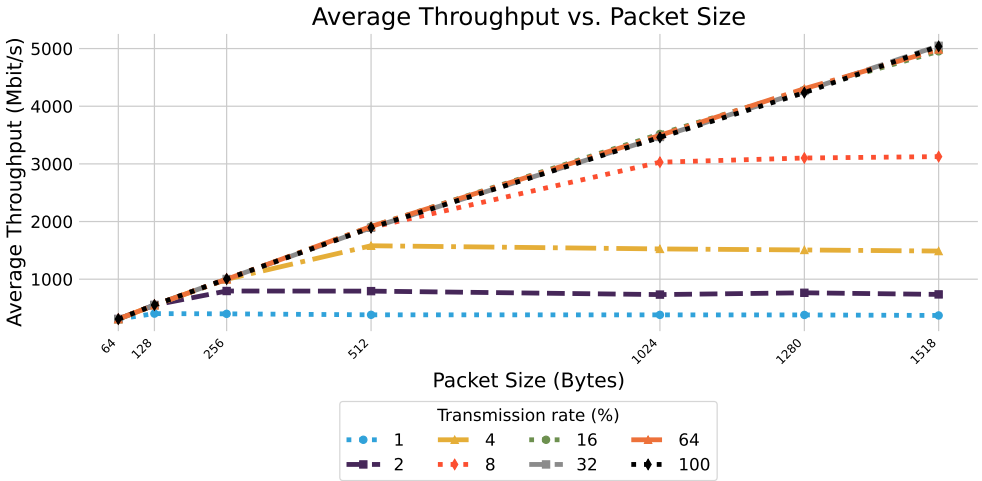

Figure A.37: Throughput of different rates and different packet sizes for io_uring with GRO turned off

A.8 netdev_max_backlog

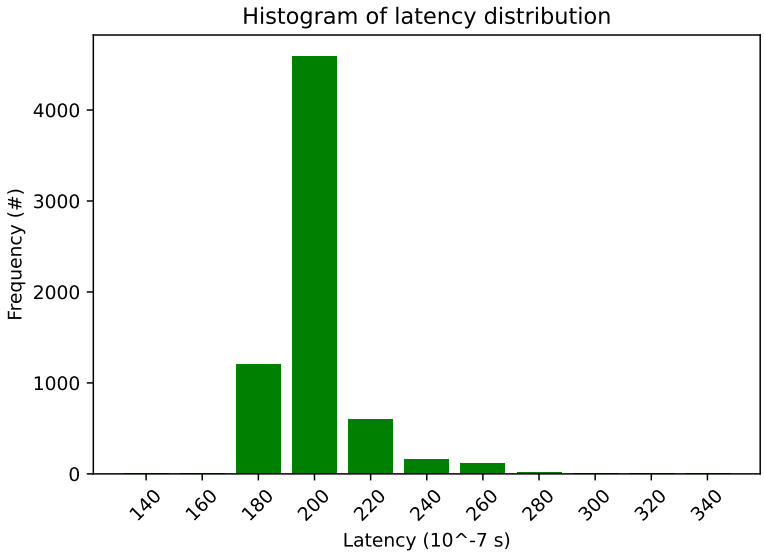

Figure A.38: Latency histogram of different rates and different packet sizes for io_uring with increased netdev_max_backlog

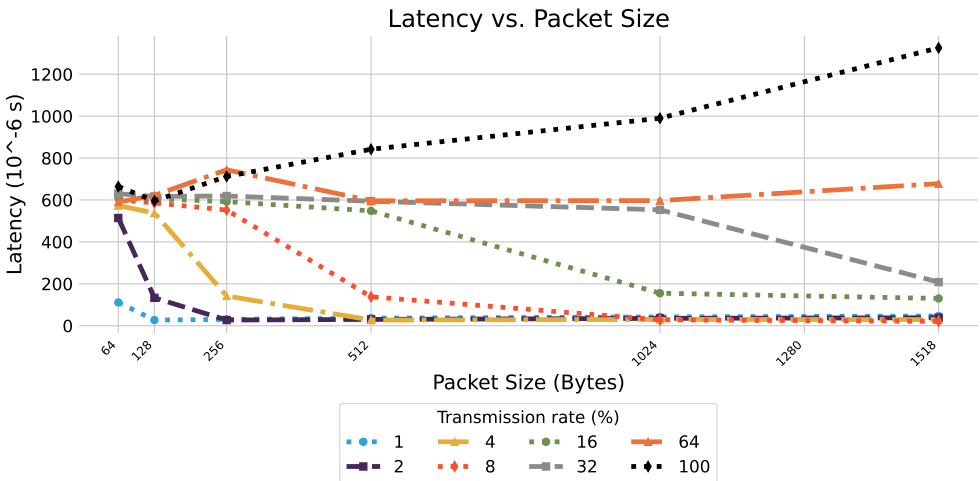

Figure A.39: Latency rates of different rates and different packet sizes for io_uring with increased netdev_max_backlog

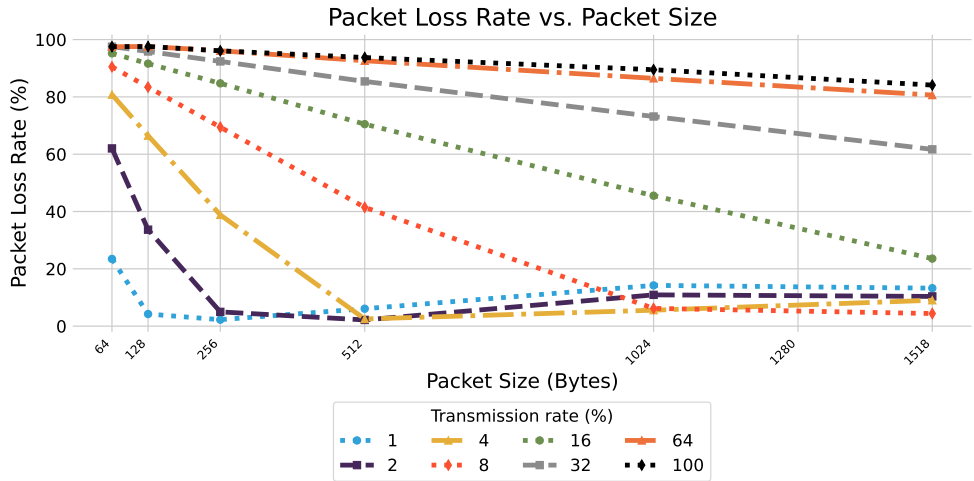

Figure A.40: Packet loss rates of different rates and different packet sizes for io_uring with increased netdev_max_backlog

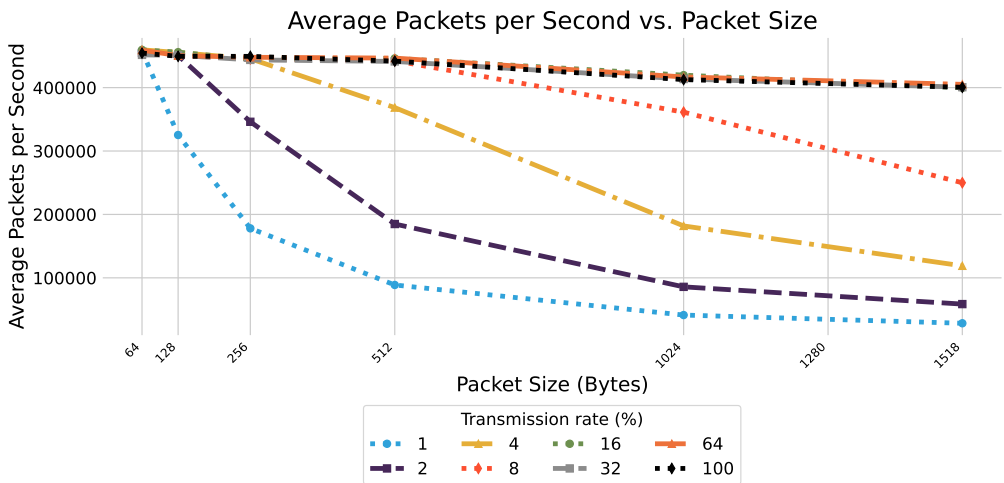

Figure A.41: Packets per second rates of different rates and different packet sizes for io_uring with increased netdev_max_backlog

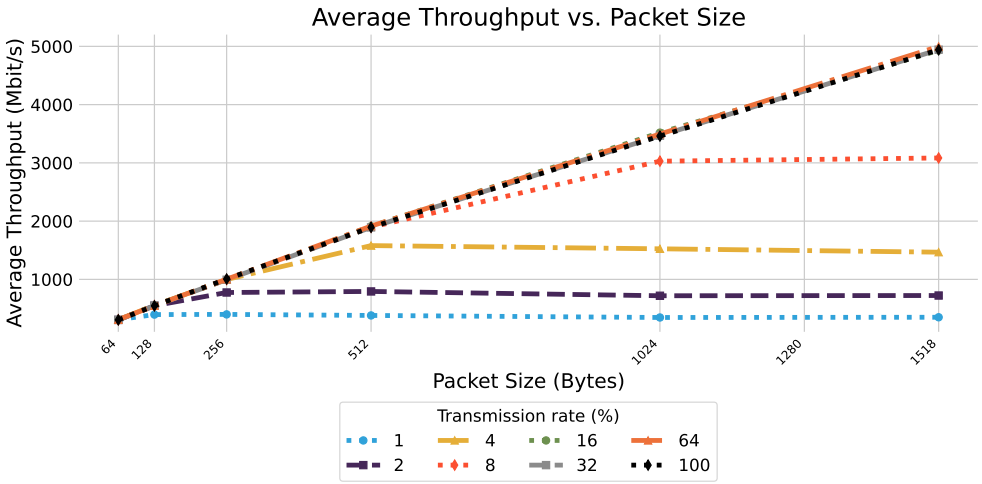

Figure A.42: Throughputs of different rates and different packet sizes for io_uring with increased netdev_max_backlog

B Code

This chapter shows the code used for testing DPDK, io_uring, and the Linux network stack. The changes made to the source code to enable the gathering of packets' latency into a hash table are also shown.

B.1 Simulating network traffic with Pktgen

Here the scripts used to simulate data traffic and calculate metrics using Pktgen are shown.

B.1.1 network_metrics.lua

```
package.path = package.path ..";?.lua;test/?.lua;app/?.lua;"

require "Pktgen"

local port = "0";
local sleep = 10;
local setup_time = 30;

local inter_frame_gap = 12;
local start_frame_delimiter = 1;
local pkt_preamble_size = 7;
local rte_ether_crc_len = 4;
local pkt_overhead_size = inter_frame_gap + start_frame_delimiter
+ pkt_preamble_size + rte_ether_crc_len;

-- IP address and port for application
local dst_ip = "<destination ip address>"
local dst_mac = "<destination mac address>";
local ip_port = 12345

-- Packet sizes
local pkt_sizes ={64,128,256,512,1024,1280,1518};
```

```
-- Packet rates
local pkt_rates = {1, 2, 4, 8, 16, 32, 64, 100};

-- File to save results to
local logfile_metrics = "metrics.txt";

local function round(num)
    return math.floor(num+0.5);
end

local function round_two_num(num)
    return math.floor((num * 1000)+0.5)/1000;
end

local function init()
    if show_status
    then
        pktgen.screen("on");
    end
    pktgen.page(port);
    pktgen.stop(port);
    pktgen.clr();
    pktgen.pause("[  INFO ] Starting script for collecting
    network metrics...\n",1000);
    print("[NOTICE ] See test log for results: "..logfile_metrics);
end

local function wait_for_connection()
    pktgen.set(port, "rate", 1);
    pktgen.start(port);
    pktgen.sleep(setup_time);
    pktgen.stop(port);
    pktgen.clr();
end

function setup_traffic()
    -- Generate hash table for storing latency values
    pktgen.create_hash_table();

    -- Set destination mac address
        pktgen.set_mac(port, "dst", dst_mac);

    -- Set destination ip address and port
    pktgen.set_ipaddr(port, "dst", dst_ip);
    pktgen.set(port, "dport", ip_port);

    -- Set internet protocol
    pktgen.set_type(port, "ipv4");

    -- Set transport protocol
    pktgen.set_proto(port, "udp");
```

```
    -- Switch to page latency
    pktgen.page("latency");

    -- Enable processing of ARP packets
    pktgen.process(port, "on");

    print("[NOTICE ] Traffic setup done...");
end

function gather_metrics(pkt_size, pkt_rate, file)
    -- Set the packets size
    pktgen.set(port, "size", pkt_size);
    pktgen.set(port, "rate", pkt_rate);

    -- Enable latency measurement on port 0 and 1
    pktgen.latency(0, "enable");
    pktgen.latency(1, "enable");

    -- Start pktgen
    pktgen.start(port);
    print("[NOTICE ] Start sending packets of size: "..pkt_size);

    pktgen.sleep(sleep);

    pktgen.stop(port);

    pktgen.latency(0, "disable");
    pktgen.latency(1, "disable");

    --Wait for all packets to to be received
    pktgen.sleep(sleep);

    --Payload size of 64-byte packets is 60 bytes
    if pkt_size == 64 then
        pkt_size = 60;
    end

    --Calculate throughput (bps)
    bits_per_frame = (pkt_size + pkt_overhead_size) * 8;
    Mbits_per_second = round((((pktgen.portStats(port, "port")[0]
    ["ipackets"]*bits_per_frame))/1000000)/sleep);

    if pkt_size == 60 then
        pkt_size = 64;
    end

    --Calculate packets per second (pps)
    avg_pps = round(pktgen.portStats(port, "port")[0]["ipackets"]/10);

    --Calculate packet loss rounded to two decimals
    pkt_loss = round_two_num(((pktgen.portStats(port, "port")[0]
    ["opackets"]-pktgen.portStats(port, "port")[0]["ipackets"])/
    pktgen.portStats(port, "port")[0]["opackets"])*100);
```

```
    --Get latency values
    avg_latency = round(pktgen.pktStats(port)[0]['latency']["avg_us"]);
    max_us = round(pktgen.pktStats(port)[0]['latency']["max_us"]);
    min_us = round(pktgen.pktStats(port)[0]['latency']["min_us"]);

    print("[NOTICE ] Results written to file: ".. logfile_metrics);

    local json_metrics = '{"pkt_rate": ' .. pkt_rate .. ',
    "pkt_size": ' .. pkt_size .. ',
    "avg_throughput": '.. Mbits_per_second .. ',
    "avg_pps": ' .. avg_pps .. ',
    "pkt_loss": ' .. pkt_loss .. ',
    "avg_latency": ' .. avg_latency .. ',
    "max_latency": ' .. max_us .. ',
    "min_latency": ' .. min_us .. '}'

    -- Write the JSON string to a new line in the file
    file:write(json_metrics .. "\n")

    pktgen.clr();
end


function main()
    init();

    setup_traffic();

    wait_for_connection();

    --Clear file for previous content
    io.open(logfile_metrics,"w"):close();

    --Open file for writing
    file = io.open(logfile_metrics, "w");

    for i=1, 7, 1
    do
        for j=1, 8,1
        do
            gather_metrics(pkt_sizes[i], pkt_rates[j] , file);
            pktgen.sleep(5);
        end

    end

    file:close();

    print("Packet metrics capture finished successfully.");

end
```

```
main();
```

B.1.2 latency_histogram.lua

```
package.path = package.path ..";?.lua;test/?.lua;app/?.lua;"

require "Pktgen"

local port = "0";
local rate      = "8";
local test_time = 10;
local setup_time = 30;

-- IP address and port for application
local dst_ip = "<destination ip address>"
local dst_mac = "<destination mac address>";
local ip_port = 12345

-- File to save results to
local logfile = "latency_hash_table.txt" ;

local function init()
    if show_status
    then
        pktgen.screen("on");
    end
    pktgen.page("0");
    pktgen.stop(port);
    pktgen.clr();
    pktgen.pause("[  INFO ] Starting script for testing
    latency...\n",1000);
end

local function wait_for_connection()
    pktgen.start(port);
    pktgen.sleep(setup_time);
    pktgen.stop(port);
    pktgen.clr();
end

local function setup_traffic()
    -- Generate hash table for storing latency values
    pktgen.create_hash_table();

    -- Set destination mac address
    pktgen.set_mac(port, "dst", dst_mac);

    -- Set destination ip address and port
    pktgen.set_ipaddr(port, "dst", dst_ip);
    pktgen.set(port, "dport", ip_port);

    -- Set internet protocol
```

```
    pktgen.set_type(port, "ipv4");

    -- Set size of packets
    pktgen.set(port, "size", 1518);

    -- Set transport protocol
    pktgen.set_proto(port, "udp");

    -- Switch to page latency
    pktgen.page("latency");

    -- Enable latency measurement on port 0 and 1
    pktgen.latency(0, "enable");
    pktgen.latency(1, "enable");

    -- Enable processing of ARP packets
    pktgen.process(port, "on");

    --- Set rate
    pktgen.set(port, "rate", rate);

    print("[NOTICE ] Traffic setup done...");
end


function main()
    init();

    setup_traffic();

    wait_for_connection();

    --Clear hash table
    pktgen.create_hash_table();

    pktgen.start(port);

    pktgen.sleep(test_time);

    pktgen.stop(port);

    -- Disable latency measurement on port 0 and 1
    pktgen.latency(0, "disable");
    pktgen.latency(1, "disable");

     --Save hash table to file
    pktgen.save_hash_table(logfile);

    print("Packet capture finished successfully.");

end

main();
```

B.1.3 bursty_traffic.lua

```
package.path = package.path ..";?.lua;test/?.lua;app/?.lua;"

require "Pktgen"

local port = "0";
local setup_time = 20;

local time_per_burst = 2
local time_between_bursts = 3

-- IP address and port for application
local dst_ip = "<destination ip address>"
local dst_mac = "<destination mac address>";
local ip_port = 12345

-- Packet size
local pkt_size = 1518;

-- File containing burst values
local bursty_traffic_data = "bursty_traffic_data.csv"

-- Arrival rates used in for histogram
local arrival_rates = {}

-- Average latency and time per burst used for latency graph
local average_latency_values = {}
local packet_loss_values = {}
local burst_times = {}
local burst_time = 0

local histogram_file= "latency_hash_table.txt" ;
local latency_file = "avg_latency_values.txt" ;
local packet_loss_file = "packet_loss.txt" ;

local function round(num)
    return math.floor(num+0.5);
end

local function init()
    if show_status
    then
        pktgen.screen("on");
    end
    pktgen.page(port);
    pktgen.stop(port);
    pktgen.clr();
    pktgen.pause("[  INFO ] Starting script for collecting
    network metrics...\n",1000);
    print("[NOTICE ] See test log for results: "..histogram_file);
end
```

```
local function wait_for_connection()
    pktgen.start(port);
    pktgen.sleep(setup_time);
    pktgen.stop(port);
    pktgen.clr();
end

local function read_burst_data()
    local file = io.open(bursty_traffic_data, "r")
    local file_contents = file:read("*all")

    local lines = {}
    for line in string.gmatch(file_contents, "[^\n]+") do
        table.insert(lines, line)
    end

    for rate in string.gmatch(lines[1], "%d+") do
        table.insert(arrival_rates, tonumber(rate))
    end
end

function setup_traffic()
    -- Generate hash table for storing latency values
    pktgen.create_hash_table();

    -- Set destination mac address
    pktgen.set_mac(port, "dst", dst_mac);

    -- Set destination ip address and port
    pktgen.set_ipaddr(port, "dst", dst_ip);
    pktgen.set(port, "dport", ip_port);

    -- Set internet protocol
    pktgen.set_type(port, "ipv4");

    -- Set size of packets
    pktgen.set(port, "size", pkt_size);

    -- Set transport protocol
    pktgen.set_proto(port, "udp");

    -- Enable processing of ARP packets
    pktgen.process(port, "on");

    -- Switch to page latency
    pktgen.page("latency");

    print("[NOTICE ] Traffic setup done...");
end


function save_avg_latency()
    -- Current average latency
```

```
    avg_latency =
    round(pktgen.pktStats(port)[0]['latency']["avg_us"]);
    print("avg latency: "..avg_latency.."\n");

    --Calculate packet loss rounded to two decimals
    pkt_loss =
    round(((pktgen.portStats(port, "port")[0]["opackets"]
    -pktgen.portStats(port, "port")[0]["ipackets"])
    /pktgen.portStats(port, "port")[0]["opackets"])*100);
    print("pkt loss: "..pkt_loss.."\n");

    -- Insert average latency for burst
    table.insert(average_latency_values, avg_latency)

    -- Insert packet loss for burst
    table.insert(packet_loss_values, pkt_loss)

    -- Insert current burst time
    table.insert(burst_times, burst_time)

    -- Clear Pktgen stats
    pktgen.clr();
end

function send_burst(rate)
    print("[NOTICE ] Sending burst during "..time_per_burst.."
    second(s) with transmission rate of: "..rate);

    -- Set transmission rate
    pktgen.set(port, "rate", rate);

    -- Enable latency measurement on port 0 and 1
    pktgen.latency(0, "enable");
    pktgen.latency(1, "enable");

    pktgen.start(port);

    pktgen.sleep(time_per_burst);

    pktgen.stop(port);

    -- Disable latency measurement on port 0 and 1
    pktgen.latency(0, "disable");
    pktgen.latency(1, "disable");

    save_avg_latency();

    burst_time = burst_time + time_per_burst;

    print("[NOTICE ] Waiting "..time_between_bursts.."
    second(s) before next burst");

    pktgen.sleep(time_between_bursts);
```

```
    burst_time = burst_time + time_between_bursts;
end

function write_tables_to_file(file_ref, table_name)
    for i, v in ipairs(table_name) do
        file_ref:write(tostring(v))
        if i ~= #table_name then
            file_ref:write(",")
        end
    end
        file_ref:write("\n")
end


function main()

    arrival_rates = {}; -- e.g. 4,2,4,4,78,77,6

    init();

    setup_traffic();

    wait_for_connection();

    io.open(histogram_file,"w"):close();

    --Open file for writing
    file = io.open(histogram_file, "w");

    for i = 1, #arrival_rates do
        send_burst(arrival_rates[i]);
    end

    --Save hash table to file
    pktgen.save_hash_table(histogram_file);

    file:close();

    file = io.open(latency_file,"w");
    write_tables_to_file(file, average_latency_values);
    write_tables_to_file(file, burst_times);

    file:close();

    file = io.open(packet_loss_file,"w");
    write_tables_to_file(file, packet_loss_values);
    write_tables_to_file(file, burst_times);

    file:close();

    print("Packet metrics capture finished successfully.");
```

```
end

main();
```

B.2 Changes to source code

In this section, the changes to the Pktgen source code are shown. The files hash_table.c and hash_table.h were added to the source code to allow the gathering of latency values into a hash table, and the changes to the already existing source code are shown in pktgen.c, lpktgenlib.c, pktgen-latency.h and meson.build.

B.2.1 hash_table.c

```
#include "hash_table.h"
#include <stdio.h>
#include <stdlib.h>

interval_t* hash_table;

#define groups 100
#define interval 20

void create_hash_table() {
    hash_table = (interval_t*) calloc(groups, sizeof(interval_t));
    for (int i = 0; i < groups; i++) {
        hash_table[i].latency = i * interval;
        hash_table[i].nr_pkts = 0;
    }
}

void add_to_hash_table(int latency_micro_sec) {
    if (latency_micro_sec < 0) {
        return;
    }
    hash_table[((latency_micro_sec/interval)*interval)
    /interval].nr_pkts++;
}

void write_hash_table_to_file(const char *filename) {
    FILE* fp = fopen(filename, "w");
    if (!fp) {
        fprintf(stderr, "Error: Could not open file %s for
        writing.\n", filename);
        return;
    }

    fprintf(fp, "{\n");
    for (int i = 0; i < groups; i++) {
        fprintf(fp, "  \"%d\": %d", i * interval,
        hash_table[i].nr_pkts);
        if (i < groups-1) {
```

```
            fprintf(fp, ",");
        }
        fprintf(fp, "\n");
    }
    fprintf(fp, "}\n");

    fclose(fp);
}
```

B.2.2 hash_table.h

```
#ifndef HASH_TABLE_H
#define HASH_TABLE_H

#include <limits.h>

typedef struct {
    int latency;
    int nr_pkts;
} interval_t;

extern interval_t* hash_table;

void create_hash_table();

void add_to_hash_table(int x);

void write_hash_table_to_file(const char* filename);

#endif /* HASH_TABLE_H */
```

B.2.3 pktgen.c

```
35a36
> #include "hash_table.h"
39a41
>
321a324
>     uint64_t current_time = pktgen_get_time();
328c331
<
---
>
333c336,343
<   cycles        = (pktgen_get_time() - tstamp->timestamp);
---
>   cycles        = (current_time - tstamp->timestamp);
>
>   // 10 for units in 10^-7 and 100 for units in milliseconds
>   int factor = 10;
>   uint64_t latency_res = ((cycles * Billion) / rte_get_tsc_hz())
```

```
    / factor;
>   if(latency_res > 0) {
>   add_to_hash_table(latency_res);
>   }
```

B.2.4 lpktgenlib.c

```
34a35
> #include "hash_table.h"
849a851,865
> static int
> pktgen_create_hash_table()
> {
>     create_hash_table();
>     return 0;
> }
>
> static int
> pktgen_save_hash_table(lua_State *L)
> {
>     write_hash_table_to_file(lua_tostring(L,1));
>     return 0;
> }
>
>
3850a3867,3868
>     {"save_hash_table", pktgen_save_hash_table},
> /* Save hash table in JSON format to file*/
>     {"create_hash_table", pktgen_create_hash_table},
> /* Create hash table */
```

B.2.5 pktgen-latency.h

```
19c19
< #define DEFAULT_LATENCY_RATE
< (10)    /**< milliseconds (min value 1)*/
---
> #define DEFAULT_LATENCY_RATE
> (1)    /**< milliseconds (min value 1)*/
```

B.2.6 meson.build

```
8c8
<       'pktgen-latency.c', 'pktgen-rate.c'
---
>       'pktgen-latency.c', 'pktgen-rate.c', 'hash_table.c'
```

B.3 Linux network stack application (udp_fwd_socket.c)

The UDP socket that was used for forwarding packets that used the Linux network stack is shown in this section.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>

#define PORT 12345
#define IP_ADDR "<ip address>"

int main() {
    int sockfd;
    struct sockaddr_in listen_adr, send_adr;
    char buffer[2048];

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("socket failed to create");
        exit(EXIT_FAILURE);
    }

    memset(&listen_adr, 0, sizeof(listen_adr));
    memset(&send_adr, 0, sizeof(send_adr));

    listen_adr.sin_family = AF_INET;
    listen_adr.sin_addr.s_addr = inet_addr(IP_ADDR);
    listen_adr.sin_port = htons(PORT);

    if (bind(sockfd, (const struct sockaddr *)&listen_adr,
    sizeof(listen_adr)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }

    socklen_t len = sizeof(&send_adr);
    int n;

    struct iovec send_iovec[1];
    struct msghdr send_msg;
    memset(&send_msg, 0, sizeof(send_msg));

    while (1) {
        n = recvfrom(sockfd, buffer, sizeof(buffer), 0,
        (struct sockaddr *)&send_adr, &len);
        send_iovec[0].iov_base = buffer;
        send_iovec[0].iov_len = n;
        send_msg.msg_name = &send_adr;
        send_msg.msg_namelen = sizeof(send_adr);
        send_msg.msg_iov = send_iovec;
        send_msg.msg_iovlen = 1;
        send_msg.msg_flags = 0;
        send_msg.msg_controllen = 0;
        send_msg.msg_control = NULL;
```

```
        sendmsg(sockfd, &send_msg, 0);
    }

    close(sockfd);
    return 0;
}
```

B.4 Poisson distribution (switched_poisson_process.py)

The script for generating bursty traffic data transmission values using a switched Poisson process is shown in this section.

```
import csv
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

time_limit = 60
time_per_burst = 1
time_between_bursts = 2
num_iterations = 20
seed_num = 1444
arrival_rates_dpdk = []
arrival_rates_kernel_based =[]
current_dpdk_state = 1
current_kernel_based_state = 1
t = 0

bursty_traffic_data = "bursty_traffic_data.csv" ;

dpdk_state_1 = {"arrival_rate": 6, "transition_rate": 0.2}
dpdk_state_2 = {"arrival_rate": 77, "transition_rate": 0.8}
kernel_bas_state_1 = {"arrival_rate": 2, "transition_rate": 0.2}
kernel_bas_state_2 = {"arrival_rate": 19, "transition_rate": 0.8}

np.random.seed(seed_num)

dpdk_poisson_1 =
list(np.random.poisson(dpdk_state_1["arrival_rate"],
num_iterations))
dpdk_poisson_2 =
list(np.random.poisson(dpdk_state_2["arrival_rate"],
num_iterations))
kernel_based_poisson_1 =
list(np.random.poisson(kernel_bas_state_1["arrival_rate"],
num_iterations))
kernel_based_poisson_2 =
list(np.random.poisson(kernel_bas_state_2["arrival_rate"],
num_iterations))

while t < time_limit:
    if current_dpdk_state == 1:
        arrival_rate_dpdk = dpdk_poisson_1[0]
```

```
        next_dpdk_state = np.random.choice([1, 2],
        p=[1 - dpdk_state_1["transition_rate"],
        dpdk_state_1["transition_rate"]])
        dpdk_poisson_1.pop(0)
    else:
        arrival_rate_dpdk = dpdk_poisson_2[0]
        next_dpdk_state = np.random.choice([1, 2],
        p=[dpdk_state_2["transition_rate"],
        1 - dpdk_state_2["transition_rate"]])
        dpdk_poisson_2.pop(0)


    if current_kernel_based_state == 1:
        arrival_rate_kernel_based = kernel_based_poisson_1[0]
        next_kernel_based_state =
        np.random.choice([1, 2],
        p=[1 - kernel_bas_state_1["transition_rate"],
        kernel_bas_state_1["transition_rate"]])
        kernel_based_poisson_1.pop(0)
    else:
        arrival_rate_kernel_based = kernel_based_poisson_2[0]
        next_kernel_based_state =
        np.random.choice([1, 2],
        p=[kernel_bas_state_2["transition_rate"],
        1 - kernel_bas_state_2["transition_rate"]])
        kernel_based_poisson_2.pop(0)

    current_dpdk_state = next_dpdk_state
    current_kernel_based_state = next_kernel_based_state
    t += time_per_burst + time_between_bursts
    arrival_rates_dpdk.append(arrival_rate_dpdk)
    arrival_rates_kernel_based.append(arrival_rate_kernel_based)

with open(bursty_traffic_data, 'w', newline='') as file:
    writer = csv.writer(file)
    writer.writerow(arrival_rates_dpdk)
    writer.writerow(arrival_rates_kernel_based)
```

B.5 io_uring application (liburing_fwd.c)

This section contains the io_uring application. The application uses io_uring-udp.c as its foundation, but incorporates some changes to decrease overheads and increase performance.

```
/*Copyright (c) 2020 Jens Axboe */
/* SPDX-License-Identifier: MIT */

#include <stdio.h>
#include <unistd.h>
#include <errno.h>
#include <sys/mman.h>
#include <stdlib.h>
#include <string.h>
#include <netinet/udp.h>
```

```
#include <arpa/inet.h>

#include "liburing.h"

#define QD 128
#define BUF_SHIFT 24 /* 4k */
#define CQES (QD * 32)
#define BUFFERS CQES

struct sendmsg_ctx {
        struct msghdr msg;
        struct iovec iov;
};

struct ctx {
        struct io_uring ring;
        struct io_uring_buf_ring *buf_ring;
        unsigned char *buffer_base;
        struct msghdr msg;
        int buf_shift;
        int af;
        struct sendmsg_ctx send[BUFFERS];
        size_t buf_ring_size;
};

static size_t buffer_size(struct ctx *ctx)
{
        return 1U << ctx->buf_shift;
}

static unsigned char *get_buffer(struct ctx *ctx, int idx)
{
        return ctx->buffer_base + (idx << ctx->buf_shift);
}

static int setup_buffer_pool(struct ctx *ctx)
{
        int ret, i;
        void *mapped;
        struct io_uring_buf_reg reg = { .ring_addr = 0,
                                        .ring_entries = BUFFERS,
                                        .bgid = 0 };

        ctx->buf_ring_size = (sizeof(struct io_uring_buf) +
    buffer_size(ctx)) * BUFFERS;
        mapped = mmap(NULL, ctx->buf_ring_size, PROT_READ |
 PROT_WRITE,
                      0x20 | MAP_PRIVATE, 0, 0);
        ctx->buf_ring = (struct io_uring_buf_ring *)mapped;

        io_uring_buf_ring_init(ctx->buf_ring);

        reg = (struct io_uring_buf_reg) {
```

```
                .ring_addr = (unsigned long)ctx->buf_ring,
                .ring_entries = BUFFERS,
                .bgid = 0
        };
        ctx->buffer_base = (unsigned char *)ctx->buf_ring +
                        sizeof(struct io_uring_buf) * BUFFERS;

        ret = io_uring_register_buf_ring(&ctx->ring, &reg, 0);

        for (i = 0; i < BUFFERS; i++) {
                io_uring_buf_ring_add(ctx->buf_ring,
  get_buffer(ctx, i),
        buffer_size(ctx), i,

          io_uring_buf_ring_mask(BUFFERS), i);
        }
        io_uring_buf_ring_advance(ctx->buf_ring, BUFFERS);

        return 0;
}

static int setup_context(struct ctx *ctx)
{
        struct io_uring_params params;
        int ret;

        memset(&params, 0, sizeof(params));
        params.cq_entries = QD * 8;
        params.flags =
 IORING_SETUP_SUBMIT_ALL |
 IORING_SETUP_COOP_TASKRUN
        |IORING_SETUP_CQSIZE |
        IORING_SETUP_SINGLE_ISSUER |
         IORING_SETUP_DEFER_TASKRUN;
        ret = io_uring_queue_init_params(QD, &ctx->ring, &params);

        ret = setup_buffer_pool(ctx);

        memset(&ctx->msg, 0, sizeof(ctx->msg));
        ctx->msg.msg_namelen = sizeof(struct sockaddr_storage);
        ctx->msg.msg_controllen = 0;
        return ret;
}

static bool get_sqe(struct ctx *ctx, struct io_uring_sqe **sqe)
{
        *sqe = io_uring_get_sqe(&ctx->ring);

        if (!*sqe) {
                io_uring_submit(&ctx->ring);
                *sqe = io_uring_get_sqe(&ctx->ring);
        }
        if (!*sqe) {
```

```
                fprintf(stderr, "cannot get sqe\n");
                return true;
        }
        return false;
}

static int add_recv(struct ctx *ctx, int idx)
{
        struct io_uring_sqe *sqe;

        if (get_sqe(ctx, &sqe))
                return -1;

        io_uring_prep_recvmsg_multishot(sqe, idx, &ctx->msg, MSG_TRUNC);
        sqe->flags |= IOSQE_FIXED_FILE;

        sqe->flags |= IOSQE_BUFFER_SELECT;
        sqe->buf_group = 0;
        io_uring_sqe_set_data64(sqe, BUFFERS + 1);
        return 0;
}

static void recycle_buffer(struct ctx *ctx, int idx)
{
        io_uring_buf_ring_add(ctx->buf_ring, get_buffer(ctx, idx),
    buffer_size(ctx), idx,
        io_uring_buf_ring_mask(BUFFERS), 0);
        io_uring_buf_ring_advance(ctx->buf_ring, 1);
}


static int process_cqe_recv(struct ctx *ctx, struct io_uring_cqe *cqe,
                            int fdidx, struct sockaddr_in s_a)
{
        int ret, idx;
        struct io_uring_recvmsg_out *o;
        struct io_uring_sqe *sqe;

        if (!(cqe->flags & IORING_CQE_F_MORE)) {
                ret = add_recv(ctx, fdidx);
        }

        idx = cqe->flags >> 16;

        o = io_uring_recvmsg_validate(get_buffer(ctx, cqe->flags >> 16),
                                      cqe->res, &ctx->msg);

        if (get_sqe(ctx, &sqe)){
                return -1;
        }

        void *payload = io_uring_recvmsg_payload(o, &ctx->msg);
        size_t payload_len =
```

```
    io_uring_recvmsg_payload_length(o, cqe->res, &ctx->msg);
        io_uring_prep_send(sqe, fdidx, payload, payload_len, 0);
        io_uring_prep_send_set_addr(sqe, (struct sockaddr *)&s_a,
    (socklen_t)sizeof(s_a));
    io_uring_sqe_set_data64(sqe, idx);
    sqe->flags |= IOSQE_FIXED_FILE;

        return 0;
}
static int process_cqe(struct ctx *ctx, struct io_uring_cqe *cqe,
int fdidx, struct sockaddr_in s_a)
{
        if (cqe->user_data < BUFFERS){
                recycle_buffer(ctx, cqe->user_data);
                return 0;
        }
        else{
                return process_cqe_recv(ctx, cqe, fdidx, s_a);
        }
}

int main()
{
        int fd;
        struct ctx ctx;
        int ret;
        int port = 12345;
        int sockfd;
        struct io_uring_cqe *cqes[CQES];
        unsigned int count, i;

        memset(&ctx, 0, sizeof(ctx));
        ctx.af = AF_INET;
        ctx.buf_shift = BUF_SHIFT;

        struct sockaddr_in s_a;
        memset(&s_a, 0, sizeof(s_a));
    s_a.sin_family = AF_INET;
    s_a.sin_addr.s_addr = inet_addr("192.168.0.1");
    s_a.sin_port = htons(1234);


        uint16_t nport = port <= 0 ? 0 : htons(port);
        fd = socket(ctx.af, SOCK_DGRAM, 0);
        struct sockaddr_in addr;
        addr.sin_family = ctx.af;
    addr.sin_port = nport;
    addr.sin_addr.s_addr =  inet_addr("192.168.1.100");
        ret = bind(fd, (struct sockaddr *) &addr, sizeof(addr));
        sockfd = fd;

        if (setup_context(&ctx)) {
                close(sockfd);
```

```
                return 1;
        }

        ret = io_uring_register_files(&ctx.ring, &sockfd, 1);

        ret = add_recv(&ctx, 0);


        while (true) {
                ret = io_uring_submit_and_wait(&ctx.ring, 1);
                if (ret == -EINTR)
                        continue;
                if (ret < 0) {
                        fprintf(stderr, "submit and wait
   failed %d\n", ret);
                        break;
                }

                count = io_uring_peek_batch_cqe(&ctx.ring,
  &cqes[0], CQES);
                for (i = 0; i < count; i++) {
                        ret = process_cqe(&ctx, cqes[i], 0, s_a);
                }
                io_uring_cq_advance(&ctx.ring, count);
        }

cleanup:

        munmap(ctx.buf_ring, ctx.buf_ring_size);
        io_uring_queue_exit(&ctx.ring);
        close(sockfd);
        return ret;
}
```